Управление процессами машинного обучения (MLOps)

Учебное пособие

Рабочая редакция для 2026–2027 учебного года Дата актуализации: 20 октября 2026 года Направление подготовки: 09.03.02 «Информационные системы и технологии» Профиль: «Машинное обучение и искусственный интеллект» Семестр: 7

Аннотация

Пособие рассматривает MLOps как управление изменениями ML-системы на всём жизненном цикле: от контрактов и версионирования данных до проверяемого выпуска, наблюдаемости, управления рисками и разбора инцидентов. Материал построен вокруг сквозного учебного проекта прогноза задержки рейса и связывает теорию с демонстрационными сценариями лабораторных работ.

Пособие не заменяет лабораторный практикум. В лекционных главах объясняются основания инженерных решений, а отдельный практикум содержит пошаговые задания, критерии приёмки и правила индивидуальной защиты.

Для кого предназначено пособие

- для студентов направления 09.03.02, изучающих жизненный цикл ML-систем;
- для преподавателей, проводящих 16 лекционных занятий по 2 академических часа;
- для команд по 2–5 человек, развивающих единый репозиторий в течение семестра;
- для самостоятельной подготовки к ЛР16 и итоговому контролю.

Как работать с пособием

Каждая глава содержит цели, ключевые понятия, инженерные ограничения, вопросы аудитории и связь с практической работой. Сначала сформулируйте проверяемый инвариант, затем изучите инструмент, после чего выполните демонстрационный или отказной сценарий. Команды и версии из примеров проверяются перед семестром по официальной документации.

Сквозной учебный проект

Команды создают систему прогноза задержки рейса не менее чем на 15 минут за 60 минут до планового отправления. Данные разделяются по времени; признаки должны быть доступны в момент прогноза; фактическое качество считается по отложенным меткам. Итоговый release manifest связывает commit, версию данных, MLflow run/model version, container image, gates и решение о выпуске.

Содержание

1. Введение в MLOps
2. Контракты данных и предотвращение утечки
3. Версионирование данных и воспроизводимые вычисления
4. Эксперименты и базовая оценка модели
5. Тестирование данных, моделей и pipeline
6. Контейнеризация и управление средой
7. Сервис предсказаний и контракт API
8. Непрерывная интеграция и quality gates
9. Реестр моделей и управляемый выпуск
10. Наблюдаемость ML-сервиса
11. Мониторинг данных и качества модели
12. Согласованность признаков и Feature Store
13. Документация, безопасность и ответственное применение ML
14. Масштабирование, стоимость и обзор LLMOps
15. Интеграция и проверяемый выпуск ML-системы
16. Архитектурный разбор, отказы и ретроспектива MLOps-системы

Дополнение А. Навигатор демонстраций Дополнение Б. Демонстрационные и отказные сценарии из практических работ

Часть I. Основания и воспроизводимость

Глава 1. Введение в MLOps

Расширенный текст для преподавателя к презентации `1.Введение в MLOps.pdf` Учебный год: 2026-2027 Рекомендуемая продолжительность: 2 академических часа Формат: объяснение по слайдам, короткие вопросы аудитории и сквозной пример

Назначение лекции

Первая лекция должна сформировать у студентов правильную рамку курса. MLOps следует представить не как перечень модных инструментов и не как способ «быстро завернуть модель в Docker», а как систему инженерных и организационных практик, которая делает выпуск и эксплуатацию ML-систем управляемыми, проверяемыми и повторяемыми.

К концу занятия студент должен понимать, почему хорошей метрики в ноутбуке недостаточно, какие дополнительные объекты появляются в ML-системе по сравнению с обычным приложением, как выглядит непрерывный жизненный цикл модели и для чего в курсе будут использоваться GitHub, DVC, MLflow, Docker, GitHub Actions, Airflow, FastAPI, Kubernetes и средства наблюдаемости.

Учебные результаты

После лекции студент должен уметь:

- объяснить MLOps своими словами без перечисления инструментов;
- различать экспериментальную модель и производственную ML-систему;
- назвать код, данные, модель, конфигурацию и среду как отдельные версионируемые объекты;
- объяснить разницу между data drift и concept drift;
- описать полный цикл от постановки задачи до мониторинга и переобучения;
- различать CI, CD и CT;
- оценить примерный уровень зрелости ML-процесса;
- связать основные технологии курса с контролируемыми ими рисками;
- сформулировать, какие доказательства нужны для выпуска новой версии модели.

Сквозной пример лекции

На протяжении занятия удобно использовать один пример: сервис, который по часу вылета и дополнительным признакам оценивает вероятность задержки рейса. Data scientist обучил модель в ноутбуке и получил приемлемую метрику. Команда хочет превратить результат в HTTP-сервис, которым сможет пользоваться другое приложение.

Этот пример позволяет последовательно задавать вопросы:

1. Какой файл данных использовался и разрешено ли его применять?
2. Как воспроизвести подготовку данных и обучение?
3. Какая версия кода создала конкретную модель?
4. Как проверить модель до выпуска?
5. Как доставить ее в среду без расхождения зависимостей?
6. Как узнать, что сервис доступен и модель по-прежнему полезна?
7. Что делать при изменении потока рейсов, расписания или поведения пользователей?

Не обязательно подробно решать эти задачи на первой лекции. Важно показать, что каждая из них станет отдельной частью курса и лабораторных работ.

Рекомендуемый тайминг

Блок	Слайды	Время
Вход в тему и ожидания	1-2	7 минут
Определение и границы MLOps	3-5	16 минут
Провал без MLOps и первый жизненный цикл	6-7	11 минут
Проблемы развертывания и последствия	8-9	10 минут
DevOps DataOps и MLOps	10	6 минут
Production ML и виды дрейфа	11	10 минут
Непрерывный жизненный цикл	12	8 минут
Уровни зрелости	13	8 минут
Архитектурная схема курса	14	9 минут
Итоги и вопросы	15-16	5 минут

Тайминг рассчитан на 90 минут с короткими ответами студентов. Если занятие длится одну академическую пару без перерыва, часть обсуждений можно перенести в начало ЛР1.

Слайд 1 Введение в MLOps

Цель слайда

Открыть курс через практическую проблему: почему модель, показавшая хороший результат в ноутбуке, еще не является работающей системой.

Текст преподавателя

Сегодня мы начинаем курс об эксплуатации систем машинного обучения. Слово «эксплуатация» здесь принципиально. На курсах по машинному обучению основное внимание обычно уделяется выбору признаков, алгоритма, функции потерь и метрики. Все это необходимо, но ценность появляется только тогда, когда результат можно надежно передать пользователю или другой системе.

Представим, что мы обучили модель прогноза задержки рейса. В ноутбуке она показывает хорошую метрику. Мы сохраняем файл модели и говорим: «Работа завершена». С точки зрения MLOps работа только начинается. Необходимо знать, на каких данных обучена эта версия, какой код ее создал, какие зависимости использовались, прошла ли она проверки, как она будет запущена, что произойдет при ошибке и как мы заметим ухудшение результата.

MLOps рассматривает модель не как одиночный файл, а как часть изменяющейся программно-данной системы. В этой системе одновременно меняются код, данные, признаки, параметры, модель, инфраструктура и требования бизнеса. Поэтому нам нужен процесс, который связывает эти изменения и оставляет проверяемые доказательства.

В ходе курса каждая команда пройдет путь от репозитория и контракта данных до контейнера, CI, развертывания и мониторинга. Инструменты важны, но мы каждый раз будем начинать с вопроса: какой риск контролирует этот инструмент и какой артефакт подтверждает, что работа выполнена.

Вопрос аудитории

Попросите студентов поднять руки, если они когда-либо обучали модель в ноутбуке. Затем спросите: «Кто сможет через три месяца точно восстановить данные, зависимости и параметры того запуска?» Обычно этот вопрос сразу показывает проблему воспроизводимости.

Переход

Чтобы понять, какие задачи будет решать курс, сначала разберем структуру сегодняшней лекции.

Слайд 2 О чем эта лекция

Цель слайда

Дать студентам карту занятия и показать логику перехода от определения к архитектуре.

Текст преподавателя

Мы будем двигаться в шесть шагов. Сначала определим MLOps и договоримся о границах термина. Затем сравним его с DevOps и DataOps. После этого рассмотрим причины, по которым ML-системы ломаются в production, и соберем полный жизненный цикл модели.

Отдельно обсудим уровни зрелости. Они позволяют оценивать не размер компании и не количество установленных платформ, а степень управляемости процесса: что выполняется вручную, что автоматизировано, какие проверки обязательны и насколько быстро команда может безопасно внести изменение.

В завершение свяжем задачи с технологическим стеком курса. Важно не запоминать список названий. К концу лекции вы должны уметь объяснить, зачем в одном проекте могут одновременно понадобиться Git, DVC, MLflow, Docker, CI и мониторинг и почему эти средства не заменяют друг друга.

Мини-задание

Предложите студентам записать одно предложение: «MLOps нужен, чтобы...». В конце лекции попросите вернуться к записи и дополнить ее. Это дает быстрый способ увидеть изменение понимания.

Переход

Начнем с рабочего определения, которое будет использоваться во всем курсе.

Слайд 3 Определение MLOps

Цель слайда

Сформировать определение MLOps как практики управления жизненным циклом и культуры совместной ответственности.

Текст преподавателя

MLOps, или Machine Learning Operations, - это совокупность практик, процессов и технических средств для управляемой разработки, выпуска и эксплуатации систем машинного обучения. В определении важны три свойства: повторяемость, автоматизируемость и наблюдаемость.

Повторяемость означает, что другой участник команды может взять зафиксированные версии кода, данных, конфигурации и среды и получить ожидаемый результат. Абсолютная побитовая идентичность не всегда достижима из-за оборудования и недетерминированных операций, но причины различий должны быть известны и контролироваться.

Автоматизируемость означает, что регулярные и проверяемые действия оформлены как код или pipeline. Это не означает автоматизацию любого решения. Например, обучение и расчет метрик можно автоматизировать, а разрешение на выпуск модели с существенным риском может оставаться за ответственным человеком.

Наблюдаемость означает, что после развертывания мы получаем сигналы о состоянии сервиса, данных, предсказаний и, когда доступна разметка, о качестве модели. Без этого production превращается в черный ящик.

Вторая часть определения - культура взаимодействия. Data scientist не должен передавать модели «через стену» инженеру эксплуатации, а DevOps-инженер не должен отвечать за смысл метрики модели в одиночку. Ответственность разделяется: владелец продукта определяет ценность и допустимый риск, специалист по данным отвечает за корректность эксперимента, ML-инженер - за воспроизводимый pipeline и serving, инженеры платформы и эксплуатации - за надежную среду, а специалисты по безопасности и предметной области участвуют в проверках.

MLOps не равен одной платформе. Нельзя купить продукт и автоматически получить зрелый процесс. Платформа может упростить версионирование, запуск или мониторинг, но не определит за команду критерий качества, владельца риска и порядок реакции на инцидент.

Пример

Для модели задержки рейса повторяемость подтверждается связью «версия данных - commit - параметры - модель - отчет». Автоматизируемость проявляется в pipeline обучения и тестах. Наблюдаемость включает latency API, долю ошибок, распределение входных часов и изменение распределения предсказаний.

Вопрос аудитории

«Если модель запускается по расписанию каждую ночь, но никто не проверяет качество новых данных и нет правила выпуска, можно ли считать процесс зрелым MLOps?» Ожидаемый ответ: автоматический запуск сам по себе недостаточен.

Переход

Чтобы лучше увидеть границы MLOps, сравним его с более знакомым DevOps.

Слайд 4 Аналогия с DevOps

Цель слайда

Показать общее основание DevOps и MLOps и объяснить появление дополнительных управляемых объектов.

Текст преподавателя

DevOps появился как ответ на разрыв между разработкой и эксплуатацией программного обеспечения. Его ключевая идея - совместная ответственность, короткая обратная связь, автоматизация сборки и поставки, инфраструктура как код и наблюдаемость.

MLOps наследует эти идеи, но добавляет новые изменяемые объекты. В обычном сервисе поведение в основном определяется кодом и конфигурацией. В ML-системе поведение дополнительно зависит от обучающих данных, определения признаков, процедуры разбиения, параметров обучения и конкретного артефакта модели.

Когда на слайде данные и модели называются first-class citizens, имеется в виду, что к ним применяются инженерные требования, сопоставимые с требованиями к коду. Мы должны знать их владельца, происхождение, версию, допустимые преобразования и связь с выпуском. Недостаточно положить файл `model.pkl` рядом с приложением.

Еще одно отличие - разделение логики обучения и логики инференса. Обучение может происходить редко и требовать больших ресурсов, а inference выполняется постоянно и должен отвечать быстро. При этом преобразования признаков должны быть согласованы между двумя средами. Несогласованность приводит к training-serving skew: модель обучалась на данных, подготовленных одним способом, а в сервисе получает иначе преобразованные признаки.

Доска или схема

Нарисуйте пять связанных версий:

`данные -> код подготовки -> параметры обучения -> модель -> сервис.`

Затем измените только один объект, например данные, и спросите, какие результаты потребуется пересчитать и перепроверить.

Переход

Теперь можно конкретно перечислить, почему перенос DevOps-практик без учета данных и моделей оставляет существенные риски.

Слайд 5 Почему MLOps не просто DevOps для ML

Цель слайда

Разобрать три свойства ML-систем: зависимость от данных, деградацию во времени и сложность воспроизводимости.

Текст преподавателя

Первое отличие: данные являются частью поведения системы. Даже если код обучения не изменился, новая выборка может привести к другой модели. Ошибка в единице измерения, пропущенная категория или изменение источника способны незаметно ухудшить результат. Поэтому данные нуждаются в контракте, проверках качества и версии.

Второе отличие: модель может деградировать без изменения программы. Пусть наш сервис продолжает возвращать HTTP 200 и укладывается в 50 миллисекунд. Технически он доступен. Но авиакомпания изменила расписание, аэропорт ввел новые ограничения, а распределение рейсов стало другим. Модель

продолжает работать как программа, но ее ответы могут стать менее полезными.

Третье отличие: эксперимент зависит от большого числа входов. Помимо commit, нужны версия данных, способ разбиения, seed, параметры, версия библиотеки, аппаратная среда и порядок преобразований. Если эти сведения остались только в памяти автора ноутбука, результат нельзя независимо проверить.

Добавьте четвертое практическое отличие: качество часто измеряется с задержкой. Истинную задержку рейса мы узнаем после его прибытия. Для кредитного риска или отказа оборудования задержка может быть еще больше. Поэтому мониторинг должен сочетать ранние косвенные сигналы - схему, диапазоны, drift, долю неизвестных категорий - и отложенную фактическую метрику.

Наконец, у модели есть риск неправильного использования. Модель может быть приемлемой для информационной подсказки и неприемлемой для автоматического решения с серьезными последствиями. Карточка модели и правила выпуска должны описывать не только среднюю метрику, но и границы применения.

Вопрос аудитории

«Что произойдет, если столбец времени вылета, ранее измерявшийся в часах, начнет поступать в минутах, но тип останется числовым?» Подведите к мысли, что техническая схема типов недостаточна без предметных диапазонов и единиц измерения.

Переход

Рассмотрим типичную ситуацию, когда эксперимент успешен, но передать его в эксплуатацию не удается.

Слайд 6 Пример провала без MLOps

Цель слайда

Разобрать причинную цепочку провала, а не ограничиться формулировкой «не совпало окружение».

Текст преподавателя

Фраза «у меня работает» обычно означает только то, что код работал в конкретном окружении автора и на доступных ему файлах. Для команды этого недостаточно.

Разберем сценарий. Data scientist обучил модель в ноутбуке. Набор данных лежит в локальной папке под именем `final_v7_really_final.csv`. Часть очистки выполнена вручную. Версии библиотек не зафиксированы. Файл модели отправлен коллеге. На сервере установлена другая версия библиотеки, порядок категорий изменился, а одно поле в запросе называется иначе. Сервис либо не загружает модель, либо, что хуже, запускается и возвращает неверные ответы.

Проблема здесь не в одном неудачном сотруднике. Система допускает неявные действия и не требует доказательств. MLOps-подход меняет процесс:

1. Код и конфигурация находятся в репозитории.
2. Происхождение и версия данных зафиксированы.
3. Подготовка данных выполняется скриптом или pipeline.
4. Зависимости и версия Python определены.
5. Модель сохраняется вместе с метаданными и схемой входа.
6. Автоматические тесты проверяют данные, модель и API.
7. Контейнер фиксирует среду запуска.
8. CI связывает артефакты с commit и сохраняет отчет.

Обратите внимание: контейнер решает только часть проблемы. Он не объясняет происхождение данных, не предотвращает утечку целевой переменной и не решает, достаточно ли хороша модель. Поэтому фраза «давайте используем Docker» не является полным MLOps-решением.

Мини-разбор

Попросите студентов разделить причины провала на четыре группы: данные, код, среда, процесс. Запишите ответы в четыре колонки. Это подготовит их к жизненному циклу.

Переход

Чтобы такие риски контролировались системно, работу нужно видеть как связанный жизненный цикл.

Слайд 7 Жизненный цикл ML проекта

Цель слайда

Представить базовый цикл и показать, что обратная связь появляется уже после первого выпуска.

Текст преподавателя

На слайде показан упрощенный цикл из четырех этапов: сбор и подготовка данных, разработка модели, тестирование и развертывание. Это удобная первая схема, но важно сразу отметить: этапы не являются независимыми коробками и редко проходят строго один раз.

На этапе данных команда определяет источник, права использования, схему, целевую переменную и правила разбиения. Ошибка здесь часто не исправляется выбором более сложного алгоритма.

На этапе разработки создается не только обученная модель. Появляется воспроизводимый код преобразований, конфигурация, журнал эксперимента, базовая модель для сравнения и описание ограничений.

Тестирование включает несколько уровней. Проверяется код, контракт данных, отсутствие утечки, диапазон метрик, сериализация и загрузка модели, схема API, производительность и безопасность. Высокая offline-метрика не отменяет остальных проверок.

Развертывание означает управляемую доставку конкретной проверенной версии. Команда должна уметь ответить, какой commit, данные и модель сейчас обслуживают запросы и как вернуться к предыдущей версии.

После развертывания цикл не заканчивается. Появляется обратная связь: технические метрики, новые данные, фактические исходы и замечания пользователей. На следующем слайде жизненного цикла этот контур будет расширен.

Вопрос аудитории

«На каком этапе следует впервые думать о мониторинге?» Правильная мысль: требования к мониторингу формулируются до развертывания, потому что для них нужны идентификаторы версий, логирование и данные обратной связи.

Переход

Между экспериментом и работающим сервисом команда сталкивается с повторяющимся набором практических препятствий.

Слайд 8 Проблемы при развертывании

Цель слайда

Связать перечисленные проблемы с конкретными инженерными мерами.

Текст преподавателя

Отсутствие стандартизированного процесса означает, что каждый выпуск выполняется по памяти отдельного человека. Такой процесс трудно проверить, повторить и улучшить. Решение начинается с общего репозитория, критериев готовности, pull request и pipeline.

Сложность окружения возникает, когда локальная среда автора отличается от тестовой или production-среды. Версии Python и библиотек, системные зависимости, архитектура процессора и переменные окружения могут влиять на результат. Фиксация зависимостей и контейнеризация уменьшают вариативность, но их также необходимо тестировать.

Ручная настройка параметров создает скрытое состояние. Если важный порог изменен через консоль и нигде не зафиксирован, следующий выпуск не воспроизведет систему. Поэтому конфигурация должна быть явной, версионируемой и отделенной от секретов.

Отсутствие мониторинга лишает команду обратной связи. Нельзя управлять тем, о состоянии чего нет сигналов. Но мониторинг - это не просто график загрузки CPU. Для ML-системы нужны технические, данные, модельные и продуктовые показатели.

Нехватка ресурсов для поддержки часто является следствием избыточной сложности. Команда внедряет Kubernetes, Airflow, feature store и несколько систем мониторинга, хотя у нее одна модель и нет владельца дежурства. MLOps требует соразмерности: выбирайте минимальный набор компонентов, который контролирует реальные риски и может поддерживаться командой.

Таблица на доске

Проблема	Контроль
неясный порядок выпуска	checklist, PR, CI и владелец решения
разные окружения	зафиксированные зависимости и контейнер
скрытые параметры	конфигурация как код и журнал эксперимента
неизвестное состояние	метрики, логи, трассировки и ML-мониторинг
дорогая платформа	поэтапное развитие и оценка стоимости поддержки

Переход

Если эти проблемы не устранять, они проявляются не только техническими ошибками, но и вполне измеримыми последствиями для проекта.

Слайд 9 Последствия

Цель слайда

Показать связь инженерных дефектов с качеством, сроками, затратами и доверием.

Текст преподавателя

Снижение точности - только одно из последствий. Часто модель не становится хуже математически, но перестает соответствовать новой среде. Без версии данных и отложенной оценки команда не знает, когда это произошло.

Увеличение времени внедрения возникает из-за ручных передач между ролями. Каждая передача сопровождается поиском файла, восстановлением окружения и повторным согласованием. Автоматизация не просто ускоряет отдельную команду; она уменьшает число неоднозначных переходов.

Рост затрат проявляется в повторных вычислениях, простоях специалистов, избыточной инфраструктуре и стоимости ошибок. Сюда же относятся облачные ресурсы, которые продолжают работать без контроля, и модели, которые переобучаются без необходимости.

Снижение доверия особенно опасно. Если пользователи несколько раз получают необъяснимый результат, они начинают обходить систему, даже когда модель исправна. Доверие поддерживается не обещанием идеальной точности, а прозрачными ограничениями, наблюдаемостью, объяснимым процессом выпуска и способностью быстро реагировать.

Вопрос аудитории

«Какое последствие труднее всего восстановить после инцидента?» Обсудите, что техническую версию можно откатить, а доверие пользователя и владельца процесса восстанавливается дольше.

Переход

Чтобы понять происхождение практик MLOps, сравним три взаимосвязанных направления.

Слайд 10 Эволюция от DevOps к MLOps

Цель слайда

Различить зоны внимания DevOps, DataOps и MLOps без представления их как взаимоисключающих этапов.

Текст преподавателя

DevOps фокусируется на надежной разработке и эксплуатации программного обеспечения: автоматическая сборка, тестирование, доставка, инфраструктура, наблюдаемость и обратная связь.

DataOps переносит сходные идеи на потоки данных: качество, происхождение, оркестрацию, проверку схем, управление изменениями и доступом. Его предмет - надежная доставка данных потребителям.

MLOps объединяет требования программной системы, данных и модели. Ему нужны DevOps-практики для кода и сервиса, DataOps-практики для надежных данных и дополнительные механизмы для экспериментов,

моделей, оценки и переобучения.

Не стоит воспринимать схему как историческую лестницу, в которой DataOps обязательно появился после DevOps, а MLOps все заменил. В реальной организации эти практики пересекаются. Один и тот же pipeline может включать подготовку данных, обучение модели, сборку сервиса и выпуск контейнера.

Введем различие CI, CD и CT. Continuous Integration проверяет изменение кода, конфигурации или pipeline и дает быструю обратную связь. Continuous Delivery или Deployment отвечает за управляемую доставку проверенного артефакта в среду. Continuous Training автоматически инициирует повторное обучение при появлении разрешенных данных, по расписанию или по сигналу. CT не означает, что новая модель обязана автоматически попасть в production: между обучением и выпуском сохраняются quality gates и решение владельца риска.

Проверочный вопрос

«Pipeline каждую неделю обучает модель и сохраняет файл, но не проводит сравнение с текущей версией. Это CT?» Ответ: технически повторное обучение автоматизировано, но процесс выпуска неполон и небезопасен.

Переход

Теперь разберем четыре проблемы, характерные именно для production ML.

Слайд 11 Проблемы Production ML

Цель слайда

Дать точные определения data drift, concept drift, воспроизводимости и масштабируемости.

Текст преподавателя

Data drift - изменение распределения входных данных по сравнению с выбранной reference-выборкой. Например, раньше большинство запросов относилось к дневным рейсам, а теперь выросла доля ночных. Drift не равен автоматически снижению качества. Он является сигналом: условия изменились, необходимо проверить влияние.

Concept drift - изменение зависимости между входом и целевой переменной. Условно меняется распределение `P(Y|X)`. При одинаковом часе вылета вероятность задержки могла измениться из-за новой организации воздушного движения. Concept drift труднее обнаружить без фактических исходов, потому что входные признаки могут выглядеть привычно, а связь с результатом уже другая.

Полезно различить еще изменение признаков из-за сбоя данных. Если `DepHour` внезапно измеряется в минутах, это не естественный drift, а нарушение контракта или data quality incident. Его следует блокировать проверкой диапазона до обращения к модели.

Воспроизводимость означает способность восстановить процедуру и объяснить результат. Для ML требуется связать:

- версию кода;
- версию и происхождение данных;
- конфигурацию и гиперпараметры;
- среду и зависимости;
- seed и известные источники недетерминизма;
- метрики, отчет и итоговый артефакт модели.

Масштабируемость имеет несколько измерений. Это не только число запросов. Нужно масштабировать объем данных, число экспериментов, количество моделей, число команд и процесс управления. Система, способная обслужить миллион запросов, но требующая ручного поиска версии модели, технически производительна, но операционно незрела.

Мини-упражнение

Прочитайте три ситуации и попросите определить тип проблемы:

1. В запросах выросла доля выходных дней - возможный data drift.
2. При тех же признаках задержки стали чаще из-за новых правил аэропорта - возможный concept drift.

3. Поле часа содержит значения до 1380 - нарушение качества или единицы измерения.

Подчеркните, что диагноз требует контекста и проверки, а не только статистического теста.

Переход

Чтобы реагировать на эти изменения, жизненный цикл должен быть замкнутым и включать эксплуатационную обратную связь.

Слайд 12 Расширенный жизненный цикл ML проекта

Цель слайда

Показать непрерывный цикл из данных, разработки, обучения, тестирования, выпуска, мониторинга и переобучения.

Текст преподавателя

На этом слайде схема становится полной. После сбора появляется отдельная подготовка данных. Это важное разделение: исходный источник не равен готовой обучающей выборке. Подготовка включает проверку контракта, очистку, построение признаков и корректное разбиение.

Разработка модели включает постановку baseline и экспериментирование. Baseline нужен, чтобы сложная модель доказывала дополнительную ценность. Если простое правило работает не хуже, сложность не оправдана.

Обучение в зрелом процессе является воспроизводимым запуском, а не серией скрытых ячеек ноутбука. Результаты регистрируются: параметры, метрики, артефакты и связь с входами.

Тестирование выполняется до выпуска и включает качество данных, код, модель, интеграцию и нефункциональные свойства. Для моделей высокого риска добавляются анализ по группам, устойчивость, безопасность и согласование ответственного лица.

Развертывание выпускает неизменяемый артефакт. Одна и та же проверенная версия должна переходить между средами, а не пересобираться заново с плавающими зависимостями.

Мониторинг собирает несколько слоев сигналов:

1. Сервис: доступность, ошибки, latency и ресурсы.
2. Данные: схема, пропуски, диапазоны и неизвестные категории.
3. Модель: распределения предсказаний, drift и отложенное качество.
4. Продукт: полезность решения и бизнес-показатель.

Переобучение начинается не просто по факту drift. Сначала проверяется причина, качество новых данных и ожидаемая ценность. Новая модель сравнивается с действующей и проходит те же gates. Таким образом, цикл является непрерывным, но управляемым.

Связь с лабораторными

Покажите студентам, что лабораторные курса проходят этот цикл постепенно: репозиторий, контракт и версия данных, эксперименты, контейнер, CI, orchestration, serving, развертывание, мониторинг и итоговый проект.

Переход

Организации выполняют этот цикл с разной степенью ручного труда и контроля. Для обсуждения используют модель зрелости.

Слайд 13 Уровни зрелости MLOps

Цель слайда

Объяснить уровни 0-2 как учебную модель оценки процесса, а не универсальный сертификат.

Текст преподавателя

Уровни на слайде - полезная упрощенная модель. Разные источники могут использовать другую нумерацию и детализацию, поэтому важнее признаки уровня, чем номер.

На уровне 0 обучение и выпуск в основном выполняются вручную. Ноутбук, локальные данные и передача файла модели могут быть достаточны для исследования или одноразового прототипа. Проблема начинается, когда такой процесс используется регулярно или влияет на пользователей. Результат зависит от отдельных людей, обновляется медленно и плохо воспроизводится.

На уровне 1 автоматизируется pipeline обучения. Появляются повторяемая подготовка данных, запуск обучения, регистрация метрик и модели. Это уже существенно снижает риск, но развертывание, одобрение и мониторинг могут оставаться ручными. Команда умеет регулярно получать кандидата модели, но еще не полностью управляет его жизнью после обучения.

На уровне 2 связываются CI, CD и CT. Изменения проверяются автоматически, pipeline может запускаться по разрешенному событию, кандидат сравнивается с действующей версией, выпуск проходит gates, а мониторинг возвращает сигналы в цикл. Возможны canary- или A/B-сценарии, но они применяются только там, где эксперимент этически и технически допустим.

Высокий уровень не означает, что каждое решение автоматическое. Зрелость - это предсказуемость и контроль. В регулируемой области ручное одобрение может быть признаком зрелого процесса, если критерии, ответственный и доказательства четко определены.

Также зрелость может быть неоднородной. Команда может иметь отличную контейнеризацию и CI, но не знать происхождение данных. Тогда нельзя поставить высокий итоговый уровень только за красивую инфраструктуру.

Быстрая самооценка

Предложите ситуацию: код хранится в Git, модель обучается вручную, Docker-образ собирается автоматически, мониторинга качества нет. Спросите, какие элементы уже зрелые и какое одно улучшение даст наибольший эффект. Ожидаемые ответы: воспроизводимый pipeline и связь данных с моделью.

Переход

Посмотрим, какие компоненты поддерживают зрелый процесс и какую ответственность несет каждый из них.

Слайд 14 Архитектурная схема

Цель слайда

Дать карту технологического стека курса и показать связи между компонентами.

Текст преподавателя

Начнем с версионирования. Git хранит код, конфигурацию и историю решений. GitHub организует Issues, pull request, review, CI и релизы. Крупные данные и модели не следует механически помещать в Git.

DVC связывает версии данных и pipeline с Git-историей. Он хранит метаданные в репозитории, а содержимое может находиться во внешнем хранилище. DVC не заменяет контракт данных: версия отвечает на вопрос «какие байты», контракт - «каким требованиям они должны соответствовать».

MLflow Tracking сохраняет параметры, метрики и артефакты экспериментов. Model Registry помогает управлять версиями моделей и их метаданными. MLflow не заменяет Git или DVC, потому что каждый инструмент отвечает за другой слой происхождения.

GitHub Actions реализует CI и учебные элементы CD. Workflow выполняет lint, тесты, проверку данных, короткое обучение, сборку контейнера и публикацию отчетов. Важна не сама автоматизация, а одинаковые обязательные проверки для всех изменений.

Airflow или другой оркестратор задает порядок и состояние долгоживущих задач pipeline: получить данные, проверить, подготовить, обучить, оценить и зарегистрировать модель. Оркестратор не должен содержать всю бизнес-логику внутри DAG и не является хранилищем больших данных.

FastAPI дает типизированный HTTP-интерфейс инференса. Pydantic проверяет структуру запросов и ответов, Uvicorn запускает приложение. Docker упаковывает сервис и зависимости в OCI-образ.

Kubernetes управляет развертыванием контейнера: количеством экземпляров, обновлением, probes, ресурсами и конфигурацией. Для небольшого проекта Kubernetes может быть избыточен, но в курсе он позволяет изучить стандартные контракты эксплуатации.

Prometheus собирает метрики, Grafana помогает их анализировать и настраивать панели, OpenTelemetry стандартизирует инструментирование. Evidently или эквивалент используется для отчетов о

распределениях и drift. Drift-отчет не принимает решение о переобучении сам: нужны пороги, контекст, владелец и runbook.

Свяжите компоненты в цепочку:

`issue -> commit -> CI -> данные и experiment -> модель -> образ -> deployment -> telemetry -> решение о следующем изменении.`

Каждая стрелка должна оставлять идентификатор или артефакт. Если команда не может от работающего endpoint вернуться к версии модели, commit и данным, трассируемость нарушена.

Вопросы по архитектуре

- Где хранится пароль к внешнему хранилищу? Не в Git; в secrets-механизме среды.
- Что произойдет, если проверка данных не пройдет? Pipeline останавливается до обучения.
- Что является единицей выпуска? Проверенный неизменяемый образ плюс идентифицированная версия модели и конфигурации.
- Можно ли заменить конкретный инструмент? Да, если сохраняются контракты, доказательства и контролируемые риски.

Переход

Теперь соберем основные идеи лекции в несколько проверяемых выводов.

Слайд 15 Итоги

Цель слайда

Закрепить концептуальную модель и подготовить переход к ЛР1.

Текст преподавателя

Первый вывод: MLOps нужен не ради инфраструктуры, а ради управляемого изменения ML-системы. Он связывает людей, правила и технические средства.

Второй вывод: объектами управления становятся не только код, но также данные, признаки, конфигурация, эксперимент, модель, образ и развертывание.

Третий вывод: production ML может ухудшаться без изменения кода. Поэтому после выпуска нужны наблюдаемость, фактическая обратная связь и процедура реакции.

Четвертый вывод: жизненный цикл непрерывен. Мониторинг не является последним этапом; он возвращает информацию в постановку задачи, данные и следующий кандидат модели.

Пятый вывод: автоматизация должна сопровождаться gates и ответственностью. Быстро автоматически выпустить непроверенную модель - не зрелость, а ускорение риска.

Вернитесь к предложению, которое студенты записали в начале. Предложите дополнить его. Хорошая итоговая формулировка может звучать так: «MLOps нужен, чтобы команда могла воспроизводимо, безопасно и наблюдаемо изменять ML-систему от данных до эксплуатации».

Проверка понимания

Попросите ответить без конспекта:

1. Назовите три объекта ML-системы помимо кода.
2. Чем data drift отличается от concept drift?
3. Почему HTTP 200 не доказывает качество модели?
4. Что должно связывать работающий endpoint с исходным экспериментом?
5. Какой риск первым контролирует ЛР1?

Переход к лабораторной работе

В ЛР1 команда создаст основу всей последующей трассируемости: общий GitHub-репозиторий, правила веток и pull request, воспроизводимое окружение, базовый CI и фиксацию границ проекта. Без этой основы дальнейшие данные, модели и deployment невозможно надежно связать между собой.

Слайд 16 Контакты

Цель слайда

Завершить занятие организационными договоренностями и дать студентам понятный следующий шаг.

Текст преподавателя

Покажите официальные каналы курса и отдельно проговорите их назначение. СКИФ используется для материалов, заданий и официальных сроков. Групповой канал подходит для организационных объявлений и общих вопросов. Вопросы, содержащие персональные данные, оценки или секреты репозитория, обсуждаются только в согласованном приватном канале.

Перед следующим занятием студентам необходимо:

1. проверить доступ к СКИФ и командному каналу;
2. создать или проверить GitHub-аккаунт;
3. установить Git и Python версии, указанной в стартовом проекте;
4. ознакомиться с правилами команд по 2-5 человек;
5. прочитать постановку ЛР1 и подготовить вопросы по репозиторию и ролям.

Завершите лекцию фразой: «На следующем шаге мы не будем обучать более сложную модель. Мы создадим процесс, в котором результат любого участника сможет проверить и воспроизвести другой участник команды».

Дополнительный разбор при наличии времени

Кейс выпуска новой модели

Команда получила новую неделю данных и обучила кандидата. Offline-метрика выросла с 0,78 до 0,80. Предложите студентам решить, достаточно ли этого для выпуска.

Ожидаемые уточнения:

- одинаково ли сформированы train и test;
- нет ли утечки и дубликатов;
- на каких сегментах изменилась метрика;
- статистически и практически значима ли разница;
- прошли ли контрактные и нагрузочные тесты;
- совместима ли схема входа;
- какова стоимость ошибки;
- предусмотрены ли canary, наблюдение и rollback;
- кто принимает решение о выпуске.

Вывод: одна агрегированная метрика является частью доказательств, но не полным решением о выпуске.

Кейс работающего но бесполезного сервиса

API доступен 99,99 процента времени, latency ниже SLO, ошибок почти нет. Однако пользователи перестали следовать рекомендациям, а отложенная точность снизилась. Спросите, можно ли считать систему надежной.

Подведите к разделению service reliability и model or product effectiveness. Для ML-системы необходимы оба слоя. Технически безупречный сервис может стабильно выдавать бесполезные ответы.

Типичные заблуждения и ответы преподавателя

MLOps нужен только крупным компаниям

Малой команде не нужна сложная платформа, но особенно нужны простые правила воспроизводимости, версии, тесты и ответственный владелец. Масштаб реализации зависит от риска и частоты изменений.

Docker решает воспроизводимость

Docker фиксирует значительную часть runtime-среды, но не происхождение данных, логику ручной очистки, параметры эксперимента и допустимость выпуска.

Если модель точная то мониторинг не нужен

Метрика относится к конкретной выборке и моменту времени. Production-поток меняется, а истинные ответы могут приходить с задержкой.

Drift означает что модель надо немедленно переобучить

Drift является сигналом расследования. Нужно оценить причину, влияние на качество и стоимость реакции. Автоматическое переобучение на поврежденных данных способно ухудшить систему.

Чем больше инструментов тем выше зрелость

Зрелость определяется контролем процесса, а не количеством платформ. Неиспользуемый registry или панель без владельца алерта создают стоимость, но не надежность.

MLOps полностью устраняет ручные решения

MLOps автоматизирует воспроизводимые действия и делает решения проверяемыми. Ручное одобрение может быть обязательным quality gate, особенно при существенном риске.

Контрольные вопросы после лекции

1. Дайте определение MLOps без названий конкретных продуктов.
2. Какие дополнительные объекты по сравнению с обычным приложением необходимо версионировать в ML-системе?
3. Почему одинаковый код обучения может создать разные модели?
4. Чем техническая доступность сервиса отличается от качества модели?
5. Приведите пример data drift, concept drift и нарушения качества данных.
6. Что требуется для воспроизводимости эксперимента кроме commit?
7. Чем CI, CD и CT отличаются друг от друга?
8. Почему CT не обязательно означает автоматический выпуск?
9. Какие четыре слоя мониторинга нужны ML-системе?
10. Почему DVC, MLflow и Git не заменяют друг друга?
11. Что означает training-serving skew?
12. Как модель зрелости помогает выбирать следующий шаг автоматизации?
13. Какие доказательства должны сопровождать новую версию модели?
14. Когда сложная MLOps-платформа может ухудшить работу команды?
15. Как ЛР1 связана с последующими этапами жизненного цикла?

Домашнее задание перед ЛР1

Студент выбирает знакомый ML-сценарий и готовит краткую карту:

- пользователь и решение, которое поддерживает модель;
- источник данных и целевая переменная;
- одна offline-метрика и одно ограничение этой метрики;
- два риска после развертывания;
- технический и ML-сигнал мониторинга;
- объекты, версии которых необходимо связать;
- минимальный rollback-сценарий.

Объем - до одной страницы. Задание можно использовать как вход для обсуждения границ командного проекта.

Источники для преподавателя

- исходная презентация `MLOps/Лекции/1.Введение в MLOps.pdf`;
- технологический справочник `MLOps/2026-2027/Технологический стек MLOps 2026-2027.md`;
- Google Cloud Architecture Center, MLOps continuous delivery and automation pipelines in machine learning;
- Google, Rules of Machine Learning;
- Microsoft Azure Architecture Center, Machine learning operations maturity model;
- документация GitHub Actions, DVC, MLflow, Docker, Apache Airflow, Kubernetes, Prometheus, OpenTelemetry и Evidently;
- NIST AI Risk Management Framework как рамка управления риском AI-систем.

Глава 2. Контракты данных и предотвращение утечки

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР2 «Контракт данных и предотвращение утечки»

Цель лекции

Показать, почему данные в ML-системе должны иметь явный договор между источником и потребителем, как определить момент доступности признака и как спроектировать хронологическое разбиение без утечки. После лекции студент должен уметь отличать data card от исполнимого контракта, сформулировать prediction timestamp, проверить point-in-time корректность и прочитать validation report как инженерное доказательство.

План и распределение времени

Время	Содержание	Действие преподавателя
0–10 мин	Связь с ЛР1 и постановка кейса	Восстановить путь от репозитория к данным и сформулировать прогноз рейса
10–25 мин	Данные как интерфейс	Разобрать владельца, производителя, потребителя и контракт
25–40 мин	Схема и семантика	Сравнить техническую и предметную корректность
40–55 мин	Момент прогноза и утечки	Построить временную шкалу признака и цели
55–70 мин	Хронологическое разбиение	Разобрать train, validation, test и преобразования
70–82 мин	Validation report и отказ	Показать положительный и отрицательный сценарии
82–90 мин	Инструктаж к ЛР2	Уточнить командные артефакты и индивидуальную защиту

1 Вход в тему

Начните с вопроса: «Если CSV успешно читается pandas, означает ли это, что модель можно безопасно обучать?» Ожидаемый ответ — нет. Файл может иметь правильные заголовки и типы, но содержать данные из будущего, иной смысл единиц, изменившееся кодирование категории или незаконно опубликованные сведения.

Свяжите тему с ЛР1. Репозиторий и воспроизводимая среда отвечают на вопрос «какой код и в каком окружении запущен». Контракт данных отвечает на вопросы «какие именно данные допустимы», «когда они были доступны» и «почему потребитель может им доверять». Без обеих частей воспроизводимость неполна.

Сформулируйте учебный кейс: за 60 минут до планового отправления система должна оценить, будет ли задержка не менее 15 минут. С этого момента любое поле нужно оценивать не по удобству для модели, а по доступности в конкретный момент времени.

2 Данные как инженерный интерфейс

Контракт данных — версионируемое соглашение между производителем и потребителем. Он описывает структуру, смысл, ограничения, временные свойства и реакцию системы на нарушение. Подчеркните, что контракт не гарантирует истинность каждой строки. Он делает ожидания явными и автоматически проверяемыми.

Разберите четыре стороны:

- источник формирует данные и сообщает об изменениях;
- владелец отвечает за разрешённое использование и качество;
- pipeline проверяет и преобразует данные;
- модель или сервис потребляет признаки в согласованном виде.

Полезный вопрос аудитории: «Кто должен исправить ошибку, если столбец исчез?» Ответ зависит от договора. Pipeline обязан быстро и понятно остановиться; исправление источника или адаптация

потребителя — отдельное управляемое решение.

3 Data card и машинный контракт

Data card предназначена человеку. Она объясняет происхождение, назначение, лицензию, ограничения, чувствительные поля, известные смещения и порядок обновления. Машинный контракт предназначен автоматической проверке: обязательные столбцы, типы, nullability, диапазоны, категории, уникальность, формат времени и политика лишних полей.

Покажите, что документы дополняют друг друга. Правило `DepHour <= 23` исполнимо, но без описания единицы и часового пояса остаётся двусмысленным. Фраза «время отправления по местному времени аэропорта» понятна, но без проверки не остановит ошибочный pipeline.

Обсудите версионирование. Изменение описания — patch; совместимое необязательное поле — minor; изменение типа, имени или смысла обязательного поля — major. Это учебное соглашение, а не универсальный закон, но команда должна применять его последовательно.

4 Схема не равна семантике

Разделите проверки на уровни:

1. Структура: столбец присутствует, тип преобразуем, формат даты корректен.
2. Значения: диапазон, допустимая категория, пропуск, дубликат.
3. Взаимосвязи: фактическое время не раньше планового без объяснимой причины, аэропорты вылета и прилёта не совпадают для обычного рейса.
4. Распределения: доля пропусков или класс задержки резко изменились.
5. Временная корректность: значение было известно к prediction timestamp.

Сделайте акцент: первые два уровня часто дают жёсткий отказ, распределительные проверки могут начинаться с предупреждения. Команда должна объяснять выбранную реакцию, а не превращать любое отличие в падение системы.

5 Момент прогноза и временная линия

Нарисуйте на доске линию: планирование рейса → момент прогноза T−60 → отправление → прибытие → появление `ArrDelay`. Для каждого признака нужны два времени:

- event time — к какому событию относится значение;
- available time — когда значение фактически стало доступно системе.

Признак корректен, если available time не позже prediction timestamp. Например, плановый час отправления обычно известен заранее. Фактическое время отправления, итоговая задержка прибытия и статистика, рассчитанная с использованием будущих рейсов, недоступны.

Объясните разновидности утечки:

- target leakage — признак прямо или косвенно содержит ответ;
- look-ahead leakage — преобразование использует будущие наблюдения;
- split leakage — одна сущность или связанная запись попала в разные части;
- preprocessing leakage — нормализация или словарь категорий обучены на всех данных;
- operational leakage — offline-признак нельзя получить в реальном сервисе к требуемому времени.

Мини-задание: предложите поля `ScheduledDepTime`, `ActualDepTime`, `WeatherObservedAt`, `ArrivalDelay`, «средняя задержка аэропорта за сутки». Студенты должны задать уточняющие вопросы о времени доступности. Правильный ответ не всегда «можно» или «нельзя»; иногда нужно изменить окно агрегации.

6 Хронологическое разбиение

Для рейсов случайное перемешивание создаёт оптимистичную оценку, потому что будущее смешивается с прошлым. Используется последовательность train → validation → test. Train обучает параметры модели и преобразований. Validation нужен для выбора модели, признаков и порога. Test имитирует окончательную проверку на будущем периоде и не должен многократно влиять на решения.

На доске зафиксируйте инвариант:

```
max(train_time) < min(validation_time) <= max(validation_time) < min(test_time)
```

Границы задаются календарными датами и числами строк. Если распределение классов неудобно, запрещено решать проблему переносом отдельных будущих строк в train. Нужно выбрать более длинное окно, изменить постановку или явно описать ограничение.

Все обучаемые преобразования fit только на train. На validation и test применяется transform. Это относится к заполнению пропусков, нормализации, отбору признаков, target encoding и формированию словарей.

7 Validation report как доказательство

Отчёт должен позволять другому человеку понять, что именно проверено. Минимальные поля: версия контракта, логический идентификатор набора, путь или URI, SHA-256, число строк и столбцов, границы периодов, результат каждого правила и общий статус.

Покажите два запуска. Первый проходит и завершается кодом 0. Во втором дата имеет неверный формат или строки переставлены; отчёт называет правило, а процесс возвращает ненулевой код. Отдельно проговорите: перехват исключения с последующим `exit 0` делает CI недостоверным.

SHA-256 связывает отчёт с байтами файла, но не доказывает качество, лицензию или отсутствие смещения. Это идентификатор, а не сертификат доверия.

8 Командная работа и индивидуальная ответственность

В команде 2–5 человек можно распределить роли data steward, contract engineer, quality engineer, leakage reviewer и documentation reviewer. Роль не является доказательством. Нужны собственный commit или review, воспроизводимый запуск и объяснение решения.

На защите каждый студент запускает валидатор, показывает SHA-256 и границы периодов, создаёт одно контролируемое нарушение и объясняет доступность выбранного признака за 60 минут до отправления. Одинаковый текст evidence card у нескольких участников не принимается.

9 Вопросы для проверки понимания

1. Почему корректный тип столбца не доказывает корректный смысл?
2. Чем data card отличается от `data_contract.json`?
3. Почему `ArrDelay` допустим для построения цели, но запрещён как признак?
4. Чем event time отличается от available time?
5. Почему нельзя fit нормализатор на объединённых train и test?
6. Что доказывает SHA-256 и чего он не доказывает?
7. Когда изменение контракта требует major-версии?
8. Почему test-период нельзя использовать для регулярного выбора гиперпараметров?

10 Завершение лекции

Верните аудиторию к исходному вопросу. Файл становится пригодным для ML-системы не тогда, когда он просто читается, а когда его происхождение, назначение, схема, временная доступность и правила отказа зафиксированы и воспроизводимо проверяются. ЛР2 должна создать этот барьер качества; ЛР3 затем свяжет проверенный набор с Git-состоянием через DVC.

Чек-лист преподавателя

- На временной линии явно отмечены prediction timestamp и появление цели.
- Разобраны event time и available time.
- Показано хронологическое train/validation/test без перемешивания.
- Продемонстрированы успешный и отказной validation report.
- Уточнено, что основной набор BTS не публикуется без проверки условий использования.
- Озвучены общий командный результат и индивидуальные доказательства.

Глава 3. Версионирование данных и воспроизводимые вычисления

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР3 «Версионирование данных и воспроизводимые вычисления»

Цель лекции

Объяснить, как связать состояние кода и данных так, чтобы другой участник мог восстановить конкретный эксперимент. После лекции студент должен различать Git commit, DVC metafile, cache и remote, понимать механизм `dvc add`, `push`, `pull` и `checkout`, а также уметь сформулировать проверяемый протокол воспроизводимости.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Проблема «тот же код, другие данные»	Разобрать неудачное воспроизведение эксперимента
10–25 мин	Git и границы его применения	Сопоставить код, метаданные и крупные бинарные объекты
25–45 мин	Архитектура DVC	Показать workspace, `.dvc`, cache и remote
45–60 мин	Жизненный цикл версии данных	Выполнить add, commit, push, switch, pull
60–72 мин	Воспроизводимые вычисления	Связать параметры, данные, окружение и результат
72–82 мин	Отказы и диагностика	Разобрать повреждение, недоступный remote и рассинхронизацию
82–90 мин	Инструктаж к ЛР3 и мост к MLflow	Зафиксировать доказательства и индивидуальную защиту

1 Почему одного Git commit недостаточно

Начните с истории: два студента запускают один commit и получают разные метрики. Причины могут быть в разных версиях CSV, порядке строк, границах временных частей, зависимостях или параметрах. Git хорошо версионирует исходный код и небольшие текстовые файлы, но хранение каждой версии крупного набора раздувает репозиторий и усложняет обмен.

Предложите студентам формулу состояния эксперимента:

```
результат = f(код, данные, параметры, окружение, случайность)
```

Если хотя бы один компонент не идентифицирован, фраза «у меня тот же эксперимент» не проверяема. На этой лекции основное внимание уделяется данным, но постоянно возвращайте аудиторию к полной формуле.

2 Что хранит Git, а что хранит DVC

Git должен содержать код, конфигурацию, контракт, небольшие разрешённые примеры, `.dvc`-метафайлы, `dvc.yaml`, `dvc.lock` и документацию. Крупные или часто меняющиеся данные размещаются вне Git.

DVC добавляет слой адресации содержимого. После `dvc add data/file.csv` в рабочем дереве появляется метафайл `data/file.csv.dvc`, а объект помещается в cache. В метафайле находится hash, путь и дополнительные свойства. Git версионирует метафайл; DVC по нему находит нужный объект.

Подчеркните: `.dvc`-файл не содержит сам набор. Если cache пуст и remote недоступен, одного Git clone недостаточно. Поэтому утверждение «всё сохранено в Git» ошибочно.

3 Четыре области DVC

Нарисуйте схему:

```
workspace ↔ cache ↔ remote ↑ Git хранит metafile
```

Разберите каждую область:

- workspace — рабочая копия, которую читает код;
- metafile — версия ссылки на содержимое, отслеживаемая Git;
- cache — локальное содержимое, адресуемое по hash;

- remote — внешнее хранилище для обмена и восстановления.

`dvc add` связывает workspace с cache и обновляет metafile. `dvc push` передаёт объекты из cache в remote. `dvc pull` получает недостающие объекты и восстанавливает workspace. `dvc checkout` приводит workspace к версии, указанной текущими метафайлами, используя доступный cache.

Не формируйте ложное представление, будто DVC автоматически создаёт бизнес-версию набора. Смысл изменения, источник и допустимость использования остаются ответственностью команды.

4 Версия данных как пара Git и DVC

Рассмотрите последовательность v1:

1. Валидатор ЛР2 подтверждает контракт и хронологический порядок.
2. `dvc add` обновляет hash.
3. `dvc push` гарантирует наличие объекта в remote.
4. Git commit фиксирует метафайл, контракт и протокол.

Для v2 процесс повторяется после осмысленного изменения. Получаются два Git commit и два DVC hash. Чтобы восстановить v1, участник переключается на соответствующий commit, получает объект через DVC и снова запускает контракт.

Отдельно обсудите порядок операций. Git commit без `dvc push` может ссылаться на объект, которого нет у коллег. `dvc push` без commit отправляет объект, но не сообщает, какое состояние проекта его использует. Оба действия нужны для совместной воспроизводимости.

5 Cache не является резервной копией

Cache ускоряет работу и устраняет дублирование, но может быть очищен. Remote обеспечивает обмен и восстановление, однако тоже требует политики доступа, резервирования и удаления. Локальный учебный remote удобен для лабораторной, но не моделирует надёжность промышленного объектного хранилища.

Обсудите типичные ошибки:

- cache случайно добавлен в Git;
- remote расположен внутри рабочего дерева;
- в общей конфигурации сохранён персональный абсолютный путь;
- ключ доступа попал в `.dvc/config`;
- объект есть только в cache автора и не отправлен в remote;
- один и тот же путь используется для наборов с разным смыслом без изменения контракта.

6 Контрольные суммы и идентификаторы

В протоколе ЛР3 используются три идентификатора:

Идентификатор	Что фиксирует	Чего не доказывает
Git commit SHA	состояние кода и метаданных	наличие объекта данных в remote
DVC hash	объект, отслеживаемый DVC	законность, качество и предметный смысл
SHA-256 файла	точные байты проверенного CSV	связь с кодом и правилами обработки без дополнительной записи

Объясните, что дублирование здесь полезно: каждый идентификатор отвечает на отдельный вопрос. Validation report соединяет их с версией контракта, периодом и результатом проверок.

7 Воспроизводимые вычисления шире версионирования данных

Версия данных необходима, но недостаточна. Нужно зафиксировать:

- Git commit кода;
- версию данных и контракт;
- границы train, validation и test;

- параметры и random seed;
- версии Python и зависимостей;
- команду запуска;
- полученные метрики и артефакты.

На следующей неделе MLflow станет журналом запусков. Сформулируйте границу ответственности: DVC отвечает за получение нужных данных, MLflow — за регистрацию выполнения и результатов. Ни один инструмент не отменяет Git и автоматические тесты.

8 DVC pipeline и отдельные `.dvc`-файлы

ЛР3 использует отдельный `.dvc`-файл, чтобы сделать механизм прозрачным. В дальнейшем воспроизводимые стадии можно описывать в `dvc.yaml`, а фактические зависимости и выходы — в `dvc.lock`. Команда не должна преждевременно усложнять pipeline: сначала требуется доказать восстановление двух версий исходных данных.

Объясните принцип: стадия пересчитывается, если изменились команда, зависимость или параметр, влияющий на выход. Но DVC не способен определить предметную правильность команды; это обеспечивают контракт, тесты и review.

9 Диагностика отказов

Разберите четыре сценария:

1. `dvc status` показывает изменённый файл — workspace не соответствует метафайлу.
2. `dvc pull` не находит объект — версия не отправлена в remote или выбран неправильный remote.
3. Hash совпадает, но контракт падает — возможно, контракт изменился или ранее была принята плохая версия.
4. Контракт проходит, но метрика не повторяется — проверяются параметры, окружение и случайность.

Главный навык — не «починить любой ценой», а локализовать слой несоответствия. Запрещайте использовать `git reset --hard` как универсальную очистку: команда может потерять незакоммиченный вклад.

10 Командная и индивидуальная работа

Для команды 2–5 человек предложите роли: владелец версии данных, DVC engineer, инженер воспроизводимости, quality reviewer и documentation reviewer. Роли ротируются относительно ЛР2.

На индивидуальной защите преподаватель назначает v1 или v2. Студент самостоятельно переключается на commit, получает данные, запускает валидатор и показывает Git SHA, DVC hash и SHA-256. Затем отвечает, на каком слое следует искать конкретный отказ.

11 Вопросы аудитории

1. Почему Git LFS и DVC решают не полностью одинаковые задачи?
2. Почему `.dvc`-файл нужно коммитить, а cache — нет?
3. Чем `dvc pull` отличается от `dvc checkout`?
4. Что произойдёт, если автор сделал commit, но не выполнил push?
5. Почему совпавший hash не доказывает качество данных?
6. Какие элементы кроме данных нужны для воспроизведения метрики?
7. Где должны храниться секреты доступа к remote?
8. Почему восстановленную версию нужно снова проверять контрактом?

12 Завершение

Подведите итог: воспроизводимость — это возможность восстановить состояние и проверить результат, а не наличие папки с названием `final`. Git commit и DVC hash образуют адрес состояния, validation report подтверждает допустимость данных, а ЛР4 добавит к этому адресу параметры, метрики и модель в MLflow run.

Чек-лист преподавателя

- Показана разница между workspace, metafile, cache и remote.
- Продемонстрирована связка Git commit и DVC hash.
- Разобраны `add`, `push`, `pull`, `checkout` и `status`.
- Объяснено, почему cache и remote не содержат бизнес-контекст сами по себе.
- Рассмотрены отказные сценарии без разрушительных команд.
- Уточнена индивидуальная демонстрация ЛР3 и связь с ЛР4.

Источники для обновления примеров

- DVC User Guide: https://doc.dvc.org/user-guide
- DVC `.dvc` files: https://doc.dvc.org/user-guide/project-structure/dvc-files

Глава 4. Эксперименты и базовая оценка модели

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР4 «Эксперименты и базовая оценка модели»

Цель лекции

Показать, как превратить обучение модели из единичного запуска в проверяемый эксперимент. После лекции студент должен уметь сформулировать гипотезу, определить изменяемый фактор, выбрать validation-метрики, зарегистрировать run в MLflow и обосновать решение без использования test-периода.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От запуска к эксперименту	Сравнить «получили score» и проверяемую гипотезу
10–25 мин	Структура experiment и run	Разобрать parameters, metrics, tags, inputs, artifacts
25–42 мин	Базовая линия и сравнимость	Построить baseline и изменить один фактор
42–60 мин	Метрики бинарной классификации	Обсудить ROC AUC, precision, recall, F1 и порог
60–72 мин	Validation и закрытый test	Показать ошибку выбора по test
72–82 мин	Прослеживаемость MLflow	Связать run с Git и DVC
82–90 мин	Инструктаж к ЛР4	Уточнить два run и индивидуальную защиту

1 Почему запуск ещё не является экспериментом

Начните с вопроса: «Студент запустил обучение и получил ROC AUC 0,81. Что мы знаем?» Без версии данных, границ частей, параметров, baseline и контекста метрики почти ничего. Эксперимент начинается с проверяемой гипотезы и заканчивается воспроизводимым решением.

Используйте структуру:

1. Вопрос — какое изменение проверяется?
2. Гипотеза — почему оно должно повлиять на результат?
3. Контроль — что остаётся неизменным?
4. Метрика — по какому наблюдаемому сигналу принимается решение?
5. Ограничение — что мешает обобщить вывод?

Пример: увеличение `max_depth` может улучшить способность модели описывать нелинейные зависимости, но повысить риск переобучения. Данные, split и random seed остаются неизменными; сравнение проводится по validation ROC AUC и F1.

2 Базовая линия

Baseline — не самая простая модель вообще, а заранее определённая точка сравнения. Для задачи задержки рейса это может быть постоянное предсказание частоты положительного класса, логистическая регрессия или Random Forest с фиксированными параметрами.

Объясните две функции baseline:

- обнаружить, что сложная модель не превосходит тривиальный подход;
- обеспечить стабильную точку для регрессионных проверок.

Результат без baseline склонен к риторике «метрика выглядит высокой». Инженерное решение требует относительного сравнения и контекста стоимости ошибок.

3 Структура MLflow Tracking

Experiment группирует связанные runs. Run представляет одно выполнение кода и содержит:

Элемент	Пример	Назначение
Parameter	`max_depth=4`	входная настройка, не меняющаяся внутри run
Metric	`validation_roc_auc=0.81`	измеряемый результат
Tag	`git.commit`, `validation.status`	поиск, контекст и решение
Dataset input	training или validation	связь запуска с набором и digest
Artifact	модель, отчёт, график, manifest	воспроизводимый выход

Подчеркните отличие parameter и metric. Количество деревьев — параметр, время обучения или ROC AUC — метрика. Git commit не следует записывать как числовую метрику; для него подходит tag.

MLflow не делает эксперимент корректным автоматически. Он сохраняет то, что логирует код. Если команда не зафиксировала split или версию данных, красивый интерфейс не устраняет пробел.

4 Сравнимость двух runs

Для учебного эксперимента изменяйте один фактор. Если одновременно заменить данные, алгоритм и порог, невозможно объяснить источник изменения результата. В реальном исследовании применяются многофакторные планы, но они требуют достаточного числа запусков и анализа взаимодействий.

Перед сравнением проверьте:

- одинаковый Git commit или документированное отличие;
- одинаковый DVC hash и контракт;
- одинаковые временные границы;
- одинаковую реализацию метрик;
- одинаковый random seed для изоляции параметра;
- успешный статус и полный набор artifacts.

Если один run выполнен на другом периоде, сравнение метрик отражает одновременно модель и данные. Такой результат можно анализировать, но нельзя называть чистой проверкой гиперпараметра.

5 ROC AUC

ROC AUC оценивает ранжирование положительных и отрицательных примеров по score во всём диапазоне порогов. Значение 0,5 соответствует случайному ранжированию в стандартной бинарной постановке, 1,0 — идеальному.

Не называйте ROC AUC «процентом правильных ответов». Метрика не фиксирует рабочий порог и может выглядеть приемлемо при неудобном качестве редкого положительного класса. На validation с одним классом ROC AUC не определён; pipeline должен завершиться понятной ошибкой.

6 Precision, recall и F1

После выбора порога появляются бинарные решения:

- precision показывает долю истинных задержек среди предупреждений;
- recall показывает долю найденных задержек среди всех фактических задержек;
- F1 является гармоническим средним precision и recall.

Обсудите стоимость ошибок. Ложное предупреждение может вызвать лишнюю операционную реакцию, пропущенная задержка — сорвать планирование. Без бизнес-контекста нельзя объявить precision важнее recall или наоборот.

Порог 0,5 — исходная точка, а не естественный закон. В ЛР4 порог фиксируется для сравнимости. Его оптимизация должна проводиться только на validation и быть отдельным экспериментом.

7 Почему accuracy недостаточно

Если задерживаются 10 процентов рейсов, модель «всегда вовремя» имеет accuracy 90 процентов, но нулевой recall положительного класса. Покажите confusion matrix и попросите студентов вычислить смысл четырёх ячеек.

Accuracy можно логировать как вспомогательную метрику, но решение должно учитывать дисбаланс, стоимость ошибок и метрики положительного класса.

8 Train, validation и test

Train используется для fit модели и обучаемых преобразований. Validation служит для выбора алгоритма, гиперпараметров и порога. Test открывается после завершения выбора и оценивает уже зафиксированное решение.

Каждый повторный взгляд на test и последующая корректировка превращают test в ещё один validation. Оценка становится оптимистичной. В ЛР4 test-период существует в split manifest, но его метрики не используются для выбора run.

Отдельно объясните, что «мы только посмотрели» уже передаёт информацию. Информационная утечка может происходить через человеческое решение, а не только через строку кода.

9 Dataset inputs и безопасность

MLflow может зарегистрировать metadata и digest набора. Это полезно для поиска и сравнения. Но нельзя бездумно загружать закрытый CSV как artifact. Политика данных, лицензия и минимизация имеют приоритет.

Для каждого input укажите context `training` или `validation`, имя, источник, схему и digest. Свяжите это с DVC hash из ЛР3: DVC обеспечивает получение объекта, MLflow фиксирует его использование в конкретном run.

10 Артефакт модели

Артефакт считается полезным, если его можно загрузить и выполнить предсказание. Файл без формата, версии признаков и smoke test не подтверждает воспроизводимость.

В учебном bundle сохраняются модель, порядок признаков, версия, параметры, границы split и validation-метрики. После run другой участник загружает artifact во временный каталог и повторяет предсказание.

11 Статусы и решения

Не смешивайте факт run и решение о продвижении. Успешно завершённый run может быть `candidate` или `rejected`. Неуспешный из-за плохих данных получает `invalid`, а не удаляется из истории.

ЛР4 не использует Model Registry и production stages. Управляемый выпуск будет в ЛР10. Сейчас цель — научиться сравнивать runs и сохранять доказательства.

12 Типичные ошибки

- два run отличаются данными и параметрами одновременно;
- метрика называется `score` без контекста;
- Git commit и DVC hash отсутствуют;
- test используется для выбора;
- модель сохранена, но не загружается;
- неуспешный run удалён вместо анализа;
- персональные или закрытые данные загружены как artifact;
- вывод «модель лучше» сделан по малой разнице на крошечной выборке.

13 Индивидуальная ответственность

Каждый участник должен уметь открыть назначенный run, найти параметры, tags, inputs и artifacts, объяснить одну метрику и воспроизвести загрузку модели. Общий скриншот MLflow не заменяет личный артефакт.

Для команды из пяти человек распределите experiment ownership, модель, оценку, воспроизводимость и quality review. Роли должны ротироваться относительно ЛР3.

14 Вопросы аудитории

1. Чем parameter отличается от metric и tag?

2. Почему два run на разных данных нельзя напрямую сравнивать?
3. Что означает ROC AUC и чего он не показывает?
4. Когда precision важнее recall?
5. Почему accuracy может вводить в заблуждение?
6. Почему test нельзя использовать для настройки порога?
7. Что должен содержать загружаемый модельный artifact?
8. Зачем хранить неуспешные runs?

15 Завершение

Сформулируйте итог: MLflow превращает запуск в журнал доказательств, но научная и инженерная корректность определяется дизайном эксперимента. Два сравнимых runs должны иметь общие данные и split, различимый фактор, корректные validation-метрики и воспроизводимый artifact. ЛР5 проверит эти функции систематическими тестами.

Чек-лист преподавателя

- Показана структура hypothesis, control, metric и decision.
- Разобраны experiment, run, parameters, metrics, tags, inputs и artifacts.
- ROC AUC не интерпретируется как accuracy.
- Precision, recall и F1 связаны со стоимостью ошибок.
- Test остаётся закрытым при выборе.
- Run связан с Git commit и DVC hash.
- Индивидуальная защита требует воспроизведения, а не скриншота.

Официальные источники

- MLflow Tracking: https://mlflow.org/docs/latest/ml/tracking/
- MLflow Dataset Tracking: https://mlflow.org/docs/latest/dataset/
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

Глава 5. Тестирование данных, моделей и pipeline

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР5 «Тестирование данных, модели и pipeline»

Цель лекции

Показать, что тестирование ML-системы проверяет не только функции Python, но и данные, временные границы, артефакт модели и связи компонентов. После занятия студент должен уметь построить пирамиду тестов, определить проверяемые инварианты, локализовать отказ и объяснить, почему высокая метрика не доказывает инженерную корректность pipeline.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Почему обычных unit-тестов недостаточно	Разобрать корректный код на ошибочных данных
10–25 мин	Пирамида тестов ML-системы	Разнести проверки по unit, contract, model и integration
25–40 мин	Fixtures и параметризация	Построить минимальные положительные и отрицательные данные
40–55 мин	Тестирование времени и утечки	Показать нарушение хронологической границы
55–68 мин	Метрики и артефакт модели	Проверить предусловия и инварианты bundle
68–80 мин	Детерминированность и flaky tests	Разделить воспроизводимость и совпадение метрики
80–90 мин	Defect injection и ЛР5	Продемонстрировать тест, который обязан упасть

1 Что именно мы тестируем

Начните с двух утверждений: «функция выполнилась без исключения» и «система выдаёт корректный воспроизводимый результат». Между ними большой разрыв. В ML-системе код может быть синтаксически правильным, а выборка — содержать будущее, метка — попасть в признаки, validation — состоять из одного класса, bundle — потерять порядок признаков.

Тест — это исполняемая формулировка ожидания. Хороший тест отвечает на три вопроса:

1. Какой риск контролируется?
2. Какой минимальный вход воспроизводит этот риск?
3. Какой наблюдаемый сигнал означает отказ?

Фраза «проверить модель» слишком широка. Формулировка «при validation только одного класса расчёт ROC AUC завершается понятным ValueError» уже задаёт вход, действие и ожидаемый результат.

2 Пирамида тестов ML-системы

Слой	Объект	Пример	Частота
Unit	чистая функция	расчёт метрик и проверка threshold	каждый commit
Data contract	схема и значения	диапазон DepHour, обязательные поля	каждый pull request
Model	сериализованный bundle	features, version, диапазон predict_proba	каждый pull request
Integration	связь компонентов	validate → train → load → predict	каждый pull request
API	внешний контракт	Pydantic, status code, response	перед контейнеризацией
End-to-end	пользовательский путь	контейнер и реальный HTTP-запрос	перед release

Чем выше слой, тем реалистичнее проверка, но тем медленнее диагностика. Один end-to-end не заменяет unit-тесты: он обнаруживает проблему, но часто не объясняет её источник.

3 Инвариант вместо конкретного числа

Инвариант — свойство, которое должно сохраняться для допустимого набора входов. Для нашего проекта:

- вероятность конечна и находится от 0 до 1;
- порядок признаков совпадает с `bundle["features"]`;
- train расположен во времени раньше validation, а validation — раньше test;
- в каждой части присутствуют оба класса;
- версия, параметры и границы split сохраняются рядом с моделью;
- временные файлы теста не меняют эталонные данные.

Не следует без основания фиксировать точную метрику небольшой случайной модели. После безопасного обновления scikit-learn или изменения реализации число может немного измениться, хотя контракт остаётся верным. Используйте `pytest.approx` там, где допустима численная погрешность, и точное сравнение там, где проверяется схема или версия.

4 Arrange, Act, Assert

Предложите студентам читать тест как короткий эксперимент:

1. Arrange — подготовить минимальный вход и ожидаемое поведение.
2. Act — выполнить одно проверяемое действие.
3. Assert — проверить наблюдаемый результат или исключение.

Если в Arrange запускается половина системы, а Assert содержит только `assert result`, тест трудно диагностировать. Имя должно объяснять условие и результат: `test_metrics_reject_single_class`, а не `test_metrics_2`.

5 Fixtures и изоляция

Fixture предоставляет тесту состояние с понятным временем жизни. `tmp_path` создаёт отдельную директорию и устраняет зависимость от файлов предыдущего запуска. Общая fixture корректных данных снижает дублирование, но не должна превращаться в неявный глобальный объект, который тесты меняют по очереди.

Покажите два принципа:

- fixture возвращает новый или безопасно копируемый объект;
- тест явно создаёт нарушение, которое проверяет.

Слишком большая fixture скрывает причину отказа. Для проверки диапазона часа достаточно нескольких строк, а не полного учебного набора.

6 Параметризация отрицательных сценариев

Параметризация позволяет применить одно правило к нескольким независимым входам. Например, проверка должна отвергать `DepHour=-1`, `DepHour=24`, неизвестную колонку и неверный формат даты. Каждому case полезно дать читаемый `id`, чтобы отчёт был понятен без открытия кода.

Не объединяйте разные причины отказа только ради короткого файла. Если сценарии требуют разных действий или разных сообщений, отдельные тесты понятнее.

7 Тестирование времени

Для временных данных случайное перемешивание может создать скрытую утечку. Контракт проверяет:

- строки упорядочены по `FlightDate`;
- максимальная дата train меньше или равна минимальной дате validation;
- максимальная дата validation меньше или равна минимальной дате test;
- признаки доступны за 60 минут до планового отправления;
- фактическая задержка используется только как отложенная цель.

Попросите аудиторию придумать тест для признака `actual_arrival_time`. Правильный ответ включает не только имя колонки, но и проверку момента доступности.

8 Тесты функции оценки

До расчёта метрик проверяются предусловия:

- длины меток и вероятностей совпадают и не равны нулю;
- метки бинарны;
- присутствуют оба класса;
- вероятности конечны и находятся в диапазоне от 0 до 1;
- threshold находится строго между 0 и 1.

После этого проверяются ключи результата и диапазоны. Тест идеального ранжирования полезен как простой эталон, но одного положительного сценария недостаточно.

9 Тесты model bundle

Сериализованный файл — это контракт между обучением и сервисом. Bundle должен включать:

Поле	Риск при отсутствии
`model`	предсказание невозможно
`features`	сервис меняет порядок входов
`version`	ответ API нельзя связать с артефактом
`parameters`	обучение не воспроизводится
`split`	временные границы теряются
`validation_metrics`	решение нельзя проверить

Помимо структуры нужен smoke test: загрузить bundle и вызвать `predict_proba` на строке с признаками в сохранённом порядке.

10 Детерминированность

Одинаковые данные, параметры, код и random seed должны давать согласованный результат. Но тест «после смены seed метрика обязана измениться» неверен: на маленьком наборе она может совпасть. Проверяйте, что seed зафиксирован, два запуска с одним seed совпадают по предсказаниям, а metadata отражает настройку.

Детерминированность не гарантирует корректность. Стабильно воспроизводимая утечка остаётся ошибкой.

11 Flaky tests

Flaky test иногда проходит, иногда падает без изменения проверяемого кода. Типичные причины:

- незафиксированный seed;
- зависимость от порядка тестов;
- общие файлы и порты;
- внешняя сеть;
- ожидание процесса фиксированным `sleep`;
- слишком строгая проверка float.

Лечение — не автоматический повтор до зелёного статуса. Сначала найдите источник недетерминизма, изолируйте состояние и используйте ожидание наблюдаемого события с timeout.

12 Integration smoke test

Минимальный сквозной тест ЛР5 выполняет `validate → train → load → predict`. Он не проверяет каждый алгоритмический случай, а подтверждает совместимость компонентов. Артефакт создаётся в `tmp_path`, загружается тем же способом, что и сервис, а вероятность проверяется на форму и диапазон.

На следующей неделе этот путь будет вынесен за границу процесса: Docker image должен пройти HTTP smoke test. Таким образом, тестовый контракт развивается вместе с системой.

13 Defect injection

Чтобы доказать чувствительность теста, студент временно вносит контролируемый дефект. Например, разрешает `DepHour=24` или удаляет поле версии bundle. До изменения студент называет test node ID и ожидаемую причину отказа; после демонстрации возвращает корректный код.

Если тест остаётся зелёным, это полезный результат: обнаружен пробел покрытия. Нельзя оставлять дефект в принимаемой ветке или подменять демонстрацию заранее подготовленным скриншотом.

14 Диагностика отказа

Учите читать traceback сверху вниз по пользовательскому коду, но искать первое полезное место, а не последнюю строку вообще. Диагностическое сообщение содержит:

- имя теста и case;
- ожидаемое и фактическое значение;
- минимальный контекст;
- причину на языке контролируемого риска.

Большой дамп DataFrame редко помогает. Полезнее показать нарушенное поле, строку и правило контракта.

15 Что не является доказательством качества

- процент coverage без связи с рисками;
- тест, который не содержит содержательного assertion;
- проверка только успешного пути;
- один общий скриншот команды;
- JUnit XML, созданный не из принимаемого commit;
- зелёный тест после удаления проверяемой логики;
- мокирование самого объекта, который нужно проверить.

Coverage показывает выполненные строки, но не полноту требований. Используйте его как навигационный сигнал, а не как единственный quality gate.

16 Индивидуальная ответственность

Каждый участник должен владеть одним наблюдаемым риском: создать тест или fixture, провести содержательное review, локализовать отказ либо улучшить диагностическое сообщение. На защите студент запускает собственный тест по node ID и выполняет назначенное изменение входа.

Для команды из пяти человек подходят зоны data contract, evaluation, model bundle, integration и quality review. Для пары роли совмещаются, но артефакт и демонстрация остаются индивидуальными.

17 Вопросы аудитории

1. Почему `assert result is not None` недостаточно для вероятности?
2. Когда нужен `pytest.raises`?
3. Почему общий временный файл делает тесты зависимыми?
4. Чем model test отличается от integration test?
5. Что доказывает defect injection?
6. Почему смена seed не обязана менять метрику?
7. Как проверить отсутствие временной утечки?
8. Почему зелёный end-to-end не локализует отказ?

18 Завершение

Сформулируйте итог: надёжность ML-pipeline строится не максимальным числом тестов, а явной связью «риск → инвариант → минимальный вход → диагностический сигнал». ЛР5 фиксирует эту связь в pytest и отчётах, а ЛР6 переносит те же гарантии в контейнерную границу.

Чек-лист преподавателя

- Показаны уровни unit, contract, model, integration и end-to-end.
- Разобраны fixtures, `tmp_path`, parameterization и `pytest.raises`.
- Временная утечка сформулирована как тестируемый риск.
- Метрики проверяют предусловия, а не только диапазон ответа.
- Bundle рассматривается как контракт обучения и сервиса.
- Flaky test не «лечится» бесконечным повтором.
- Defect injection выполняется индивидуально и откатывается.
- Лекция связана с переходом к контейнерному smoke test.

Официальные источники

- pytest documentation: https://docs.pytest.org/en/stable/
- pytest fixtures: https://docs.pytest.org/en/stable/how-to/fixtures.html
- pytest parametrization: https://docs.pytest.org/en/stable/how-to/parametrize.html
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

Часть II. Доставка модели как программной системы

Глава 6. Контейнеризация и управление средой

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР6 «Контейнеризация и управление средой»

Цель лекции

Показать контейнер как проверяемую границу поставки ML-сервиса, а не как архив исходного кода. После занятия студент должен различать image и container, build-time и runtime, объяснять layers и cache, создавать минимальный non-root runtime, настраивать readiness и связывать image с версией кода и модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Зачем ML-системе контейнер	Сопоставить «работает у меня» и переносимый контракт
10–25 мин	Image, container, registry и digest	Разобрать неизменяемый артефакт и экземпляр процесса
25–40 мин	Build context, layers и cache	Показать влияние порядка COPY и `.dockerignore`
40–55 мин	Многостадийная сборка	Отделить quality/build от runtime
55–68 мин	Безопасность runtime	Разобрать USER, read-only, capabilities и secrets
68–80 мин	Health, readiness и Compose	Показать контейнер без готовой модели
80–90 мин	Диагностика и ЛР6	Связать build log, inspect и smoke test

1 Контейнер как контракт поставки

Начните с вопроса: «Если requirements и исходный код находятся в Git, зачем нужен image?» Репозиторий описывает материалы для сборки, но не фиксирует итоговую файловую систему, системные библиотеки, пользователя процесса, команду запуска и готовность сервиса.

Контейнеризация должна обеспечить:

- одинаковый способ запуска на машине студента и runner;
- известную версию runtime и зависимостей;
- ограниченную поверхность исполнения;
- наблюдаемую готовность модели;
- прослеживаемость от commit до image ID или digest.

Контейнер не исправляет невоспроизводимые данные, утечку или плохие тесты. Он воспроизводимо упаковывает и хорошие, и плохие решения.

2 Image и container

Image — неизменяемый шаблон файловой системы и конфигурации. Container — запущенный экземпляр image с собственным состоянием процесса и writable layer. Один image может породить несколько containers.

Понятие	Вопрос	Пример
Dockerfile	как собрать?	инструкции FROM, COPY, RUN, USER
Image	что поставляем?	приложение, зависимости, модель, metadata
Container	что сейчас выполняется?	PID, сеть, health и runtime environment
Registry	где хранится image?	репозиторий образов
Tag	удобное имя ссылки	`lr6-a1b2c3d`
Digest	content-addressed идентификатор	`sha256:...`

Tag можно переназначить, digest зависит от содержимого. Поэтому `latest` не доказывает, какая версия развёрнута. В ЛР6 используется tag с short SHA и локальный image ID; к итоговому release добавится registry digest.

3 Build context

Build context передаётся builder и определяет, что доступно `COPY`. Если context содержит `.git`, данные, секреты и отчёты, они могут увеличить сборку или попасть в image layer.

`.dockerignore` исключает файлы до передачи. Он полезен для производительности и снижения риска, но не является хранилищем секретов. Если секрет уже попал в опубликованный image, его нужно отозвать, а не только дописать правило ignore.

Попросите аудиторию классифицировать каталоги starter-project:

- `src`, `tests`, `data` нужны quality-стадии;
- `models` не копируется, потому что учебная модель строится внутри build;
- `.git`, `.env`, `reports`, `mlruns` и документация не нужны runtime.

4 Layers и cache

Каждая инструкция создаёт слой или меняет конфигурацию image. Cache повторно использует слой, если инструкция и её входы не изменились. Поэтому стабильные зависимости копируют и устанавливают раньше часто меняющегося кода.

Плохой порядок:

```
COPY . .
RUN pip install .
```

Любое изменение README инвалидирует установку. Более управляемый подход копирует dependency metadata отдельно, а исходники — после. В учебном проекте простота также важна, поэтому оптимизацию нужно измерять по build log, а не превращать Dockerfile в нечитабельную головоломку.

5 Build-time и runtime

Build-time — создание image. Здесь допустимы tests, linters, компиляторы и исходные данные для учебной модели. Runtime — работа сервиса; здесь остаётся только необходимое для обслуживания запросов.

Смешение границ приводит к рискам:

- тестовые зависимости увеличивают поверхность атаки;
- исходные данные попадают в поставку;
- модель обучается при каждом запуске container;
- runtime требует доступ к сети и package index;
- ошибка становится зависимой от внешней доступности.

Обучение модели в build используется в ЛР6 как учебная гарантия воспроизводимости. В зрелом pipeline image и модель могут версионироваться отдельно, но release manifest обязан связывать их.

6 Многостадийная сборка

Первая стадия `quality` выполняет Ruff, pytest, contract validation, обучение и подготовку wheels. Вторая стадия `runtime` получает только wheels и модель.

Преимущества:

- failed test останавливает поставку;
- runtime не содержит tests и dev tools;
- граница между проверкой и эксплуатацией видна в коде;
- каждую стадию можно диагностировать отдельно.

Важно: наличие стадии tests ещё не означает gate. Финальная стадия должна зависеть от её успешного результата. Иначе builder может пропустить независимую стадию.

7 Base image

Base image является зависимостью. Общий тег вроде `python:3.12-slim` удобен для лабораторной, но может обновляться. `--pull` позволяет проверить свежую базу; production release обычно фиксирует проверенный digest и регулярно обновляет его отдельным процессом.

Не утверждайте, что самый маленький image автоматически самый безопасный. Важны происхождение, обновляемость, совместимость wheels, диагностируемость и реально необходимые пакеты.

8 Non-root runtime

`USER` задаёт пользователя для следующих инструкций и runtime-команды. Если сервису не нужны административные права, root нарушает принцип минимальных привилегий.

Проверка:

```
docker image inspect IMAGE --format '{{.Config.User}}'

docker run --rm --entrypoint id IMAGE
```

Non-root не является полной песочницей. Его дополняют `cap_drop`, `no-new-privileges`, read-only filesystem, ограничения ресурсов и безопасная конфигурация хоста.

9 Read-only filesystem и временные данные

Сервис предсказаний не должен менять код и модель. `read_only: true` делает случайную запись наблюдаемой ошибкой. Для действительно временных данных предоставляется небольшой `tmpfs` в `/tmp`.

Если приложение требует запись в произвольную директорию, сначала выясните причину. Добавлять широкий writable volume только ради устранения ошибки — значит скрыть нарушение контракта.

10 Секреты и конфигурация

Runtime configuration передаётся через environment или внешние механизмы, но значение секрета не должно попадать в Dockerfile, `ARG`, label, image history и отчёт. `.env` удобен локально, но исключается из Git и build context.

Разделяйте:

- несекретную конфигурацию — порт, путь модели, режим логов;
- секрет — token, пароль, ключ;
- артефакт — модель, contract, release manifest.

11 CMD и жизненный цикл процесса

Exec-форма `CMD` запускает процесс без лишней shell-обёртки и корректнее получает сигналы остановки. Container живёт, пока работает основной процесс PID 1. Фоновый запуск Uvicorn внутри shell-скрипта может нарушить обработку сигналов.

`EXPOSE 8080` документирует порт image, но не публикует его на host. Публикацию задают `docker run -p` или Compose.

12 Health и readiness

Различайте:

- liveness/health — процесс способен отвечать;
- readiness — сервис загрузил модель и может выполнить полезную работу.

Контейнер с работающим Uvicorn, но отсутствующей моделью, жив, однако не готов. Поэтому healthcheck ЛР6 обращается к `/readyz`, а не только `/healthz`.

Healthcheck имеет `start_period`, `interval`, `timeout` и `retries`. Статус `starting` нормален во время инициализации. Нельзя заменять проверку готовности длинным фиксированным `sleep`.

13 Docker Compose

Compose фиксирует локальный способ запуска сервиса: build target, image tag, port mapping, environment, healthcheck и runtime restrictions. Современный `compose.yaml` не требует верхнеуровневого поля `version`.

Compose не превращает локальный стенд в production orchestrator. Его задача в курсе — дать одной команде воспроизводимый контур для ЛР6, ЛР7 и последующей наблюдаемости.

14 HTTP smoke test

Smoke test проверяет минимальный пользовательский путь через реальную сетевую границу:

1. дождаться `/readyz` до timeout;
2. вызвать `/predict`;
3. проверить status 200;
4. проверить форму и диапазоны ответа;
5. сохранить компактный отчёт.

Он не заменяет unit/API tests, но обнаруживает ошибки packaging, port mapping, модели и команды запуска.

15 Диагностика по слоям

Симптом	Первый источник	Возможная причина
build падает на pip	build log	dependency или сеть
build падает на pytest	test report	регрессия кода
container сразу завершился	`docker compose logs`	CMD или import error
health остаётся unhealthy	inspect health log	модель, URL или порт
503 на `/readyz`	application log	неверный MODEL_PATH
permission denied	user и mounts	запись вне разрешённого пути

Учите сначала локализовать фазу: build, create, start, readiness или request. Это сокращает область поиска.

16 Что не следует делать

- копировать весь репозиторий в runtime;
- использовать `latest` как доказательство версии;
- устанавливать пакеты при старте container;
- запускать сервис от root «для удобства»;
- встраивать token через `ARG`;
- считать `EXPOSE` публикацией порта;
- объявлять healthy процесс без загруженной модели;
- удалять секрет в позднем layer после его копирования;
- редактировать container вручную вместо пересборки image.

17 Индивидуальная ответственность

Участник должен владеть проверяемой частью границы: слоями сборки, безопасностью runtime, переносом модели, smoke test или review build context. На защите он показывает image ID, связанную строку Dockerfile/Compose и диагностирует назначенный отказ.

Для команды из пяти роли распределяются по независимым зонам. Для пары роли совмещаются, но у каждого остаётся собственный PR/review и отдельная операция диагностики.

18 Вопросы аудитории

1. Почему image не равен container?
2. Чем tag отличается от digest?
3. Может ли `.dockerignore` удалить секрет из существующего image?
4. Почему отдельная test stage может не быть gate?
5. Зачем runtime нужен non-root пользователь?

6. Почему `/healthz` недостаточно для ML-сервиса?
7. Что произойдёт при `read_only: true` и записи в `/app`?
8. Как связать image с commit и моделью?

19 Завершение

Контейнеризация завершена, когда команда может доказать происхождение image, успешность проверок, минимальность runtime, готовность модели и воспроизводимость HTTP-запроса. В ЛР7 эта граница получит формальный OpenAPI-контракт, типизированные ошибки и негативные сценарии клиента.

Чек-лист преподавателя

- Различены Dockerfile, image, container, tag и digest.
- Объяснены build context, layers, cache и `.dockerignore`.
- Quality stage является реальным gate runtime.
- Runtime не содержит tests, data и dev tools.
- Non-root дополнен read-only и security options.
- Health отделён от readiness модели.
- Smoke test проходит реальную HTTP-границу.
- Индивидуальная защита включает диагностику отказа.

Официальные источники

- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker build best practices: https://docs.docker.com/build/building/best-practices/
- Multi-stage builds: https://docs.docker.com/get-started/docker-concepts/building-images/multi-stage-builds/
- Build context: https://docs.docker.com/build/concepts/context/
- Compose services: https://docs.docker.com/reference/compose-file/services/

Глава 7. Сервис предсказаний и контракт API

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР7 «Сервис предсказаний и контракт API»

Цель лекции

Показать API как устойчивую границу между моделью и клиентом. После занятия студент должен уметь проектировать строгий request/response contract, объяснять OpenAPI, readiness, status codes и request ID, оценивать совместимость изменений и связывать контракт с автоматическими тестами.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Модель и сервис — разные продукты	Разобрать путь запроса от клиента к артефакту модели
10–25 мин	HTTP-контракт и момент прогноза	Вывести минимальный request из data contract
25–40 мин	Pydantic и строгая валидация	Показать ошибки неявного приведения и лишних полей
40–52 мин	Response и ошибки	Сопоставить 200, 422, 500 и 503
52–65 мин	OpenAPI и совместимость	Прочитать сгенерированную JSON Schema
65–76 мин	Lifespan, health и readiness	Смоделировать процесс без загруженной модели
76–84 мин	Наблюдаемость и request ID	Связать запрос, ответ, лог и метрику
84–90 мин	Contract tests и ЛР7	Сформулировать доказательства индивидуальной сдачи

1 От модели к обслуживаемому решению

Начните с вопроса: «Что должен знать клиент, чтобы использовать модель?» Клиенту не нужен объект scikit-learn, путь к `joblib` или порядок колонок внутри DataFrame. Ему нужен стабильный договор: адрес, метод, допустимый JSON, ответ, ошибки и обещания по готовности.

Модель отвечает на вопрос о вычислении. Сервис дополнительно решает:

- когда и как загрузить модель;
- какие входы допустимы в момент прогноза;
- как преобразовать вход в признаки;
- как обозначить неготовность;
- как ограничить и документировать выход;
- как найти конкретный запрос в диагностике;
- как изменить контракт, не разрушив клиентов.

Поэтому «обернуть модель в FastAPI» недостаточно. Сначала проектируют внешний контракт, затем связывают его с реализацией.

2 Граница доверия

HTTP-вход — недоверенные данные. Клиент может прислать пропущенное поле, строку вместо числа, дополнительный ключ, чрезмерно длинный идентификатор или признак, который существует в датасете, но неизвестен на момент прогноза.

Для проекта задержки рейса прогноз строится за 60 минут до планового отправления. В request допустимы `dep_hour` и `is_weekend`, но недопустимы фактическая задержка и любые признаки будущего. Это не только защита формата: API продолжает data contract ЛР2.

Попросите студентов разделить признаки на три группы: доступны клиенту; вычисляются сервером; запрещены из-за утечки. Такой разбор предотвращает ситуацию, когда удобство демонстрации незаметно меняет постановку ML-задачи.

3 Request model

Pydantic-модель задаёт типы, диапазоны, обязательность и политику дополнительных полей. Для учебного сервиса важна строгая семантика: строка `"9"` не должна молча становиться integer, а неизвестный ключ — игнорироваться.

```
class PredictionRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    dep_hour: StrictInt = Field(ge=0, le=23)
    is_weekend: StrictInt = Field(ge=0, le=1)
```

Объясните компромисс. Строгость раньше обнаруживает несовместимость, но требует дисциплины клиента. Допустимая нормализация тоже возможна, если она явная, документированная и покрыта тестом. Опасна не конверсия сама по себе, а случайная конверсия без договорённости.

Покажите четыре отрицательных примера: `dep_hour=24`, отсутствие `is_weekend`, строка вместо integer и лишнее поле `actual_delay`. Студент должен до запуска назвать ожидаемый status и элемент schema, который вызовет отказ.

4 Response model

Ответ также является контрактом. Если endpoint возвращает произвольный словарь, внутреннее поле может случайно стать публичным, а рефакторинг — сломать клиента.

Учебный response содержит вероятность, бинарное решение, threshold, версию модели и request ID. Важны инварианты:

- вероятность принадлежит `[0, 1]`;
- `delayed` вычисляется относительно возвращённого threshold;
- версия относится к фактически загруженному bundle;
- request ID совпадает с заголовком.

Версия приложения и версия модели — разные координаты. API `0.3.0` может обслуживать model bundle `0.2.0`; manifest или response должен позволять их связать.

5 Ошибка — тоже ответ

Необработанный stack trace не является API. Ошибка должна иметь стабильный код для программы, понятное сообщение и идентификатор для диагностики, но не раскрывать внутренние пути, секреты и персональные данные.

Status	Смысл	Действие клиента
200	запрос выполнен	использовать результат
404	путь не существует	исправить URL или версию
422	request нарушает schema	исправить данные
500	необработанный дефект сервера	зарегистрировать инцидент
503	сервис жив, но зависимость не готова	повторить с backoff

Подчеркните: ответ 200 с полем `error` разрушает машинную семантику. Полный текст сообщения Pydantic может меняться; стабильнее тестировать status, `error_code`, location и наличие деталей.

6 Health и readiness

`/healthz` отвечает на вопрос «процесс способен отвечать?». `/readyz` — «можно ли направлять сюда рабочие запросы?». Если модель отсутствует, процесс может быть жив, но не готов.

Смешение проверок создаёт две крайности: тяжёлый health вызывает перезапуски при временной проблеме зависимости; поверхностный readiness отправляет запросы в экземпляр без модели. Для ЛР7:

- `/healthz` не выполняет inference;
- `/readyz` проверяет загруженный bundle;
- `/predict` возвращает согласованный 503, если модель не готова.

7 Lifespan модели

Модель загружается один раз при старте приложения. Загрузка на каждый request увеличивает latency, создаёт файловую гонку и затрудняет диагностику. Lifespan задаёт переход состояния «инициализация → готов → завершение».

Различайте отсутствующий и повреждённый файл. Отсутствие модели может дать наблюдаемый режим not ready, если он предусмотрен эксплуатацией. Повреждённый bundle обычно должен остановить startup: принимать трафик с частично инициализированным состоянием опаснее явного отказа.

8 OpenAPI как исполняемая спецификация

FastAPI строит OpenAPI из route declarations и моделей. Это полезно только тогда, когда схема проверяется. Swagger UI — удобный клиент для исследования, но скриншот интерфейса не фиксирует весь контракт.

Предложите открыть `openapi.json` и найти:

1. `info.version`;
2. POST `/predict`;
3. ссылку на `PredictionRequest`;
4. обязательные поля и запрет дополнительных;
5. schemas ответов 200, 422 и 503;
6. `PredictionResponse` и `ErrorResponse`.

Схема сохраняется как build evidence и генерируется из принимаемого commit. Ручное редактирование JSON уничтожает прослеживаемость.

9 Совместимость

Сервер запускается — не значит, что изменение совместимо. Удаление поля, новое обязательное поле request, сужение диапазона и смена смысла существующего поля ломают часть клиентов.

Добавление необязательного поля ответа обычно совместимо, если клиенты игнорируют неизвестные ключи. Однако строгий клиент может упасть, поэтому фактические потребители важнее формальной таблицы.

Для несовместимого изменения нужны новая версия API, миграционное окно или параллельная поддержка. Версионирование без описания перехода лишь присваивает имя поломке.

10 Request ID

Request ID связывает клиентский симптом, response, лог и метрику. Middleware принимает безопасный идентификатор ограниченной длины или генерирует UUID и возвращает значение в header. Для JSON-ответов учебного API оно также включается в body.

Request ID не должен содержать email, token или payload. Он помогает корреляции, но не заменяет trace context. В распределённой системе дополнительно применяют стандартные trace/span identifiers.

Проведите демонстрацию: отправьте `X-Request-ID: lecture-case-1`, получите 422 и найдите то же значение в заголовке и теле. Затем повторите без header и покажите сгенерированный UUID.

11 Contract tests

Contract test проверяет наблюдаемое поведение границы. Минимальный набор включает:

- валидный request и полный response;
- граничные и ошибочные входы;
- not-ready mode;
- согласованность request ID;
- структуру OpenAPI;
- реальный HTTP-smoke внутри container.

`TestClient` даёт быстрый и детерминированный контур, но не обнаруживает ошибку команды запуска, порта или container network. Поэтому unit contract tests дополняются HTTP-smoke после Compose.

Избегайте хрупких проверок полного текста ошибок и порядка ключей JSON. Проверяйте семантические поля и инварианты.

12 Безопасность и эксплуатационные ограничения

Учебный сервис не включает полноценную аутентификацию, rate limiting и TLS termination. Это осознанное ограничение, а не рекомендация для production.

Даже в лаборатории необходимо:

- не логировать полный payload по умолчанию;
- не возвращать stack trace;
- ограничивать размеры и типы полей;
- не принимать признаки будущего;
- не включать секреты в image и OpenAPI examples;
- запускать container от non-root пользователя.

13 Демонстрационный сценарий

1. Запустить tests и экспортировать OpenAPI.
2. Запустить Compose и проверить `/healthz` и `/readyz`.
3. Выполнить валидный `/predict` с заданным request ID.
4. Передать строку вместо integer и объяснить 422.
5. Передать неизвестное поле и связать отказ с `additionalProperties: false`.
6. Запустить HTTP contract-smoke и открыть JSON-отчёт.
7. Временно убрать model bundle и сопоставить health=200, readiness=503.

Перед каждым действием аудитория прогнозирует результат. Это превращает показ команд в проверку ментальной модели.

14 Типичные ошибки

Ошибка	Почему опасна	Как обнаружить
`dict` вместо request model	контракт не ограничен	schema и unknown-field test
неявное приведение типов	дефект клиента скрывается	wrong-type test
модель загружается на request	нестабильная latency	профилирование и review
health равен readiness	трафик идёт в неготовый pod	тест без модели
произвольный error JSON	клиент не может ветвиться	response-schema test
ручной OpenAPI	схема расходится с кодом	экспорт в CI
общий скриншот команды	нет индивидуального доказательства	личный negative case

15 Вопросы аудитории

1. Почему дополнительное поле request может быть опасным, даже если код его игнорирует?
2. В каком случае 503 корректнее 500?
3. Почему `/docs` не заменяет сохранённый OpenAPI JSON?
4. Что произойдёт со строгим клиентом после добавления поля ответа?
5. Где должна загружаться модель и почему?
6. Какие данные нельзя включать в request ID?

16 Связь с ЛР7 и ЛР8

В ЛР7 команда реализует контракт, а каждый участник проводит собственный defect injection и защиту. В ЛР8 экспорт OpenAPI и contract tests становятся обязательными quality gates. Тем самым договор переходит из текста в правило слияния: несовместимое изменение не попадает в `main` без видимого красного check.

17 Итог

Надёжный inference API — не сетевой фасад над `predict`. Это проверяемый договор о доступных входах, ответах, ошибках, готовности, версиях и диагностике. Сильный контракт уменьшает число скрытых предположений между владельцами данных, модели, сервиса и клиента.

18 Официальные источники

- FastAPI request body: https://fastapi.tiangolo.com/tutorial/body/
- FastAPI response models: https://fastapi.tiangolo.com/tutorial/response-model/
- FastAPI error handling: https://fastapi.tiangolo.com/tutorial/handling-errors/
- FastAPI lifespan events: https://fastapi.tiangolo.com/advanced/events/
- FastAPI OpenAPI metadata: https://fastapi.tiangolo.com/tutorial/metadata/
- Pydantic models: https://docs.pydantic.dev/latest/concepts/models/
- OpenAPI Specification: https://spec.openapis.org/oas/latest.html

Глава 8. Непрерывная интеграция и quality gates

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР8 «Непрерывная интеграция и quality gates»

Цель лекции

Показать непрерывную интеграцию как проверяемый механизм принятия изменений, а не как набор команд в облачном runner. После занятия студент должен проектировать события, jobs и gates, ограничивать permissions, диагностировать отказ, сохранять evidence и связывать branch protection с конкретным commit.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От локального теста к правилу интеграции	Сравнить «у меня прошло» и required check
10–22 мин	Event, workflow, job, step, runner	Разобрать структуру `ci.yml`
22–35 мин	Дизайн quality gates	Расставить проверки по стоимости и риску
35–47 мин	Permissions и недоверенный PR	Показать границу token и опасность контекста
47–58 мин	Concurrency и актуальность run	Связать branch, ref, SHA и отмену
58–70 мин	Artifacts и доказательства	Открыть JUnit, validation и OpenAPI reports
70–82 мин	Диагностика красного run	Найти первый failed step и корневую причину
82–90 мин	Branch protection и ЛР8	Зафиксировать индивидуальный failure/fix

1 Что именно интегрирует CI

Начните с ситуации: два участника по отдельности получили зелёные tests, но объединённый pull request ломает сборку. CI проверяет не намерение автора и не состояние его ноутбука, а конкретный commit в известной среде.

Непрерывная интеграция отвечает на вопросы:

- какое изменение проверяется;
- на каком событии началась проверка;
- какие обязательные контракты применены;
- какие артефакты получены;
- может ли изменение попасть в `main`;
- кто и по какому доказательству диагностировал отказ.

Зелёная иконка без известного SHA и списка gates — слабое доказательство. Run должен относиться к HEAD принимаемого pull request.

2 Локальная проверка и remote check

Локальные команды дают быстрый feedback и помогают не тратить runner time. Remote check выполняется в чистой среде и применяется одинаково к участникам.

Локально	В CI
среда могла быть изменена вручную	runner создаётся заново
доступны незакоммиченные файлы	проверяется commit
могут существовать скрытые caches	cache объявляется явно
автор может пропустить команду	workflow задаёт обязательный набор
результат легко потерять	status и artifact связаны с run

Оба контура нужны. CI не должен быть первым местом, где студент узнаёт о синтаксической ошибке.

3 Модель GitHub Actions

Workflow — YAML-файл автоматизации. Event создаёт run; run содержит jobs; job выполняется на runner; job состоит из steps. Step запускает shell-команду или action.

Проведите чтение сверху вниз:

1. `name` — имя workflow;
2. `on` — события;
3. `permissions` — возможности token;
4. `concurrency` — политика одновременных runs;
5. `jobs` — независимые границы выполнения;
6. `needs` — зависимость jobs;
7. `steps` — фактические команды.

YAML корректен синтаксически — не значит, что процесс корректен инженерно. Workflow может всегда завершаться успешно, проверять не тот каталог или собирать image до tests.

4 События и контекст доверия

Для учебного CI используются:

- `pull_request` — проверка предлагаемого изменения;
- push в `main` — контроль принятой истории;
- `workflow_dispatch` — управляемый ручной запуск.

`pull_request_target` выполняется в контексте базового репозитория и требует особенно осторожного обращения с кодом из fork. Не используйте его как замену `pull_request` для выполнения недоверенного student code.

Попросите студентов назвать, какие значения меняются между событиями: ref, SHA, actor, доступность secrets и права token.

5 Принцип минимальных полномочий

```
permissions:
  contents: read
```

Если workflow только читает код и запускает проверки, write permission не требуется. Checkout выполняется с `persist-credentials: false`, чтобы последующие команды не получили сохранённый Git credential.

Объясните, что permission — часть модели угроз. Скомпрометированная зависимость или ошибочный скрипт ограничены тем, что разрешено token. Нельзя компенсировать неясность выдачей `write-all`.

6 Quality gate как решение о риске

Gate — проверка, неуспех которой блокирует продвижение изменения. Хороший gate:

- контролирует названный риск;
- воспроизводим локально или имеет ясный remote-контур;
- завершает процесс ненулевым exit code;
- выдаёт диагностическое доказательство;
- не зависит от ручного редактирования отчёта;
- имеет владельца и понятный порядок исключения.

В проекте применяются lint, format, data contract, tests, OpenAPI export, smoke training и container build. Не каждая метрика должна сразу стать gate: порог вводится после определения владельца, данных и реакции на отказ.

7 Порядок gates

Сначала выполняют быстрые и дешёвые проверки. Если lint занимает секунды, нет смысла до него строить container несколько минут.

Рекомендуемая последовательность:

1. установка воспроизводимых зависимостей;
2. lint и format check;
3. data contract;
4. unit и contract tests;
5. OpenAPI export;
6. smoke training;
7. container build.

Разделение на jobs `quality` и `container` делает зависимость видимой. `needs: quality` не только оптимизирует время, но и отражает решение: непроверенный код не становится поставляемым образом.

8 Fail fast и полный отчёт

Fail fast уменьшает стоимость: после первой критической ошибки обычные steps не выполняются. Но диагностика уже созданных проверок должна сохраниться.

```
- name: Upload quality evidence
  if: always()
  uses: actions/upload-artifact@v4
```

`always()` не превращает упавший job в успешный. Он разрешает сервисному шагу загрузить доступные reports. Если ранний lint упал до создания каталога, `if-no-files-found: warn` фиксирует отсутствие без подмены причины.

9 Diagnostic artifacts

Лог удобен для последовательности событий; структурированный report — для анализа и сравнения. В ЛР8 сохраняются:

- JUnit XML с test node ID и failure;
- JSON проверки данных;
- OpenAPI JSON принимаемой версии.

Artifact имеет срок хранения. Он не заменяет долговременный registry и не должен содержать `.env`, secrets, персональные данные или полный закрытый dataset.

Попросите открыть artifact и проверить содержимое. Скриншот вкладки Actions не доказывает, что нужный test действительно выполнялся.

10 Concurrency и устаревший результат

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

После нового push предыдущий run той же ветки уже не проверяет HEAD. Его отмена уменьшает очередь. В group входит ref, поэтому runs разных веток не отменяют друг друга.

Отменённый run не равен failed run: он не подтверждает дефект кода. Для индивидуальной сдачи нужен контролируемый отказ конкретного gate.

11 Cache и artifact — не одно и то же

Cache ускоряет повторное получение зависимостей и может быть удалён без потери результата. Artifact сохраняет выход run для человека или следующего процесса.

Нельзя считать cache источником происхождения модели. Для release нужны registry, digest и manifest. Нельзя также кэшировать секреты или полагаться на cache для корректности.

12 Версии Actions

Официальные actions используют major tags, например `actions/checkout@v7`. Major tag удобен для получения совместимых исправлений, но может перемещаться. Полный commit SHA неизменяем и сильнее для supply-chain контроля.

Для учебного шаблона преподаватель проверяет major versions перед семестром. Для критичного production workflow организация может закреплять action по SHA и обновлять его управляемым ботом.

13 Диагностика отказа

На красной странице сначала найдите первый failed step. Последующие steps часто имеют status skipped и не являются причиной.

Алгоритм:

1. проверить event, branch и SHA;
2. отличить failure от cancellation;
3. найти первую неуспешную команду;
4. прочитать exit code и первую содержательную ошибку;
5. открыть report или annotation;
6. воспроизвести команду локально;
7. сформулировать корневую причину;
8. исправить отдельным commit;
9. сопоставить новый run с исправлением.

Кнопка Re-run проверяет нестабильность среды. Она не исправляет детерминированный дефект.

14 Branch protection

CI становится gate только тогда, когда политика ветки требует его успешного завершения. Для `main` нужны pull request, review другого участника, разрешённые обсуждения и required checks `quality` и `container`.

Имена checks должны быть уникальными. Два workflows с одинаковым job name создают неоднозначность. Обход администратором разрушает учебное доказательство, если не оформлен как исключение с причиной.

15 Контролируемый отказ как учебный эксперимент

Каждый студент заранее выбирает один обратимый дефект, прогнозирует failed step, создаёт commit, фиксирует красный run, затем исправляет отдельным commit.

Подходящие варианты: Ruff violation, broken test, invalid data, изменение OpenAPI или ошибка Dockerfile. Ручная отмена, выключение workflow и общий дефект на команду не принимаются.

Эксперимент показывает способность диагностировать систему. Финальный tag содержит только исправленное состояние, но история отказа сохраняется.

16 Типичные антишаблоны

Антишаблон	Последствие	Исправление
`continue-on-error` у обязательного test	красный риск становится зелёным	убрать или оформить явное исключение
secrets доступны PR из fork	утечка полномочий	безопасный event и минимальные permissions
mutable `latest`	нет связи image с commit	SHA/tag/digest
один огромный shell step	плохая диагностика	именованные steps по контрактам
только screenshots	нет машиночитаемого evidence	artifacts и ссылки на run
re-run вместо fix	дефект остаётся	отдельный исправляющий commit

17 Демонстрационный сценарий

1. Открыть workflow и предсказать порядок steps.

2. Показать permission и concurrency group.
3. Ввести безопасный Ruff defect в ветке.
4. Найти failure и annotation.
5. Исправить отдельным commit.
6. Скачать artifact зелёного run.
7. Показать блокировку merge при красном check.
8. Сопоставить финальный tag с SHA успешного run.

18 Вопросы аудитории

1. Почему успешный run старого commit не разрешает merge нового?
2. Чем cancellation отличается от failure?
3. Когда `if: always()` полезен и чего он не меняет?
4. Почему `contents: write` является риском?
5. Чем cache отличается от artifact?
6. Почему container build расположен после tests?

19 Связь с ЛР8 и ЛР9

В ЛР8 студенты превращают проверки в обязательное правило интеграции. В ЛР9 тот же принцип переносится внутрь ML-workflow: стадии получают состояния, параметры, retries и отдельные отчёты. CI проверяет код flow, а оркестратор наблюдает конкретные запуски обработки.

20 Официальные источники

- GitHub workflow syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- GitHub concurrency: https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency
- GitHub workflow artifacts: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts
- GitHub protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
- GitHub secure use reference: https://docs.github.com/en/actions/reference/security/secure-use
- actions/checkout releases: https://github.com/actions/checkout/releases
- actions/setup-python releases: https://github.com/actions/setup-python/releases

Глава 9. Реестр моделей и управляемый выпуск

Курс: «Управление процессами машинного обучения MLOps» Продолжительность: 2 академических часа Связанная работа: ЛР10 «MLflow Registry, release manifest и rollback» Актуализация: 20 октября 2026 года

Цель лекции

Показать, как превратить удачный эксперимент в управляемый объект выпуска. После занятия студент должен различать run, logged model, registered model и model version; применять aliases и tags вместо устаревших stages; связывать версию модели с данными, кодом, проверками и решением человека; проектировать promotion и rollback без переобучения модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От run к выпуску	Разобрать риск «лучшая метрика означает production»
10–24 мин	Объекты MLflow Registry	Сопоставить run, logged model, registered model и version
24–38 мин	Signature, environment и происхождение	Показать минимальный проверяемый комплект версии
38–52 мин	Aliases и tags	Переназначить `candidate` и `champion`
52–65 мин	Quality gate и approval	Отделить автоматическую проверку от решения о выпуске
65–76 мин	Release manifest	Проследить commit, data hash, run и model version
76–84 мин	Rollback	Вернуть alias на предыдущую версию без retraining
84–90 мин	Связь с ЛР10	Назначить индивидуальные сценарии и evidence

1. Почему артефакта модели недостаточно

Файл `model.joblib` отвечает только на вопрос «какие байты сохранены». Он не сообщает, на каких данных и каким кодом выполнено обучение, какие параметры использованы, какой контракт входа ожидается, какие проверки пройдены и кто разрешил выпуск. Копирование файла под именем `final_v7_really_final.joblib` не создаёт процесс управления версиями.

Начните с инцидента: сервис загрузил новый файл, метрика в offline-отчёте выросла, но входной schema изменился. Без signature и связи с run команда не может быстро доказать совместимость. Без предыдущей версии и атомарного указателя rollback превращается в поиск резервной копии.

2. Четыре разных объекта

Объект	Что фиксирует	Типичный идентификатор
Experiment run	один запуск, параметры, метрики и артефакты	`run_id`
Logged model	модель внутри run с flavor, signature и environment	`runs:/<run_id>/model`
Registered model	устойчивое логическое имя продукта	`flight-delay-classifier`
Model version	неизменяемая запись о конкретной модели	имя + номер версии

Один run может содержать несколько артефактов. Один registered model может иметь много версий. Номер версии не означает качество: версия 12 может быть хуже версии 8. Поэтому выбор обслуживаемой версии выражается policy и alias, а не максимальным номером.

3. Tracking и Registry решают разные задачи

Tracking помогает сравнивать эксперименты. Registry управляет идентичностью и жизненным циклом выпуска. Переход между ними — отдельное решение: модель должна пройти проверки, получить описание происхождения и быть зарегистрирована. Не регистрируйте каждый пробный run автоматически: реестр станет ещё одним неструктурированным каталогом.

Для локальной учебной работы MLflow Registry требует database-backed backend store. Используйте SQLite URI и один локальный server или прямой клиент. Каталог артефактов и база метаданных являются разными частями; резервная копия только одного из них неполна.

4. Model signature как исполнимый контракт

Signature описывает ожидаемые входы и выходы модели. Она не заменяет предметный data contract, потому что обычно не выражает момент доступности признака, источник, допустимое запаздывание или бизнес-смысл. Но signature позволяет обнаружить несовместимые имена и типы на границе inference.

Покажите цепочку:

1. взять репрезентативный `input_example` без персональных данных;
2. получить прогноз тем же объектом модели;
3. вызвать `infer_signature(inputs, outputs)`;
4. сохранить signature вместе с logged model;
5. на загрузке выполнить smoke prediction.

Отсутствующая signature — стоп-фактор выпуска в ЛР10.

5. Environment и воспроизводимость

MLflow model содержит сведения о среде, но они не отменяют lock-файл и контейнер. Версия Python, библиотек, системные зависимости и CPU-инструкции могут влиять на загрузку и результат. Для учебного выпуска достаточно связать model version с `pyproject.toml`, commit SHA и результатом CI; в следующих работах manifest дополнится image digest.

Запретите использовать «у меня загрузилось» как критерий. Приёмочный smoke test должен загрузить модель по registry URI и выполнить прогноз на фиксированном примере.

6. Почему stages больше не используются

MLflow model stages `Staging`, `Production` и `Archived` deprecated с MLflow 2.9 и должны быть заменены aliases и tags. Fixed stages смешивали среду, статус проверки и deployment pointer в одном поле.

В новом подходе:

- alias `candidate` указывает версию, рассматриваемую для выпуска;
- alias `champion` указывает текущую обслуживаемую версию;
- tag `validation_status=passed` фиксирует результат автоматической проверки;
- tag `release_status=approved` фиксирует решение;
- отдельные registered model names могут представлять разные среды при необходимости разграничения доступа.

Не показывайте `transition_model_version_stage()` как рекомендуемый API. Он допустим только в разборе миграции старых материалов.

7. Alias — изменяемый указатель

Model version неизменяема, alias изменяем. URI `models:/flight-delay-classifier@champion` позволяет сервису запрашивать выбранную версию без изменения кода. Это удобно, но создаёт ответственность: переназначение alias является release-операцией и должно оставлять аудит.

Обсудите два режима загрузки:

- разрешить alias только при старте процесса и записать resolved version в лог;
- разрешить периодическое обновление, но тогда нужны атомарная замена, health check и откат.

Alias не заменяет immutable reference в manifest. В manifest сохраняют конкретный номер версии, а alias описывает текущее решение.

8. Tags описывают состояние, но не доказывают его

Tag — метаданные. Любой пользователь с правом записи может поставить `validation_status=passed`. Поэтому tag должен ссылаться на evidence: run ID, отчет проверки, commit и CI run. В учебном проекте значение принимается только вместе с воспроизводимым JSON-отчётом.

Рекомендуемый минимум version tags:

- `validation_status`: `pending`, `passed` или `failed`;
- `source_commit`: полный Git SHA;
- `data_sha256`: hash подготовленных данных;
- `quality_report`: путь или URI отчёта;
- `release_status`: `candidate`, `approved`, `rolled_back`;
- `owner_team`: код команды.

Не помещайте в tags secrets, персональные данные и длинные отчёты.

9. Candidate/champion workflow

Базовая последовательность:

1. Prefect flow завершается успешно и создаёт `run-summary.json`.
2. Модель логируется в MLflow с signature и input example.
3. Создаётся новая model version.
4. Version получает tags происхождения и alias `candidate`.
5. Автоматический gate проверяет метрики, signature, загрузку и smoke prediction.
6. Другой участник выполняет review evidence.
7. Уполномоченный участник переназначает `champion` на candidate.
8. Manifest сохраняет новую и предыдущую champion version.

Важно: candidate может обновляться много раз, champion — только после явного решения.

10. Quality gate — это решение по контракту

Gate должен иметь входы, пороги и объяснимый результат. Недостаточно проверить только ROC AUC. Для ЛР10 минимум включает:

- успешный upstream flow;
- test ROC AUC не ниже утверждённого порога;
- наличие signature и input example;
- загрузку версии из Registry;
- smoke prediction;
- совпадение data hash и commit с evidence;
- отсутствие secrets в manifest.

Порог задаётся в конфигурации или команде запуска и отражается в отчёте. Изменение порога — отдельное reviewable решение, а не ручная правка результата.

11. Автоматизация не отменяет approval

Зелёный gate означает «объявленные проверки пройдены», а не «модель безопасна во всех условиях». Выпуск может учитывать latency, стоимость, grouped metrics, юридические ограничения и доступность rollback. Поэтому отделите машинное доказательство от человеческого решения.

В учебном процессе автор candidate не должен единолично выполнять approval. Reviewer проверяет evidence, а имя или GitHub-логин approving participant фиксируется в manifest. Для команды из двух человек допускается перекрёстное review.

12. Release manifest

Manifest — машинно читаемый снимок решения о выпуске. Минимальная структура:

```
{
  "schema_version": "1.0.0",
  "model_name": "flight-delay-classifier",
  "model_version": "3",
  "aliases": ["candidate", "champion"],
  "previous_champion_version": "2",
  "mlflow_run_id": "...",
  "source_commit": "...",
  "data_sha256": "...",
  "quality_gate": {"passed": true, "minimum_roc_auc": 0.7},
  "decision": {"status": "approved", "approved_by": "student-login"}
}
```

Manifest не должен содержать токен доступа, содержимое dataset или локальный абсолютный путь преподавателя. Пути внутри репозитория допустимы; внешние артефакты указываются проверяемым URI.

13. Promotion как транзакция

Логически promotion состоит из проверки preconditions, получения текущего champion, переназначения alias и записи manifest. Между этими шагами возможен отказ. В production нужен механизм блокировки или optimistic concurrency; в учебной работе достаточно запретить параллельные promotion и проверить, что candidate не изменился между review и командой выпуска.

Перед переназначением сохраните номер предыдущего champion. После переназначения снова прочитайте alias и убедитесь, что он указывает ожидаемую версию. Только затем manifest получает `status=approved`.

14. Rollback без переобучения

Rollback не создаёт новую модель и не повторяет обучение. Он переназначает `champion` на заранее известную проверенную version. Команда должна заранее знать target version и причину отката.

Порядок:

1. зафиксировать инцидент или контролируемый учебный сценарий;
2. прочитать текущий champion;
3. проверить существование target version;
4. переназначить alias;
5. выполнить load и smoke prediction;
6. создать rollback manifest;
7. пометить отозванную версию tag, не удаляя историю.

Удаление плохой версии до разбора разрушает аудит.

15. Контролируемый отказ

Для демонстрации задайте недостижимый порог ROC AUC или удалите обязательное поле из копии summary. Ожидается, что новая version не получит `champion`, текущий champion останется прежним, а failure report объяснит точную причину.

Не имитируйте отказ удалением общей MLflow database. Сценарий должен быть обратимым и не разрушать evidence других участников.

16. Имена и области ответственности

Имя registered model описывает продукт и задачу, например `team-03-flight-delay-classifier`. Не включайте случайный run ID в имя: для этого существуют versions. Для разных команд нужен namespace, чтобы они не переназначали aliases друг друга.

У владельца модели, reviewer и release operator разные задачи. В команде 2–5 человек роли можно совмещать, но author и reviewer конкретного изменения различаются. Роли ротируются относительно ЛР9.

17. Диагностика типичных отказов

Симптом	Вероятная причина	Проверка
Registry недоступен	backend не database-backed или неверный URI	вывести tracking/registry URI
Version создана, модель не грузится	потерян artifact store или environment	открыть source и загрузить exact version
Alias указывает не туда	конкурентный promotion или ручная правка	прочитать version by alias и аудит
Gate passed, manifest failed	сериализация пути или отсутствующее поле	проверить schema manifest
Rollback повторяет проблему	target не прошёл smoke в текущей среде	загрузить exact target до alias update

Учите начинать с exact identifiers: model name, version, run ID и commit. Скриншот страницы Registry не заменяет эти значения.

18. Демонстрационный сценарий

1. Открыть два успешных Prefect run summary.
2. Зарегистрировать их как две версии одного model name.
3. Показать version tags и source run.
4. Назначить лучшей версии alias `candidate`.
5. Выполнить gate и smoke prediction.
6. Назначить `champion`, сохранив previous champion.
7. Загрузить `models:/<name>@champion` и exact version.
8. Выполнить rollback на предыдущую version.
9. Сопоставить два manifest и журнал действий.

19. Вопросы аудитории

1. Почему максимальный номер version не означает лучшую модель?
2. Чем alias отличается от tag?
3. Почему в manifest сохраняется exact version, если сервис использует alias?
4. Что произойдёт, если candidate переназначили после review?
5. Какие проверки нельзя выразить одной метрикой?
6. Почему rollback не должен запускать retraining?
7. Какие данные запрещено помещать в tags и manifest?

20. Связь с ЛР10 и ЛР11

В ЛР10 команда создаёт локальный Registry, регистрирует минимум две версии, применяет `candidate` и `champion`, сохраняет positive и rejected gate reports, формирует release manifest и демонстрирует rollback. Каждый студент защищает собственную version, проверку, review или rollback-сценарий.

В ЛР11 resolved model version станет обязательным label/полем структурированного лога и позволит связать метрики сервиса с конкретным выпуском.

21. Официальные источники

- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/
- MLflow Registry tutorial: https://mlflow.org/docs/latest/ml/model-registry/tutorial
- MLflow Client API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html
- MLflow 3 migration guide: https://mlflow.org/docs/latest/ml/mlflow-3/
- MLflow package releases: https://pypi.org/project/mlflow/

Глава 10. Наблюдаемость ML-сервиса

Курс: «Управление процессами машинного обучения MLOps» Продолжительность: 2 академических часа Связанная работа: ЛР11 «Prometheus, Grafana, структурированные логи и alert» Актуализация: 20 октября 2026 года

Цель лекции

Научить проектировать наблюдаемость как проверяемый контракт эксплуатации, а не как набор красивых графиков. После занятия студент должен различать telemetry и evidence, выбирать ограниченные labels, вычислять request rate, error ratio и latency quantiles, связывать запрос с версией модели, формулировать SLI/SLO, создавать alert с окном и runbook и объяснять границу между мониторингом сервиса и мониторингом качества модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Наблюдаемость и диагностика	Сравнить dashboard и ответ на вопрос «почему отказ?»
10–23 мин	Метрики, логи, traces	Разделить сигналы и их роли
23–37 мин	RED и ML identity	Спроектировать rate, errors, duration и model info
37–50 мин	Prometheus и labels	Показать route template и cardinality explosion
50–62 мин	PromQL и histogram	Вычислить rate, 5xx ratio и p95
62–72 мин	SLI, SLO и error budget	Перевести ожидание в окно и долю хороших событий
72–82 мин	Alert и runbook	Разобрать pending, firing, resolved и действие
82–90 мин	ЛР11	Назначить нагрузку и индивидуальные инциденты

1. Наблюдаемость — способность задавать новые вопросы

Мониторинг обычно отвечает на заранее выбранные вопросы: доступен ли сервис, сколько запросов, превышен ли порог. Наблюдаемость шире: по внешним сигналам инженер должен объяснить внутреннее состояние и исследовать ситуацию, которую не описывал заранее.

Начните с вопроса: «После выпуска версии 7 выросли ошибки только у `/predict`; какие запросы, экземпляры и model version затронуты?» Один график CPU не отвечает. Нужны метрики для масштаба, структурированные логи для конкретного события и, в распределённой системе, traces для пути запроса.

2. Telemetry не равно evidence

Telemetry — поток измерений. Evidence — сохранённое доказательство конкретного вывода: PromQL-запрос с окном, снимок значений, request ID, JSON-log, версия dashboard и связанный release manifest. Скриншот без времени, запроса и версии конфигурации слаб.

В ЛР11 evidence создаётся воспроизводимой нагрузкой и JSON-report. Каждый студент должен связать свой запрос с логом и resolved model version.

3. Три основных сигнала

Сигнал	Сильная сторона	Ограничение
Metrics	дешёвая агрегация и alerting	теряют детали события
Logs	контекст конкретного события	поиск и хранение дороже
Traces	путь запроса через компоненты	требуют propagation и sampling

В обязательной части курса реализуются metrics и JSON logs. Traces рассматриваются как расширение: request ID не является trace ID, но уже создаёт учебную корреляцию.

4. RED для HTTP-сервиса

RED — Rate, Errors, Duration:

1. Rate: запросов в секунду по route template.
2. Errors: доля 5xx и отдельно наблюдаемые 4xx.
3. Duration: распределение времени ответа, а не только среднее.

4xx не всегда ошибка сервиса: 422 может означать корректно отклонённый невалидный запрос. Однако рост 422 может указывать на несовместимый клиентский контракт. Поэтому dashboard показывает статусы, а SLO явно определяет, какие события считаются плохими.

5. ML identity как эксплуатационный сигнал

Одинаковый endpoint может обслуживаться разными версиями модели. Лог и метрика `model_info` должны сообщать `model_name`, `model_version` и `source_commit`. Это связывает ЛР11 с release manifest ЛР10.

Не добавляйте model version к каждой высокочастотной метрике без оценки cardinality. Для учебного сервиса одна активная version допустима; при большом числе моделей identity лучше хранить в отдельной info metric и логах.

6. Метрики Prometheus

Четыре типа:

- Counter только растёт, кроме reset при рестарте процесса.
- Gauge может расти и уменьшаться.
- Histogram распределяет наблюдения по bucket и создаёт `_bucket`, `_sum`, `_count`.
- Summary вычисляет quantiles на клиенте и хуже агрегируется между экземплярами.

Для запросов используйте counter и histogram. `rate()` применяют к counter до агрегации, чтобы корректно учитывать resets.

7. Имена и единицы

Имя должно содержать предмет, единицу и тип, когда это принято: `http_request_duration_seconds`, а не `latency_ms_value`. Базовые единицы Prometheus — seconds, bytes и безразмерные ratios.

Описание метрики отвечает на вопрос, что измеряется. Метрика не должна менять смысл между версиями, сохраняя имя.

8. Labels и cardinality

Каждая уникальная комбинация labels — отдельный time series. Безопасные labels имеют ограниченное множество: `method`, route template, status class, controlled model version. Опасные labels: `request_id`, user ID, полный URL, timestamp, текст ошибки, prediction probability.

Сравните:

```
route="/predict"                # ограниченно
path="/users/846291/predict"    # потенциально безгранично
```

Request ID помещается в JSON-log, а не в metric label. Официальная практика Prometheus рекомендует расследовать dimensions с потенциальной cardinality порядка сотен и выше.

9. Middleware как граница измерения

HTTP middleware видит метод, route, статус и полную продолжительность. Он должен измерять и успешные, и отклонённые запросы. Route template обычно становится известен после маршрутизации, поэтому label читается из resolved route, а неизвестные пути объединяются в `unmatched`.

Нельзя использовать сырой `request.url.path`: `/items/1`, `/items/2` создадут разные series.

10. PromQL: request rate

```
sum by (route) (rate(http_requests_total[5m]))
```

Сначала `rate`, затем `sum`. Окно 5 минут сглаживает шум, но медленнее реагирует. На малом учебном потоке короткие окна могут возвращать нестабильные значения; генератор нагрузки делает наблюдение воспроизводимым.

11. PromQL: error ratio

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
clamp_min(sum(rate(http_requests_total[5m])), 0.001)
```

Знаменатель защищён от деления на ноль. Для production лучше отдельно обработать отсутствие трафика: нулевая error ratio при отсутствии запросов не доказывает здоровье сервиса.

12. Histogram и p95

```
histogram_quantile(
  0.95,
  sum by (route, le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Классические histogram buckets cumulative; label `le` обязателен при агрегации. p95 — оценка внутри bucket. Она не означает, что каждый запрос быстрее порога, и зависит от выбранных границ bucket.

Среднее скрывает хвост. p95 без request rate также вводит в заблуждение: один медленный запрос в редком потоке и массовая деградация — разные ситуации.

13. Structured JSON logs

Минимальное событие запроса:

```
{
  "event": "http_request",
  "request_id": "lr11-student-01-0007",
  "method": "POST",
  "route": "/predict",
  "status": 200,
  "duration_ms": 12.4,
  "model_name": "flight-delay-classifier",
  "model_version": "3"
}
```

Не логируйте признаки, токены, персональные данные, полный request body и prediction probability по умолчанию. Structured означает стабильные ключи и типы, а не строку, похожую на JSON.

14. Correlation

Клиент может передать безопасный `X-Request-ID`; сервис ограничивает длину и генерирует UUID при отсутствии. ID возвращается в response header и body ошибки. По нему студент связывает HTTP-response и JSON-log.

Доверять произвольному ID как секрету нельзя. Его используют только для корреляции, экранируют при выводе и не помещают в labels.

15. SLI и SLO

SLI — измеряемый показатель. SLO — целевое значение SLI за окно.

Пример учебного SLO:

```
За 15 минут не менее 99% валидных запросов /predict завершаются без 5xx,
а p95 latency не превышает 500 ms при не менее 50 запросах.
```

Определение включает population, хорошие события, порог, окно и минимум трафика. «Сервис должен быть быстрым» не является SLO.

16. Error budget

Для availability SLO 99% допустима 1% плохих событий в выбранном окне. Error budget помогает обсуждать риск изменений, но на короткой ЛР статистика мала. Используйте его как модель решения, не имитируйте production SLA.

Покажите, что 1 ошибка из 20 запросов даёт 95%, а из 10 000 — 99.99%. Контекст объёма обязателен.

17. Alert как проверяемая гипотеза

Alert rule содержит выражение, длительность `for`, severity и annotations. Состояния:

1. inactive — условие ложно;
2. pending — условие истинно, но время `for` не прошло;
3. firing — условие сохраняется достаточно долго;
4. resolved — условие перестало выполняться.

Без `for` краткий всплеск немедленно активирует alert. Слишком длинный `for` скрывает реальный инцидент. Значение выбирают по стоимости ложного и пропущенного сигнала.

18. Alert должен вести к действию

Alert без владельца и runbook создаёт шум. Аннотация содержит краткое описание и ссылку на runbook. Runbook задаёт первые проверки, безопасные действия, критерий восстановления и требуемое evidence.

Не советуйте «перезапустить всё». Сначала фиксируются состояние, версия выпуска, target status и логи; затем выполняется минимальное обратимое действие.

19. Dashboard as code

Grafana datasource и dashboard provisioned из файлов. Это даёт review, одинаковый старт команд и восстановление на чистой машине. Ручные правки UI могут быть перезаписаны provisioning source и не принимаются без экспорта в Git.

Минимальные панели ЛР11: request rate, 5xx ratio, p95 по route, prediction outcomes, `up` и resolved model. Каждая панель содержит единицу, окно и осмысленный title.

20. Граница с мониторингом модели

ЛР11 измеряет service health и identity. Распределение prediction outcomes полезно, но без фактических labels не доказывает качество модели. Drift, delayed labels, coverage и оконные quality metrics рассматриваются в ЛР12.

Не называйте рост доли `delayed` деградацией модели без анализа входного распределения и фактических исходов.

21. Демонстрационный сценарий

1. Запустить app, Prometheus и Grafana через Compose.
2. Проверить targets и provisioned dashboard.
3. Создать 100 запросов с 10% ожидаемых 422.
4. Выполнить PromQL для rate, 5xx ratio и p95.
5. Найти JSON-log по request ID.
6. Показать model name/version из response, log и `model_info`.
7. Остановить app и наблюдать `up=0`, pending и firing.
8. Выполнить runbook и восстановить сервис.

22. Вопросы аудитории

1. Почему request ID опасен как label?
2. Почему 422 не включён в 5xx ratio?
3. Зачем `rate()` выполнять до `sum()`?

4. Почему p95 является оценкой?
5. Что доказывает `model_info`, а что не доказывает?
6. Когда alert должен иметь `for`?
7. Почему dashboard без provisioned source слаб как evidence?
8. Почему prediction distribution не равна model quality?

23. Связь с ЛР11 и ЛР12

В ЛР11 команда создаёт воспроизводимый observability stack, генерирует нагрузку, проверяет PromQL, JSON-log correlation, alert lifecycle и runbook. Каждый участник отвечает за отдельную metric/query/panel/alert/log/runbook часть и собственный сценарий.

В ЛР12 те же окна и release identity будут использованы для drift, delayed labels, coverage и оценки качества модели.

24. Официальные источники

- Prometheus instrumentation practices: https://prometheus.io/docs/practices/instrumentation/
- Prometheus query functions: https://prometheus.io/docs/prometheus/latest/querying/functions/
- Prometheus alerting rules: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/
- Grafana provisioning: https://grafana.com/docs/grafana/latest/administration/provisioning/
- Grafana Docker installation: https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- Python Prometheus client releases: https://pypi.org/project/prometheus-client/

Часть III. Эксплуатация, управление рисками и развитие

Глава 11. Мониторинг данных и качества модели

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР11 «Наблюдаемость ML-сервиса», ЛР12 «Drift и отложенное качество»

1. Цель лекции

Научить студентов строить доказательный контур мониторинга модели: отделять исправность сервиса от качества прогнозов, сравнивать согласованные временные окна, учитывать задержку фактических меток и принимать управляемое решение о реакции. После лекции студент должен уметь объяснить, почему drift является сигналом расследования, но не доказательством необходимости retraining.

2. Результаты обучения

Студент сможет:

- различать data drift, prediction drift, concept drift и data quality incident;
- выбирать reference и current окна без утечки будущего;
- объяснять PSI и ограничения статистических тестов;
- вычислять label coverage только по зрелым прогнозам;
- считать качество по model version и временному окну;
- отличать отсутствие доказательств от доказательства отсутствия проблемы;
- формулировать policy и decision gate;
- связывать отчёт с release manifest, runbook и postmortem.

3. Место в сквозном проекте

В ЛР11 команда доказала, что API доступен, измерим и связан с model version. Это ещё не подтверждает полезность модели. Сервис может отвечать быстро и без 5xx, но выдавать устаревшие прогнозы. ЛР12 добавляет второй слой: характеристики входов, распределение прогнозов и качество после появления фактической задержки рейса.

На доске удобно провести границу: service health отвечает на вопрос «работает ли система», model health — «сохраняет ли результат ожидаемое качество и применимость».

4. Единица наблюдения

Минимальная запись связывает `request_id`, `event_time`, признаки, probability, решение, `model_version` и `source_commit`. Позже к ней присоединяются `label_time` и фактический исход. Полезно подчеркнуть: без устойчивого идентификатора и времени нельзя надёжно соединить прогноз с меткой, а без версии модели нельзя понять, какой release деградировал.

Сырые признаки не обязаны попадать в оперативные логи. Их хранение должно соответствовать политике приватности. Для учебного кейса используются обезличенные `DepHour` и `IsWeekend`.

5. Data quality incident не равен drift

Если `DepHour` внезапно принимает значение 900, сначала нарушен контракт диапазона. Если столбец перестал поступать — это schema/data quality incident. Drift описывает изменение допустимого распределения, например рост доли вечерних рейсов. Сначала применяются контрактные проверки, затем статистический мониторинг. Иначе поломка источника будет ошибочно названа естественным изменением.

6. Четыре разных сигнала

Сигнал	Что изменилось	Нужны labels	Что доказывает
Data drift	распределение X	нет	входы отличаются от reference
Prediction drift	распределение score или класса	нет	поведение модели изменилось
Concept drift	условное распределение Y при X	обычно да	связь входов и результата изменилась
Quality degradation	метрики по Y и prediction	да	результат хуже принятого порога

Один сигнал не подменяет другой. Prediction drift может быть нормальной реакцией на сезонность, а качество может ухудшиться без заметного marginal drift отдельных признаков.

7. Reference-окно

Reference должно быть связано с решением: обучающая часть, validation перед выпуском или стабильное production-окно champion-модели. Нельзя незаметно заменять reference после каждого предупреждения — так система «забывает» исходную норму. Версия reference, границы времени, фильтры и model version фиксируются в policy и отчёте.

8. Current-окно

Current-окно выбирается по частоте решений и объёму потока: сутки, неделя или последние N событий. Слишком короткое окно шумит, слишком длинное скрывает резкое изменение. Для каждой проверки нужны начало, конец, число строк и правило включения. Окна сравнивают одинаковую семантику признаков.

9. Хронология и утечка

Все границы строятся по event time, а не по удобному порядку строк. Reference не должен содержать события после current. При backfill отчёт сохраняет явный `as_of`, иначе результат невозможно воспроизвести. Фактическая задержка рейса никогда не используется как входной признак.

10. Population Stability Index

Для категорий или bins используются доли reference `p_i` и current `q_i`:

```
PSI = sum((q_i - p_i) * ln(q_i / p_i)).
```

Нулевые доли сглаживаются малым epsilon. Границы числовых bins строятся только по reference, иначе current влияет на измерительную шкалу. В учебной policy значения 0.10 и 0.25 служат warning и alert, но это договорные пороги, а не универсальный закон.

11. Ограничения PSI

PSI зависит от binning, размера окна и редких категорий. Он не сообщает причину и не измеряет бизнес-ущерб. При большом потоке даже небольшое отличие может быть устойчивым, но незначимым для решения; при малом потоке оценка нестабильна. Поэтому отчёт всегда содержит объём окна и метод.

12. Статистическая и практическая значимость

P-value отвечает не на вопрос о величине эффекта. При большом N тест способен найти микроскопическое отличие. Обсудите с аудиторией пару: статистический тест плюс effect size и инженерный threshold. До расчёта команда должна определить, какое изменение способно повлиять на решение или риск.

13. Многократные проверки

Если отслеживаются десятки признаков, вероятность случайного срабатывания растёт. Возможные меры: коррекция множественных сравнений, требование доли drifting features, повторное подтверждение на следующем окне или приоритизация ключевых признаков. Один случайный флаг не должен автоматически создавать новую champion-модель.

14. Prediction drift

Распределение probabilities и доля положительного класса доступны сразу. Это ранний сигнал, но он смешивает изменение входов и реакцию модели. В отчёте его связывают с model version и threshold. После изменения threshold доля положительных решений может измениться без изменения самой модели.

15. Почему labels задерживаются

Прогноз строится за 60 минут до вылета, а фактическая задержка прибытия известна после события. Метка может дополнительно задержаться из-за ETL. В кредитном риске или обслуживании оборудования лаг измеряется неделями и месяцами. Поэтому «нет метки сейчас» не означает ошибку прогноза.

16. Mature cohort

Пусть ожидаемая задержка метки равна 24 часам, а отчёт строится в момент `as_of`. В знаменатель coverage входят только прогнозы с `event_time <= as_of - 24h`. Более свежие события ещё не обязаны иметь метку и исключаются. Это центральное правило ЛР12.

17. Label coverage

`coverage = matched mature labels / eligible mature predictions.`

Coverage указывается вместе с числами числителя и знаменателя. Низкий coverage может быть вызван ошибкой join, задержкой источника, несовпадающим часовым поясом или выборочным поступлением меток. Считать F1 на небольшой смещённой подвыборке опасно даже при технически корректной формуле.

18. Join прогноза и метки

`request_id` должен быть уникальным в обоих журналах. Join проверяется как one-to-one. Время хранится в UTC. Повторная метка не должна молча дублировать прогноз и менять вес наблюдения. Для исправления истории применяется версионированный backfill, а не удаление неудобных строк.

19. Оконные метрики качества

После достижения минимального coverage рассчитываются ROC AUC, precision, recall и F1 для одной model version и одного окна. Если в mature cohort представлен только один класс, ROC AUC не определён: статус должен быть `insufficient_evidence`, а не ноль и не успех.

20. Сегменты

Общая метрика может скрывать провал сегмента: будние/выходные, часы суток, аэропорты или регионы. Сегмент включается только при достаточном N и допустимости обработки данных. Нельзя публиковать чувствительные малые группы. В ЛР12 обязателен хотя бы один безопасный учебный сегмент.

21. Decision policy

Полезная policy содержит окна, minimum rows, PSI thresholds, expected label delay, minimum coverage, quality thresholds, владельца и требование review. Пример действий:

- `hold` — доказательств недостаточно;
- `investigate` — drift без подтверждённой деградации;
- `incident_review` — качество упало без очевидного drift;
- `retraining_review` — drift и quality breach подтверждены;
- `no_action` — пороги не нарушены.

22. Почему нет automatic retraining

Переобучение на повреждённых или неполных labels закрепит дефект. Даже корректный candidate должен пройти validation, Registry gates и review ЛР10. Автоматизировать можно расчёт сигнала и создание issue; продвижение champion остаётся управляемым решением с rollback.

23. Runbook и postmortem

Runbook отвечает, что делать сейчас: проверить окна, контракт, coverage, model version и источник labels. Postmortem отвечает, почему инцидент произошёл, каков эффект и какое изменение предотвратит повторение. Фраза «переобучили модель» без анализа первопричины недостаточна.

24. Evidently и прозрачная реализация

Evidently предоставляет Reports, Presets, тесты и экспорт HTML/JSON. На 20 октября 2026 года в материалах фиксируется ветка `evidently>=0.7.23,<0.8`. Обязательная реализация курса остаётся на pandas/NumPy: студент должен понимать окна, bins, coverage и gate. Evidently используется для сравнения результатов, а версия и конфигурация сохраняются.

25. Демонстрация преподавателя

1. Запустить `python -m mlops_flight_delay.monitoring` на учебных CSV.
2. Показать `eligible_cutoff`, числитель и знаменатель coverage.
3. Изменить `as_of` так, чтобы свежие прогнозы стали mature.
4. Удалить несколько labels и получить `insufficient_evidence`.
5. Показать drift alert при хорошем качестве: решение остаётся `investigate`.

6. Показать сочетание drift и quality breach: появляется `retraining_review`, но не выпуск.

26. Вопросы аудитории

1. Почему свежие прогнозы нельзя считать отсутствующими labels?
2. Чем data quality incident отличается от data drift?
3. Почему bins должны определяться по reference?
4. Когда ROC AUC нельзя вычислить?
5. Может ли качество упасть без data drift?
6. Почему высокий drift не означает обязательное retraining?
7. Какие поля обеспечивают связь с release?
8. Что доказывает coverage и чего оно не доказывает?

27. Связь с ЛР12 и ЛР13

В ЛР12 команда создаёт version-controlled policy, JSON-отчёт, тесты, runbook и индивидуальные сценарии. В ЛР13 те же timestamps используются для point-in-time join признаков. Ошибка времени в feature join способна одновременно создать leakage при обучении и ложный monitoring signal.

28. Официальные источники

- Evidently PyPI: https://pypi.org/project/evidently/
- Evidently API `Report` и `DataDriftPreset`: https://evidentlyai.github.io/evidently/api-reference/v0.7.19/evidently.html
- Evidently presets API: https://evidentlyai.github.io/evidently/api-reference/main/evidently/presets.html
- scikit-learn classification metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

Глава 12. Согласованность признаков и Feature Store

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР12 «Drift и отложенное качество», ЛР13 «Point-in-time признаки»

1. Цель лекции

Научить студентов формировать исторически корректные признаки и предотвращать training-serving skew. Ключевой результат — понимание, что корректность признака определяется не только значением, но также сущностью, временем события, временем наблюдения, доступностью и версией определения.

2. Результаты обучения

Студент сможет:

- различать event time, feature time, available time и processing time;
- объяснить утечку при latest join;
- построить point-in-time join;
- применить entity key и TTL;
- проверить coverage, feature age и уникальность результата;
- разделить offline historical retrieval и online lookup;
- объяснить назначение entity, data source, feature view и feature service;
- доказать согласованность обучения и inference тестами.

3. Место в сквозном проекте

В ЛР2 команда определила момент прогноза — за 60 минут до планового вылета. В ЛР12 появились временные окна и отложенные labels. Теперь эти правила применяются к вычисленным признакам. Например, trailing delay rate аэропорта полезен только в той версии, которая была известна к моменту прогноза, а не рассчитана задним числом на полном дне.

4. Почему обычный merge опасен

`merge` по `airport_id` отвечает только на вопрос о совпадении сущности. Если после него выбрать последнюю строку истории вообще, рейс утром получит статистику, вычисленную вечером. Модель увидит будущее, validation станет завышенной, а online-сервис не сможет воспроизвести такой вход.

5. Четыре времени

Время	Смысл	Пример
Event time	момент прогнозируемого события	08:00 рейс или момент запроса
Feature time	момент, к которому относится значение	статистика на 07:00
Available time	когда значение стало доступно системе	расчёт завершился в 07:30
Processing time	когда pipeline обрабатывает строку	backfill в 12:00 следующего дня

Feature time может быть раньше event time, но available time позже. В этом случае значение всё равно недоступно для исторического решения.

6. Entity key

Entity key определяет объект, для которого выбирается признак: аэропорт, маршрут, самолёт или клиент. Название сущности и физический join key — разные понятия. Ключ должен иметь стабильную семантику, тип и область уникальности. Смена кода аэропорта требует явной миграции.

7. Point-in-time условие

Для entity event допустима feature row, если:

- entity key совпадает;
- `feature_time <= event_time`;
- `available_time <= event_time`;

- `event_time - feature_time <= TTL`;
- схема и версия определения совместимы.

Из допустимых строк выбирается последняя по feature time, затем по available time как tie-breaker.

8. Наивный и корректный join

Наивный baseline выбирает последнее значение для entity без учёта события. Корректный алгоритм строит кандидатов для каждой entity event, отбрасывает будущее и недоступные строки, применяет TTL, а затем выбирает последнюю допустимую. В ЛР13 оба результата сохраняются, чтобы leakage был виден.

9. Available time и backfill

Предположим, статистика относится к 07:00, но из-за задержки источника сформирована в 09:00. Для прогноза в 08:00 она недоступна. Backfill не должен переписывать историю так, будто значение существовало раньше. `available_time` делает эту задержку наблюдаемой.

10. Created timestamp

В системах Feature Store created timestamp помогает выбрать последнюю физическую версию записи при одинаковом event timestamp. В учебном контракте `available_time` одновременно показывает момент наблюдаемости. В production эти понятия могут разделяться; команда документирует семантику.

11. TTL

TTL ограничивает возраст признака. Слишком короткий TTL создаёт пропуски, слишком длинный допускает устаревшие значения. TTL не делает join корректным сам по себе: значение из будущего остаётся запрещённым независимо от возраста. Порог связывается с частотой обновления и риском stale data.

12. Feature age и coverage

Для каждой строки сохраняется `feature_age_hours`. Coverage — доля entity events, для которых все обязательные признаки найдены. Низкий coverage может означать короткий TTL, пропуски сущностей, задержку materialization или ошибку ключа. Заполнение средним без анализа скрывает причину.

13. Дубликаты и tie-breaking

История признаков должна иметь определённую временную идентичность. Если для entity и timestamp существуют две строки, алгоритм обязан иметь правило выбора или отклонить данные. Недетерминированный join разрушает воспроизводимость даже без явной утечки.

14. Label time не является feature time

Фактическая задержка — отложенная цель. Её нельзя превратить в признак того же рейса. Агрегат по прошлым labels допустим только если окно заканчивается до event time и результат уже доступен. Названия `rolling` и `historical` не гарантируют отсутствие leakage.

15. Training-serving skew

Skew возникает, когда обучение и inference используют разные определения, код, источники, timestamp semantics, defaults или версии. Типичный случай: notebook считает trailing average по полному датасету, а сервис получает последний материализованный показатель.

16. Offline retrieval

Historical retrieval принимает entity dataframe с ключами и timestamps и возвращает значения, корректные для каждого исторического момента. Он используется для training и batch scoring. Результат должен быть версионирован или воспроизводим из определения и исходных данных.

17. Online retrieval

Online lookup получает ключ сущности и обычно возвращает последнее материализованное значение с низкой задержкой. Он не предназначен для построения исторической обучающей выборки: сегодня online store уже содержит знания, которых не существовало в прошлом.

18. Offline-online parity

Parity не означает одинаковый storage engine. Она означает одинаковую семантику и значения для одной версии feature definition в сопоставимый момент. Проверяются имена, типы, units, defaults, transformations и допустимая freshness. Различие требует отчёта, а не молчаливого tolerance.

19. Что такое Feature Store

Feature Store — не просто база признаков. Он сочетает registry определений, retrieval интерфейсы, offline/online источники, materialization и governance. Он не исправляет плохую временную модель автоматически: неверный timestamp в source создаст воспроизводимую, но всё равно ошибочную выборку.

20. Feast Entity и Data Source

В Feast `Entity` задаёт доменную сущность и join key. Data Source указывает, где находятся timestamped feature values. Для исторического retrieval entity dataframe содержит ключи и время, по которому выполняется point-in-time join.

21. Feature View

Feature View группирует признаки, source, entities, schema, TTL и metadata. По официальной документации он используется и для historical datasets, и для загрузки/чтения online values. `enable_validation` помогает проверять наличие и типы столбцов, но не заменяет содержательные тесты.

22. Feature Service

Feature Service объединяет набор feature views или выбранных полей для конкретной версии модели. Это полезная граница совместимости: модель запрашивает логический набор признаков, а release manifest может ссылаться на имя service и registry revision.

23. Feast в плане курса

На 20 октября 2026 года stable release Feast — 0.67.0. В РПД Feast является обзорным или демонстрационным инструментом. Локальная установка не является условием допуска. Обязательны прозрачный pandas join, тест leakage и audit JSON. Feast-демо подтверждает перенос понятий.

24. Definitions as code

Entity, source, feature view и feature service хранятся в Git и проходят review. В definitions указываются owner tags, schema и TTL. Изменение семантики создаёт новую совместимую версию, а не тихую замену поля под прежним именем.

25. Тесты корректности

Минимальный набор:

- никакой `feature_time` не позже `event_time`;
- никакой `available_time` не позже `event_time`;
- feature age не превышает TTL;
- одна строка результата на `request_id`;
- наивный fixture действительно содержит leakage;
- корректный fixture выбирает ожидаемое предыдущее значение;
- coverage соответствует policy.

26. Демонстрация преподавателя

1. Запустить модуль `mlops_flight_delay.features`.
2. Открыть `naive.csv` и показать будущие значения.
3. Сравнить с `point-in-time.csv`.
4. Для события в 08:00 показать feature 07:00, доступную только в 09:00, и её исключение.
5. Уменьшить TTL и объяснить падение coverage.

6. Показать Feast definitions без требования поднимать сервис.

27. Вопросы аудитории

1. Почему `feature_time <= event_time` недостаточно?
2. Чем available time отличается от processing time?
3. Почему online lookup нельзя применять к историческому training set?
4. Что доказывает TTL и чего не доказывает?
5. Как обнаружить training-serving skew?
6. Может ли Feature Store гарантировать отсутствие leakage?
7. Зачем Feature Service связывать с model version?
8. Какой результат считается индивидуальным evidence в ЛР13?

28. Связь с ЛР13 и ЛР14

В ЛР13 команда реализует и защищает временную корректность. В ЛР14 определения признаков, владельцы, ограничения и возможное влияние на группы войдут в model card, data card и review ответственного применения.

29. Официальные источники

- Feast feature retrieval: https://docs.feast.dev/getting-started/concepts/feature-retrieval
- Feast feature view: https://docs.feast.dev/getting-started/concepts/feature-view
- Feast concepts: https://docs.feast.dev/getting-started/concepts
- Feast PyPI: https://pypi.org/project/feast/

Глава 13. Документация, безопасность и ответственное применение ML

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР13 «Point-in-time признаки», ЛР14 «Документация, безопасность и Responsible ML»

1. Цель лекции

Показать, как превратить утверждения о модели в проверяемые release evidence: model card, data card, групповые метрики, threat model, проверку лицензий и ответственное решение о выпуске. Студент должен различать наличие документа и подтверждённый контроль.

2. Результаты обучения

Студент сможет:

- описать назначение, пользователей и недопустимые сценарии модели;
- связать model card с конкретной версией данных, кода, модели и threshold;
- оценить качество по группам и учесть малый размер выборки;
- объяснить, почему одна метрика или один gap не доказывает fairness;
- построить простой threat model и определить границы доверия;
- организовать работу с секретами, логами, зависимостями и лицензиями;
- применить Govern–Map–Measure–Manage к учебному release;
- принять решение release, review, stop или rollback с указанием владельца.

3. Место в сквозном проекте

К этому моменту команда умеет воспроизводить данные и модель, выпускать сервис, наблюдать его и контролировать временную корректность признаков. ЛР14 отвечает на следующий вопрос: достаточно ли у нас контекста, доказательств и контроля риска, чтобы разрешить очередной выпуск?

4. Документация как часть системы

Документ полезен, если он version-controlled, проходит review, ссылается на фактические артефакты и может остановить pipeline. Карточка с общими фразами, не связанная с model version и test period, создаёт ложное чувство контроля. Поэтому обязательные разделы проверяются в CI, а содержательное решение остаётся за человеком.

5. Четыре уровня доказательства

1. Утверждение: «модель пригодна для учебного сценария».
2. Измерение: метрика с определённым набором, периодом и threshold.
3. Артефакт: JSON, карточка, commit SHA, model version и логи CI.
4. Решение: кто и на основании какой policy разрешил или остановил выпуск.

Отсутствие любого уровня делает цепочку аудита неполной.

6. Model card

Model card объясняет, что представляет собой модель, для чего она предназначена, как оценивалась и где её использование опасно. Это не рекламный паспорт качества. Карточка должна помогать разработчику, владельцу процесса, reviewer и пользователю принять обоснованное решение.

7. Обязательные разделы model card

Для курса обязательны: краткое описание; назначение; недопустимое использование; модель и данные; оценка качества; групповые метрики; ограничения и риски; мониторинг и реакция; безопасность и конфиденциальность; надзор человека; владелец и журнал изменений.

8. Intended use и out-of-scope

Фраза «прогноз задержки» слишком широка. Нужны момент решения, пользователь, действие и допустимый масштаб. В учебном проекте разрешена демонстрация за 60 минут до вылета. Решения о безопасности, компенсациях, работниках или реальной эксплуатации явно запрещены.

9. Версия и provenance

Model card связывается с commit SHA, model registry version, data/feature contract, test window, threshold и release manifest. Если карточка описывает «последнюю модель», её невозможно проверить после следующего выпуска. Источник данных и право использования входят в provenance наравне с hash.

10. Data card

Data card описывает происхождение, назначение, схему, сбор, качество, временное разбиение, права, хранение и известные ограничения. Контракт отвечает «что машина проверяет», карточка — «что это означает и почему допустимо». Эти документы дополняют, а не заменяют друг друга.

11. Минимизация и хранение данных

Собирать нужно только поля, необходимые для заявленного сценария. Для каждого внешнего источника фиксируются владелец, основание использования, срок хранения, удаление и получатели. Учебный проект использует синтетические данные и не даёт разрешения добавлять реальные персональные сведения.

12. Порог и стоимость ошибок

Threshold преобразует вероятность в действие. Его нельзя выбирать только по максимальному F1: ложноположительная и ложноотрицательная ошибки имеют разную стоимость. Изменение threshold меняет precision, recall, нагрузку на оператора и групповые различия, поэтому считается изменением policy.

13. Зачем нужны групповые метрики

Среднее качество может скрыть сегмент, где модель систематически ошибается. Группы выбираются не потому, что столбец доступен, а по гипотезе риска и предполагаемому воздействию. Сначала фиксируется вопрос, затем набор, метрика и критерий решения.

14. Учебный безопасный сегмент

В курсе используется `IsWeekend`, чтобы отработать вычисление, minimum support и CI gate без обработки чувствительных атрибутов. Это операционный сегмент, не защищённая группа. Результат нельзя называть оценкой дискриминации или переносить на людей.

15. Минимальный размер группы

Метрика на двух строках математически вычислима, но неустойчива и может раскрывать отдельные наблюдения. Policy задаёт minimum group rows. Недостаточная группа получает статус `insufficient_sample`; её значение не публикуется и не заменяется нулём.

16. Что сравнивать

Для каждой достаточной группы сохраняются n, доля положительного класса, доля положительных предсказаний, precision, recall, F1 и ROC AUC, если присутствуют оба класса. Gap имеет смысл только вместе с объёмами, доверительными интервалами, базовыми частотами и стоимостью ошибок.

17. Почему parity не равна fairness

Одинаковый F1 может скрывать разные recall и false-positive rate. Равенство конкретной метрики может противоречить другой цели. Защищённый атрибут может отсутствовать, быть неточным или иметь proxy. Fairness является контекстным социотехническим требованием, а не одним универсальным числом.

18. Решение при нарушении gate

Нарушение порога переводит выпуск в review, а не автоматически доказывает вред. Команда проверяет качество данных, размер и период группы, threshold, стоимость ошибки и альтернативы. Возможные решения: собрать данные, изменить сценарий, ограничить использование, откалибровать модель или остановить выпуск.

19. NIST AI RMF

AI RMF организует работу вокруг четырёх функций. Govern задаёт роли, policy и культуру риска; Map — контекст, участников и возможный вред; Measure — методы и evidence; Manage — приоритеты, реакцию и мониторинг. Govern является сквозной функцией. Это не линейный универсальный чек-лист.

20. Применение RMF к проекту

Govern: CODEOWNERS, policy, владелец и stop authority. Map: пользователи, out-of-scope, данные, границы и затронутые стороны. Measure: метрики, тесты, threat model и completeness audit. Manage: release decision, mitigation, monitoring, incident response и rollback.

21. Threat modeling

Threat model начинает не с продукта-сканера, а с активов, границ доверия, злоумышленника и последствия. Для ML-сервиса активами являются код, данные, модель, registry, credentials, CI и логи. Угрозы включают утечку секрета, poisoning, подмену артефакта, supply-chain и избыточный лог.

22. Секреты

Секрет не хранится в репозитории, `.env.example` содержит только имена и безопасные заполнители. Secret scanning и push protection снижают риск случайного commit, но не заменяют rotation. После утечки секрет сначала отзывают или меняют, затем исправляют историю и источник появления.

23. Безопасные логи

Observability следует принципу allowlist: записываются request ID, route template, status, latency и версия модели, но не сырые признаки, token, пароль или персональные данные. Нельзя сначала собрать всё, а потом надеяться удалить чувствительные поля.

24. Цепочка поставки и лицензии

SBOM отвечает, что вошло в артефакт; сканер — какие известные проблемы найдены; license review — можно ли распространять компонент выбранным способом. Версия и лицензия проверяются по фактическому пакету и официальному источнику. Неизвестный статус блокирует release.

25. Human oversight

Автоматический gate хорошо обнаруживает неполный заголовок или превышенный gap. Он не понимает общественную цену ошибки и законность сценария. Ответственный человек должен иметь контекст, полномочие остановить выпуск и альтернативу модели — например, ручную обработку или отказ.

26. Демонстрация преподавателя

1. Запустить `mlops_flight_delay.governance` на baseline.
2. Показать overall и две группы с n.
3. Удалить строки одной группы и получить `insufficient_sample`.
4. Удалить заголовок model card и показать fail completeness check.
5. Изменить threshold и обсудить метрики/стоимость ошибок.
6. Открыть threat model и проследить одну угрозу до контроля и residual risk.

27. Вопросы аудитории

1. Почему наличие model card не доказывает безопасность?
2. Что изменится при новом threshold?
3. Почему малую группу нельзя публиковать как обычную метрику?
4. Можно ли считать равный F1 доказательством fairness?
5. Чем data contract отличается от data card?
6. Что делать первым после утечки секрета?
7. Зачем threat model содержит остаточный риск?

8. Кто имеет право остановить учебный release?

28. Связь с ЛР14 и ЛР15

В ЛР14 команда создаёт и проверяет release evidence. В ЛР15 эти ограничения войдут в нагрузочное и масштабное решение: производительность и стоимость нельзя оптимизировать без SLO, security и ответственного назначения системы.

29. Официальные источники

- NIST AI RMF: https://www.nist.gov/itl/ai-risk-management-framework
- NIST AI RMF Core: https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
- Model Card Toolkit repository (архивирован; источник структуры, не новая зависимость): https://github.com/tensorflow/model-card-toolkit
- GitHub secret security: https://docs.github.com/en/code-security/concepts/secret-security
- scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

Глава 14. Масштабирование, стоимость и обзор LLMOps

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР14 «Responsible ML», ЛР15 «Нагрузочный протокол и масштабирование»

1. Цель лекции

Научить студентов измерять производительность ML-сервиса, отличать симптом от bottleneck и принимать решение о масштабировании с учётом SLO, восстановления, стоимости и рисков. Показать, как знакомые MLOps-инварианты расширяются на LLMOps без обязательного вызова внешней модели.

2. Результаты обучения

Студент сможет:

- определить workload model и воспроизводимый нагрузочный протокол;
- интерпретировать throughput, error rate, p50, p95, max и saturation;
- объяснить warm-up, burst, soak и recovery scenario;
- обнаружить client bottleneck и coordinated omission;
- различать вертикальное и горизонтальное масштабирование;
- связать Kubernetes requests, limits, probes и HPA;
- оценить стоимость baseline и candidate capacity;
- перенести versioning, tracing, evaluation и security в LLMOps.

3. Место в сквозном проекте

ЛР11 добавила метрики сервиса, ЛР12 — качество модели, ЛР14 — ограничения ответственного выпуска. Теперь команда проверяет, выдерживает ли конкретная версия image и модели заявленный профиль и возвращается ли сервис в готовность после контролируемого нарушения.

4. Производительность — свойство сценария

Число «100 запросов в секунду» бессодержательно без payload, concurrency/arrival rate, среды, длительности, warm-up, timeout и критерия успеха. Сравнивать можно только прогоны с одинаковым протоколом или с явно объяснённым изменением одного фактора.

5. Workload model

Закрытая модель держит заданное число одновременных клиентов: новый запрос начинается после ответа. Открытая модель задаёт arrival rate независимо от ответа. Первая проще для учебной лаборатории, но может сама уменьшить нагрузку при замедлении сервиса. Этот эффект нужно назвать ограничением.

6. Профили нагрузки

- warm-up — загрузка модели, соединений и cache; его метрики не смешиваются со steady;
- steady — ожидаемая длительная нагрузка;
- burst — кратковременный пик;
- soak — длительный прогон для утечек и накопления очереди;
- recovery — восстановление readiness после сбоя или restart.

В двухчасовой ЛР обязательны steady, burst и короткий recovery drill; soak остаётся расширением.

7. Throughput и concurrency

Throughput — завершённые запросы за секунду. Concurrency — число запросов в работе. При стабильной системе закон Литтла связывает среднее число задач N, интенсивность λ и время W: $N = \lambda W$. Это проверка здравого смысла, а не замена измерению распределения latency.

8. Latency percentiles

p50 описывает типичный ответ, p95 — хвост для большинства пользователей, max помогает увидеть аномалию. Среднее скрывает тяжёлый хвост. В проекте используется nearest-rank: сортируем n значений и берём элемент с индексом `ceil(q*n)`. Метод фиксируется, чтобы разные инструменты не дали спор.

9. Error rate

Server errors, timeouts и connection failures относятся к capacity/reliability. Намеренные 4xx проверяют контракт клиента и анализируются отдельно. Если смешать 422 с 5xx, можно объявить здоровый сервис перегруженным.

10. Размер выборки и повторяемость

Двадцать запросов — только учебный минимум для механики. В реальном решении нужны повторные прогоны, доверительные интервалы и контроль фоновой нагрузки. Команда сохраняет сырые latency, а не только готовый p95, и повторяет сценарий не менее трёх раз.

11. Client bottleneck

Генератор может ограничивать throughput раньше сервиса: один поток, медленный DNS, лимит сокетов, CPU ноутбука или сериализация. Нагрузочный клиент и сервис наблюдаются отдельно. Рост latency клиента без saturation сервиса требует проверки генератора и сети.

12. Coordinated omission

В закрытой модели медленный ответ откладывает следующий запрос, поэтому худший период получает меньше измерений. Это coordinated omission. Для high-stakes теста используют arrival-rate генератор или корректируют методику; в ЛР ограничение явно записывается в отчёте.

13. Saturation и bottleneck

Наблюдаются CPU, memory, queue/backlog, connection pool, disk/network I/O, garbage collection и число worker. Высокий CPU с ростом p95 отличается от низкого CPU и медленного внешнего I/O. Решение «добавить pods» без локализации bottleneck может увеличить стоимость без эффекта.

14. Один фактор за эксперимент

Сначала фиксируют baseline. Затем меняют только workers, replica count, CPU request, batch size или другой фактор. Одновременная смена image, модели, payload и инфраструктуры делает вывод неидентифицируемым.

15. Recovery time

Recovery измеряется от контролируемого нарушения до устойчивой готовности, например трёх успешных readiness checks. Один ответ 200 может быть случайным. В отчёте фиксируются команда restart, timestamps, потерянные запросы и причина восстановления.

16. Readiness, liveness и startup probes

Readiness убирает pod из Service endpoints, liveness перезапускает зависший контейнер, startup защищает медленный старт от преждевременной liveness. Неверная liveness probe под нагрузкой может создать каскад перезапусков. Эти три сигнала нельзя считать взаимозаменяемыми.

17. Requests и limits

CPU/memory requests используются планировщиком и задают базу для CPU utilization HPA. Limits ограничивают использование: CPU может throttling, превышение memory limit — завершить контейнер. Поднимать limit без измерения не означает повысить гарантированную capacity.

18. Вертикальное и горизонтальное масштабирование

Вертикальное масштабирование увеличивает ресурсы экземпляра; горизонтальное — число replicas. Горизонтальный подход требует stateless API или внешнего состояния, корректной балансировки и достаточной downstream capacity. Модель, загруженная в память каждого pod, умножает memory cost.

19. Horizontal Pod Autoscaler

HPA `autoscaling/v2` меняет replicas по CPU, memory или custom/external metrics. CPU HPA не видит p95 напрямую. Если latency растёт при низком CPU из-за I/O, такой HPA может не помочь. Metrics pipeline, resource requests и readiness являются предпосылками корректного поведения.

20. Stabilization и flapping

Без стабилизации шумная метрика заставляет replicas колебаться. В демо быстрый scale-up ограничен политикой, а scale-down использует окно 300 секунд. Медленное уменьшение capacity снижает риск удалить pods сразу после короткого спада.

21. PodDisruptionBudget

PDB ограничивает добровольные disruptions, но не гарантирует доступность при crash, исчерпании узла или ошибке приложения. Он дополняет replicas, probes и rolling update. В демо `minAvailable: 1` соответствует учебной конфигурации, но production-значение выводится из SLO.

22. Kubernetes в курсе

Deployment, Service, HPA и PDB хранятся как manifests для архитектурного review. Локальный cluster, metrics-server и `kubectl apply` не являются условием допуска. Студент обязан объяснить поля и предпосылки, а не предоставлять скриншот чужого кластера.

23. Стоимость capacity

Минимальная оценка: replicas × цена replica-hour × часы. Полная стоимость включает registry, балансировщик, monitoring, storage, egress, idle headroom и инженерное сопровождение. Учебные условные единицы сравнивают варианты, но не выдаются за коммерческое предложение провайдера.

24. Decision record

Решение содержит baseline, SLO, evidence, bottleneck hypothesis, альтернативы, candidate capacity, ожидаемый эффект, стоимость, риски, rollback trigger и владельца. Допустимые действия: keep, optimize, scale, limit workload или redesign.

25. Почему autoscaling не заменяет capacity planning

HPA реагирует после наблюдения метрики и имеет задержку запуска pod. Он не создаёт мгновенную capacity, не чинит downstream и не знает бизнес-стоимость ошибки. Нужны headroom, admission control, timeout, retry budget и graceful degradation.

26. Переход к LLMOps

Генеративное приложение добавляет prompt, system instructions, provider/model, temperature, tools, retrieval corpus и conversation state. Эти элементы становятся версионируемыми зависимостями, подобно коду, данным и модели классического ML.

27. Tracing LLM-приложения

Trace связывает запрос, шаги retrieval/tool/model, output, latency, token usage и ошибки. Нельзя безусловно сохранять полный prompt/response: применяется минимизация, маскирование и retention. Trace ID помогает воспроизвести сбой, но не является лицензией на сбор чувствительного текста.

28. Eval dataset и scorers

Набор должен включать типовые и критические кейсы: формат, корректность, grounding, отказ, injection, утечка и tool misuse. Scorer может быть детерминированным, основанным на модели-судье или human review. Для judge фиксируются модель, prompt, шкала и обработка разногласий.

29. MLflow GenAI

Актуальный MLflow разделяет `mlflow.genai.evaluate()` для GenAI/traces и `mlflow.models.evaluate()` для классического ML. Tracing помогает видеть шаги, latency и token usage. В курсе это обзор: внешний API, ключ и платный вызов не требуются.

30. Latency и стоимость LLM

Измеряются time-to-first-token, полная latency, input/output tokens, tool calls, cache hit и стоимость на успешный кейс. Дешёвый ответ, не прошедший critical scorer, не является оптимизацией. Batch, caching и routing меняют одновременно качество, latency и риск устаревшего контекста.

31. Демонстрация преподавателя

1. Запустить capacity audit на baseline sample.
2. Увеличить две burst latency и получить `review`.
3. Сравнить throughput, p95 и CPU-график; обсудить bottleneck.
4. Показать probes, requests/limits, HPA stabilization и PDB.
5. Рассчитать стоимость двух и трёх replicas.
6. Открыть LLMOps eval cases и классифицировать deterministic/model/human scorer.

32. Вопросы аудитории

1. Почему рост p95 не всегда требует нового pod?
2. Чем concurrency отличается от arrival rate?
3. Как coordinated omission искажает результат?
4. Почему readiness не должна перезапускать контейнер?
5. Что необходимо CPU HPA для расчёта utilization?
6. Почему PDB не является гарантией availability?
7. Что кроме prompt нужно версионировать в LLMOps?
8. Когда model-as-judge требует human review?

33. Связь с ЛР15 и ЛР16

В ЛР15 команда строит профиль, recovery evidence и capacity decision. В ЛР16 эти результаты вместе с model/data governance, registry и observability войдут в итоговый release manifest и защиту.

34. Официальные источники

- Kubernetes HPA: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
- Kubernetes resources: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- Kubernetes probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/
- MLflow GenAI tracing: https://mlflow.org/docs/latest/genai/getting-started/tracing
- MLflow GenAI evaluation API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.genai.html

Глава 15. Интеграция и проверяемый выпуск ML-системы

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР15 «Нагрузочный протокол», ЛР16 «Интеграция и защита проекта»

1. Цель лекции

Научить студентов собирать результаты всего семестра в проверяемый выпуск: связать identities кода, данных, модели, API и контейнера, подтвердить gates, оформить исключения, независимое approval и воспроизводимый rollback. Итогом является не презентация, а доказуемое состояние системы.

2. Результаты обучения

Студент сможет:

- определить release identity и границы выпуска;
- построить manifest со ссылками и hashes evidence;
- отличить alias модели от точной model version;
- применить build once, promote the same artifact;
- разделить подготовку и одобрение выпуска;
- оформить временное исключение без скрытого обхода gate;
- проверить происхождение и целостность артефакта;
- выполнить и доказать rollback на точные предыдущие identities.

3. Интеграция не равна повтору ЛР

На финальной неделе команда не пересказывает 15 отчётов. Она доказывает, что артефакты образуют одну цепочку: commit создал pipeline run, run зарегистрировал exact model version, сервис использует эту версию и image, monitoring относится к тому же выпуску, а rollback возвращает известное состояние.

4. Release identity

Release identity включает version/tag, full source commit SHA, data/feature contract versions, MLflow run и exact model version, immutable image ref, OpenAPI version и environment. Если хотя бы один элемент обозначен «latest» или «текущий», состояние нельзя воспроизвести.

5. Release manifest

Manifest — машинно-читаемый индекс выпуска. Он не копирует все отчёты, а связывает identities, gates, SHA-256 evidence, approvals, ограничения и rollback. Manifest тоже является артефактом и фиксируется release tag или hash.

6. Evidence graph

Удобно читать manifest как граф: source commit → CI run → image; data contract → prepared dataset → MLflow run → model version; model/API/image → monitoring; risk/capacity gates → release decision. Несвязанная карточка или dashboard не является доказательством именно этого выпуска.

7. Код

Используется полный 40-символьный commit SHA. Ветка и PR важны для процесса review, но могут перемещаться. Release tag удобен для человека, однако manifest хранит и tag, и commit, чтобы обнаружить перемещение или ошибочную сборку.

8. Данные и признаки

Фиксируются data contract version, identity исходного/подготовленного набора, временные границы и feature contract. Hash без provenance подтверждает байты, но не право и смысл данных; data card и license evidence остаются частью выпуска.

9. MLflow run и model version

Run ID связывает параметры, метрики и артефакты обучения. Registry model version неизменяема как конкретный объект; alias `champion` — управляемый указатель. Manifest хранит оба и resolved URI `models:/name/version`, чтобы последующее перемещение alias не переписало историю выпуска.

10. Container identity

Плавающий `latest` запрещён. Надёжный вариант — registry digest; учебный минимум — tag, равный full commit SHA при включённой политике неизменяемости. SemVer удобен пользователю, но без immutability может быть перезаписан и потому дополняется digest или commit identity.

11. API identity

Версия OpenAPI и contract tests должны соответствовать запущенному image. Совместимость описывается явно: backward-compatible или breaking. Изменение схемы без version/review является release risk, даже если модель не изменилась.

12. Build once, promote the same artifact

Нельзя заново собирать image для production после теста candidate: новая сборка может получить иные зависимости. Один artifact проходит gates и продвигается между средами. Конфигурация среды отделена от содержимого image и также контролируется.

13. Release gates

Gate имеет имя, policy version, вход, статус и evidence. В курсе обязательны data contract, point-in-time features, monitoring decision, responsible AI, capacity, model card и security. Команда может добавить, но не молча удалить обязательный gate.

14. Gate pass и доказательство

Статус `pass` без пути, CI run или hash не проверяем. Manifest builder читает файл и вычисляет SHA-256. Если evidence изменилось после решения, `verify` завершается ошибкой. Это защищает от «тихого исправления» отчёта после approval.

15. Управляемое исключение

Не каждый alert означает немедленный stop. Например, drift при стабильном отложенном качестве может быть принят на короткий срок. Статус `accepted_with_investigation` требует причину, владельца, срок пересмотра и mitigation. Бессрочное исключение превращается в скрытое отключение gate.

16. Draft и ready

Учебный example остаётся `draft`, потому что не имеет реального CI и независимого approval. Статус `ready` возникает только при успешном CI и разных prepared/approved identities. Наличие исключения даёт `ready_with_exception`, а не маскируется как обычный pass.

17. Separation of duties

Автор изменения не должен единолично одобрять выпуск. Для маленькой команды reviewer может быть другим участником, преподавателем или назначенным risk owner. Важно не название роли, а независимая проверка критерия и возможность остановить release.

18. CI и CD

CI доказывает качество конкретного commit: lint, tests, contracts, model/data/security gates. CD продвигает уже проверенный artifact с environment protection и approval. Зелёный CI не означает, что production deployment автоматически разрешён.

19. GitHub environment protection

Environment может требовать reviewer, ограничивать ветки и выдавать secrets только deployment job. В учебном проекте фактический внешний deploy не обязателен, но students должны показать, где была бы граница approval и почему секрет не нужен pull-request job.

20. Artifact provenance и attestation

Attestation криптографически связывает artifact с workflow, repository, commit и событием. Она усиливает provenance, но не доказывает безопасность, качество модели или корректность policy. Польза возникает только при проверке attestation потребителем.

21. Минимальные permissions

Workflow объявляет минимальные `permissions`. Право `contents: read` достаточно большинству quality jobs; публикация package, OIDC или attestation получает write только в нужном job. Full-SHA pinning или lockfile для actions снижает риск подмены зависимости workflow.

22. Rollback как отдельный release

Rollback — управляемое изменение состояния с причиной, approver и evidence. Он возвращает exact previous model version и immutable image, затем выполняет smoke/readiness checks. «Поставить старое» без identities не является процедурой.

23. Model и application rollback

Иногда достаточно переместить alias `champion`, иногда требуется вернуть image/API. Если новая версия API несовместима со старой моделью, частичный rollback опасен. Manifest фиксирует совместимую пару model/image и порядок действий.

24. Roll forward

Небольшая безопасная ошибка может исправляться новым release быстрее rollback. Выбор зависит от времени, риска несовместимости, данных и доступной предыдущей версии. Решение и критерий записываются, а не принимаются только по интуиции во время инцидента.

25. Post-release verification

После deploy/rollback проверяются resolved identities, readiness, smoke prediction, error rate и ключевые SLO. Успех команды deployment ещё не означает, что сервис использует нужную модель и отвечает корректно.

26. Audit trail

Issue формулирует критерий; PR содержит diff и review; CI — результаты; manifest — identities; approval — решение; monitoring — последствия; rollback/postmortem — реакцию. Удаление первой неуспешной попытки разрушает обучение и аудит.

27. Сценарий итоговой демонстрации

За 10–12 минут команда показывает manifest, воспроизводимость данных/признаков, exact model, контейнер/API, observability, governance/capacity decision и rollback. Каждая операция имеет заранее определённый ожидаемый результат и запасной безопасный путь.

28. Индивидуальная защита

Общая демонстрация не подтверждает понимание каждого. Студент за 3–4 минуты показывает личный артефакт, прослеживает его связи и выполняет изменение/диагностику: проверить hash, resolved alias, сломать gate в копии, объяснить alert или провести smoke после rollback.

29. Оценка вклада

Количество commit и объём речи не являются метриками вклада. Принимаются код, тест, contract, manifest evidence, содержательный review, runbook, security/capacity decision и диагностика, если результат встроен в общую версию и воспроизводим.

30. Типичные анти-паттерны

- tag без commit SHA;
- alias без resolved version;
- `latest` или пересобранный production image;
- pass без evidence/hash;
- исключение без owner/expiry;

- author равен approver;
- rollback только на словах;
- dashboard из другого периода;
- общая презентация вместо индивидуальной защиты.

31. Демонстрация преподавателя

1. Собрать draft manifest из example spec.
2. Проверить hashes командой `release verify`.
3. Изменить копию evidence и показать отказ.
4. Заменить image на `latest` и показать отказ.
5. Добавить CI success/independent approval и получить `ready_with_exception`.
6. Проследить exception drift до owner и срока.
7. Сопоставить previous model/image с rollback steps.

32. Вопросы аудитории

1. Почему alias недостаточен для исторического manifest?
2. Чем digest отличается от SemVer tag?
3. Почему attestation не доказывает безопасность?
4. Когда допустим `ready_with_exception`?
5. Зачем хешировать evidence?
6. Почему partial rollback может быть опасен?
7. Что проверяется после deployment?
8. Как отличить личный вклад от общей демонстрации?

33. Связь с ЛР16 и Лекцией 16

ЛР16 реализует полный release review и защиту. Лекция 16 завершит курс архитектурным разбором, типичными отказами, ретроспективой инженерных решений и подготовкой к экзаменационному контролю.

34. Официальные источники

- GitHub artifact attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations
- GitHub deployment environments: https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments
- GitHub Actions security: https://docs.github.com/en/actions/how-tos/secure-your-work
- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/

Глава 16. Архитектурный разбор, отказы и ретроспектива MLOps-системы

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанная работа: ЛР16 «Интеграция, release manifest и защита проекта» Формат: итоговая лекция, разбор кейса и подготовка к экзаменационному контролю

1. Цель лекции

Собрать знания курса в целостную модель эксплуатации ML-системы: научить студентов читать архитектуру через риски и доказательства, локализовать отказ, выбирать rollback или roll forward, проводить содержательный postmortem и обосновывать дальнейшее развитие системы.

2. Результаты обучения

После лекции студент сможет:

- описать полный контур от данных до решения пользователя;
- выделить архитектурные инварианты и владельцев компонентов;
- классифицировать отказ по слою, влиянию и обнаружению;
- отличить симптом, непосредственную причину и системные факторы;
- предложить безопасный порядок диагностики и восстановления;
- связать incident evidence с release manifest и наблюдаемостью;
- оформить blameless postmortem с проверяемыми action items;
- оценить технический долг и приоритет следующего улучшения;
- защитить инженерное решение, альтернативу и остаточный риск.

3. Финальная лекция — не каталог инструментов

За семестр были использованы GitHub, DVC, MLflow, pytest, Ruff, FastAPI, Docker, Prefect, Prometheus и Grafana. Знание команд этих инструментов полезно, но итоговая компетенция выше: студент должен понимать, какое утверждение доказывает каждый артефакт и что произойдёт при его отказе. Инструмент можно заменить; контракт, identity, gate и обратная связь должны сохраниться.

4. Сквозной объект управления

ML-система принимает не только HTTP-запрос. Она принимает данные, код, зависимости, конфигурацию, решения людей и время. Выходом является не только prediction, а действие пользователя или бизнес-процесса. Поэтому граница анализа проходит от provenance данных до наблюдаемого последствия решения.

5. Архитектурные инварианты курса

Инвариант — утверждение, которое должно оставаться истинным при изменении реализации:

- данные и признаки доступны к моменту прогноза;
- любой результат связан с версией кода, данных и параметров;
- модель выпускается только после объявленных gates;
- API проверяет контракт до вызова модели;
- image и model version идентифицируются неизменяемо;
- сервис сообщает readiness и ключевые SLI;
- решение о выпуске и исключения имеют владельца;
- восстановление возвращает совместимое проверенное состояние;
- личный вклад и review прослеживаются по evidence.

Если команда не может быстро показать доказательство инварианта, инвариант считается неподтверждённым, даже когда демонстрация один раз завершилась успешно.

6. Три связанных контура управления

Первый контур — software delivery: commit → CI → image → deployment → telemetry. Второй — model lifecycle: data → features → training run → evaluation → registry → serving → delayed quality. Третий — governance: назначение системы → риски → policy → evidence → approval → review. Ошибка часто возникает на стыке контуров: новый image запущен, но alias указывает на другую модель; качество модели приемлемо, но данные собраны с нарушением временной доступности.

7. Граф доказательств

Итоговый release manifest является индексом графа доказательств. Узлы — identities и отчёты; рёбра — проверяемые связи: этот commit породил этот image, этот run использовал эту версию данных, этот deployment обслуживает эту model version, этот dashboard относится к этому окну. Разрыв связи опаснее отсутствия красивого отчёта, потому что делает вывод непроверяемым.

8. Классификация отказов

Для первичной диагностики удобно использовать четыре измерения:

1. слой: data, feature, model, orchestration, API, infrastructure, observability или governance;
2. время: до выпуска, при deployment, сразу после запуска или после накопления drift;
3. влияние: недоступность, неверные ответы, деградация качества, утечка данных или потеря аудита;
4. обнаружение: test, gate, alert, пользователь, аудит или случайная проверка.

Такая запись лучше фразы «сломалась модель»: она задаёт следующий проверочный шаг.

9. Отказ данных

Примеры: изменилась схема, дубли выросли, временная зона стала иной, source задержался, лицензия или право использования не подтверждены. Проверяются data contract, freshness, объём, ключи, временные границы и data card. Автоматическое обучение на неполной выборке запрещается, если policy не описывает допустимую деградацию и способ её обнаружения.

10. Отказ признаков

Наиболее опасен тихий training-serving skew: pipeline обучения и API вычисляют похожий признак по разным правилам. Второй класс — temporal leakage, когда offline join использует сведения из будущего. Доказательства: единая функция или контракт вычисления, point-in-time test, timestamp available_at, TTL и сравнение распределений offline/online.

11. Отказ эксперимента и оценки

Высокая метрика не означает корректный эксперимент. Проверяются baseline, split, seed, набор метрик, uncertainty, групповые срезы и воспроизводимость run. Если candidate победил только на одном случайном разбиении или за счёт недоступного признака, проблема возникла до Registry, хотя проявится после выпуска.

12. Отказ Registry и release identity

Alias `champion` является управляемым указателем, а не исторической identity. Диагностика начинается с resolved exact version, run ID, tags и release manifest. Если manifest хранит только alias, нельзя доказать, какая модель обслуживала запрос в момент инцидента. Аналогично image tag `latest` не даёт воспроизводимой связи с commit.

13. Отказ API и контракта

Сервис может быть доступен, но неверно интерпретировать единицы измерения, обязательные поля или категории. Проверяются OpenAPI version, Pydantic validation, отрицательные contract tests, request ID, совместимость model/image и отсутствие raw features в логах. HTTP 200 не является достаточным критерием корректности prediction.

14. Отказ оркестрации

Типичные причины: неидемпотентный task, неправильный retry, частично записанный артефакт, потеря параметров запуска и передача большого объекта через внутреннее состояние оркестратора. Безопасный pipeline пишет внешние versioned artifacts, различает retryable и terminal error и не продвигает модель после неполного run.

15. Отказ инфраструктуры и capacity

Рост latency может быть вызван CPU throttling, памятью, очередью, зависимостью или размером batch, а не моделью как таковой. Нужны percentile latency, error rate, saturation, requests/limits, steady и burst profiles. Масштабирование не исправляет неправильный алгоритм и иногда лишь увеличивает стоимость ошибочных запросов.

16. Отказ наблюдаемости

«Нет alert» не означает «нет проблемы». Dashboard может показывать неправильный период, label может создавать неконтролируемую cardinality, метрика качества может не иметь coverage, а лог — терять request ID. Наблюдаемость также тестируется: известный synthetic request должен появиться в метрике, логе и, при наличии, trace с согласованной identity выпуска.

17. Отказ безопасности и цепочки поставки

Проверяются секреты, permissions workflow, версии зависимостей, источник image и сторонних Actions. Full-length commit SHA делает ссылку на Action неизменяемой, но команда всё равно должна проверить происхождение и назначение кода. Attestation усиливает provenance артефакта, но не доказывает качество модели, отсутствие уязвимостей или корректность бизнес-решения.

18. Отказ governance

Governance failure возникает, когда технически успешный выпуск не имеет владельца риска, срока исключения, основания использования данных или процедуры вывода системы из эксплуатации. Рамка NIST AI RMF полезна как цикл `Govern → Map → Measure → Manage`: контекст риска задаётся до измерения, а результат измерения должен вести к управляемому решению, а не оставаться отдельным отчётом.

19. Человеческие и организационные факторы

Ошибку редко объясняет один человек. Системные факторы включают неясные роли, противоречивые приоритеты, отсутствие времени на review, перегруженный runbook, недоступный staging и стимул скрыть неуспешную попытку. Blameless подход не отменяет ответственности: он переносит внимание с поиска виновного на изменение условий, при которых ошибка стала возможной и незаметной.

20. Каскадный отказ: итоговый кейс

Рассмотрим цепочку: источник опоздал; freshness gate был вручную принят без срока; training job создал модель на неполных данных; alias перемещён; image с новым API пересобран отдельно; alert по coverage отсутствует; rollback вернул только alias и оставил несовместимый API. Каждый компонент «работал», но система нарушила несколько связей и усилила исходную проблему.

21. От симптома к гипотезе

Нельзя сразу менять модель по первому сигналу. Зафиксируйте время и влияние, затем identities deployment, image, model и data window. Сформулируйте проверяемые гипотезы: «latency выросла после image X», «доля неизвестных категорий выросла в source Y», «alias сменился без compatible API». Для каждой гипотезы укажите evidence, способ опровержения и безопасный следующий шаг.

22. Triage и приоритет

Первая цель triage — ограничить влияние, а не найти идеальную root cause. Приоритет определяется масштабом воздействия, обратимостью, безопасностью данных и скоростью ухудшения. Команда назначает incident lead, технических исполнителей и канал фиксации решений. Одновременные несогласованные изменения усложняют диагностику и должны быть остановлены.

23. Containment

Возможные действия: остановить продвижение, отключить проблемный source, переключить на безопасный baseline, ограничить трафик, вернуть предыдущую совместимую пару model/image, перевести результат в режим рекомендации или временно прекратить prediction. Containment выбирается так, чтобы не стереть evidence и не создать более опасное состояние.

24. Rollback или roll forward

Rollback предпочтителен, когда предыдущая версия известна, совместима и её возврат быстрее анализа. Roll forward разумен для локальной безопасной ошибки с коротким исправлением и высоким риском обратной миграции. Решение содержит trigger, from/to identities, approver, ожидаемый результат и post-change checks. Оба варианта являются новым управляемым изменением, а не отменой истории.

25. Проверка восстановления

Завершение команды deployment не закрывает incident. Проверяются readiness, smoke prediction, resolved model version, error rate, latency, ключевой бизнес-инвариант и отсутствие дальнейшего повреждения данных. Время восстановления фиксируется по достижению проверенного состояния, а не по моменту нажатия кнопки rollback.

26. Postmortem как инженерный артефакт

Postmortem содержит impact, timeline, detection, response, contributing factors, что сработало, что не сработало и action items. Он не переписывает историю и не маскирует неопределённость. Практика Google SRE рекомендует blameless анализ: корректируются технология, процесс и обучение, а не формулируется действие «быть внимательнее».

27. Непосредственная причина и системные факторы

Фраза «неверный config» описывает событие, но не объясняет, почему config прошёл review, gate и deployment. Полезные вопросы: какое допущение оказалось неверным; какой контроль отсутствовал; почему сигнал не был замечен; что усилило влияние; что помогло восстановлению. Root cause может быть несколько, а линейная история часто скрывает взаимодействие факторов.

28. Action items

Хороший action item имеет владельца, срок, приоритет и проверяемый критерий закрытия. Он либо предотвращает класс отказов, либо ускоряет обнаружение, либо уменьшает impact, либо улучшает восстановление. «Добавить тест» уточняется входом, ожидаемым отказом и местом gate. Просроченный пункт возвращается в risk register, а не исчезает из документа.

29. Технический долг и risk register

Технический долг — не список всего несовершенного. Запись содержит решение или ограничение, последствие, вероятность, blast radius, сигнал наступления, mitigation, owner и review date. Приоритет получают элементы, которые нарушают инвариант, затрудняют восстановление или повторяются в инцидентах. Косметический рефакторинг не должен вытеснять проверяемый safety gap.

30. Метрики развития системы

Одна accuracy и число deployment не описывают здоровье MLOps. Набор включает delivery throughput и stability, SLO сервиса, freshness/coverage данных, delayed model quality, долю успешных gates, время обнаружения и проверенного восстановления, повторяемость инцидентов и закрытие action items. Метрики используются совместно: ускорение выпуска при росте неуспешных изменений не является зрелостью.

31. Уровни зрелости учебного проекта

Уровень	Наблюдаемое состояние
0. Ручной	Результат существует только на машине автора, identities отсутствуют
1. Воспроизводимый	Среда, данные и обучение можно повторить по инструкции
2. Проверяемый	Tests, contracts, CI и evidence подтверждают качество конкретного commit
3. Управляемый	Registry, release gates, observability, approvals и rollback связаны
4. Обучающийся	Incidents, drift и postmortems меняют policy, backlog и архитектуру

Уровень не присваивается по наличию инструмента. Он подтверждается воспроизводимой демонстрацией и тем, что студент способен объяснить ограничения контроля.

32. Архитектурный review

Review начинается не со схемы компонентов, а с цели, пользователей и unacceptable outcomes. Затем команда показывает границы доверия, data flow, identities, gates, observability и recovery. Для каждого критического решения отвечают:

1. какое требование оно обеспечивает;
2. какая альтернатива рассматривалась;
3. какое evidence подтверждает выбор;
4. какой failure mode остаётся;
5. как обнаружить и безопасно восстановить систему.

33. Выбор следующего улучшения

После защиты не нужно одновременно внедрять Kubernetes, feature store и новый registry. Выберите один bottleneck с доказанным влиянием. Если большая часть времени уходит на восстановление среды, улучшайте reproducibility; если инциденты поздно обнаруживаются — telemetry; если release нельзя связать с моделью — identities и manifest; если риск неизвестен — сначала measurement, а не новый инструмент.

34. Подготовка к экзаменационному контролю

Ответ строится как инженерное рассуждение, а не перечисление определений:

1. определить границу и цель системы;
2. назвать входы, выходы и identities;
3. сформулировать инвариант и риск;
4. указать preventive и detective controls;
5. назвать evidence и owner решения;
6. описать отказ, containment и recovery;
7. объяснить ограничение и следующий шаг.

Если вопрос относится к конкретному инструменту, сначала объясняется функция в системе, затем реализация этим инструментом.

35. Пример экзаменационного кейса

После выпуска error rate не изменился, но через две недели снизилась delayed precision для одного направления аэропорта. Coverage меток — 55 %, PSI вырос, latency стабильна. Студент должен отделить факт от гипотезы, проверить сегмент и coverage, сопоставить data window и release identity, выбрать между наблюдением, retraining и rollback, задать gate и назвать остаточный риск. Автоматический retraining только по PSI не является достаточным ответом.

36. Индивидуальная итоговая защита

Каждый участник связывает личный артефакт с архитектурным инвариантом и release tag команды. За 3–4 минуты студент показывает evidence, выполняет диагностическую операцию и отвечает на вопрос по соседнему компоненту. Общая презентация не заменяет личную часть; количество commit не заменяет содержательный diff, review, тест, runbook или решение о риске.

37. Ретроспектива команды

Команда отвечает на четыре вопроса:

- какой контроль дал наибольшую уверенность и чем это доказано;
- где было больше всего ручной работы или неопределённости;
- какой отказ проект обнаруживает поздно или не обнаруживает;
- какое одно улучшение команда реализовала бы следующим и по какому критерию проверила эффект.

Ретроспектива не влияет на уже зафиксированную историю выпуска, но создаёт backlog для следующей версии и показывает зрелость инженерного мышления.

38. Сценарий занятия для преподавателя

1. 0–10 минут: восстановить три контура управления и архитектурные инварианты.
2. 10–25 минут: разобрать failure taxonomy на примерах студентов.
3. 25–45 минут: провести каскадный incident от delayed source до несовместимого rollback.
4. 45–60 минут: группы формируют triage, containment и recovery с evidence.
5. 60–75 минут: превратить timeline в blameless postmortem и action items.
6. 75–85 минут: разобрать структуру экзаменационного ответа и индивидуальной защиты.
7. 85–90 минут: exit ticket и фиксация следующего улучшения проекта.

39. Демонстрация преподавателя

Возьмите manifest ЛР16 и подготовьте копию incident-пакета. Покажите:

1. исходный alert и границу известного;
2. поиск source commit, image и exact model version;
3. гипотезу, которую telemetry опровергает;
4. решение rollback/roll forward с критерием;
5. post-change smoke и проверку model identity;
6. короткий postmortem с двумя action items;
7. перенос незакрытого риска в backlog с owner и review date.

Не меняйте production-like учебные артефакты во время демонстрации: используйте копию spec/evidence и сохраните исходный release неизменным.

40. Задание аудитории

Команды получают разные симптомы: рост p95, падение coverage, несовпадение alias, schema error или утечку признака. За семь минут они заполняют таблицу:

Поле	Ответ команды
Наблюдаемый симптом	Что известно без предположений
Impact	Пользователи, решения, данные, период
Identities	Commit, data, run, model, image, environment
Две гипотезы	Какую evidence запросить и чем опровергнуть
Containment	Как ограничить влияние без потери evidence
Recovery	Rollback или roll forward и post-check

Поле	Ответ команды
Action item	Owner, срок и критерий закрытия

Затем другой участник команды выполняет review: ищет необоснованный вывод, непроверяемую identity или опасное действие.

41. Вопросы аудитории

1. Почему зелёный CI не доказывает корректность production decision?
2. Чем симптом отличается от contributing factor?
3. Почему rollback может усилить incident?
4. Как проверить, что dashboard относится к нужному release и окну?
5. Когда отсутствие alert является отказом наблюдаемости?
6. Почему blameless postmortem не отменяет ответственность?
7. Как отличить полезный action item от формальной записи?
8. Какие риски нельзя закрыть автоматическим gate?
9. Как выбрать между новым инструментом и улучшением текущего контроля?
10. Как доказать достижение четвёртого уровня зрелости?

42. Exit ticket

Каждый студент индивидуально сдаёт три коротких утверждения:

1. самый опасный неподтверждённый инвариант проекта;
2. evidence, которое могло бы его подтвердить или опровергнуть;
3. одно улучшение с owner, сроком и измеримым критерием.

Совпадающие формулировки без связи с личным артефактом не принимаются. Exit ticket используется как последний вход преподавателя при выборе индивидуального вопроса на защите.

43. Итог курса

MLOps — это дисциплина управления изменениями ML-системы. Зрелая команда не обещает отсутствие ошибок; она делает состояние воспроизводимым, риски видимыми, решения проверяемыми, выпуск обратимым, а инциденты — источником улучшения. Итоговый критерий курса: студент способен не только запустить pipeline, но и обосновать, почему системе можно доверять в известных границах.

44. Официальные источники

- Google SRE, Postmortem Culture: https://sre.google/resources/book-update/postmortem-culture/
- Google SRE, Incident Management Guide: https://sre.google/resources/practices-and-processes/incident-management-guide/
- NIST AI RMF Core: https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
- NIST AI RMF Playbook: https://airc.nist.gov/airmf-resources/playbook/
- GitHub Actions Secure Use Reference: https://docs.github.com/en/actions/reference/security/secure-use
- GitHub Artifact Attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations

Дополнение А. Навигатор демонстраций

Глава	Демонстрация	Наблюдаемый результат
1	Воспроизводимый запуск	Чистое окружение → установка → тест → фиксация commit
2	Контракт и утечка	Валидная выборка и намеренно нарушенные schema/time constraints
3	Восстановление данных	Два Git/DVC-состояния и возврат к точной версии
4	Сравнение экспериментов	Два MLflow run с одинаковым split и различимыми параметрами
5	Негативные тесты	Поломка schema/metric/pipeline и объяснение сообщения об ошибке
6	Контейнер	Сборка non-root image, smoke и проверка воспроизводимости
7	Контракт API	Valid/invalid request, request ID, health/readiness/metrics
8	Quality gate	Зелёный и красный GitHub Actions run с diagnostic artifact
9	Оркестрация	Успешный flow, retry и terminal failure без ложного promotion
10	Registry	candidate/champion, exact version, manifest и rollback
11	Наблюдаемость	Нагрузка → метрики → alert → runbook
12	Drift	Reference/current window, delayed labels, coverage и управляемое решение
13	Point-in-time join	Наивный join против корректного available-time join
14	Governance	Model/data cards, grouped metrics, threat model и release gate
15	Capacity	Steady/burst/recovery и решение о масштабировании
16	Интеграция	Release manifest, подмена evidence, rollback и индивидуальная защита

Дополнение Б. Демонстрационные и отказные сценарии из практических работ

ЛР2. Контракт данных и предотвращение утечки

11 Шаг 5 Негативные сценарии

Положительный тест доказывает только то, что текущий пример проходит. Для приемки создайте во временном каталоге не менее двух испорченных вариантов:

1. значение `DepHour=25`, нарушающее верхнюю границу;
2. отсутствие `ArrDelay` или недопустимое значение `IsWeekend=2`.

Не изменяйте эталонный `flights_sample.csv` ради демонстрации. Тест должен построить временный файл, вызвать валидатор и проверить, что нужное правило стало неуспешным. Дополнительно рекомендуются сценарии с пропуском, дробным значением целочисленного поля, неизвестным столбцом, полным дубликатом и недостаточным числом строк.

Запустите проверки:

```
python -m pytest -q tests/test_data_contract.py
python -m ruff check src tests
```

В CI валидатор должен выполняться до обучения модели. Если данные не соответствуют контракту, дальнейшие этапы не запускаются.

15.2 Демонстрация на защите

Каждый студент без подсказки другого участника должен:

1. получить актуальный tag или commit команды;
2. запустить валидатор на корректном образце;
3. назвать версию контракта и показать SHA-256 в отчете;
4. создать во временной копии одно нарушение и получить ненулевой код завершения;
5. объяснить собственный вклад по diff;
6. объяснить, почему выбранный преподавателем признак доступен или недоступен за 60 минут до отправления.

Преподаватель вправе выбрать другой столбец или нарушение, чем указано в карточке. Демонстрация проводится на собственном окружении студента или в чистом клоне по указанию преподавателя.

ЛР3. Версионирование данных и воспроизводимые вычисления

12 Отказные сценарии

Проверьте минимум два сценария:

1. Рабочий CSV изменён вручную без `dvc add`: `dvc status` показывает несогласованность.
2. Восстановленная версия нарушает контракт или порядок дат: валидатор возвращает ненулевой код.

Дополнительно можно временно сделать remote недоступным и описать понятное сообщение DVC. После эксперимента восстановите конфигурацию.

ЛР6. Контейнеризация и управление средой

15 Негативные сценарии

Команда воспроизводит минимум два отказа:

Дефект	Ожидаемый сигнал	Диагностика
модель не перенесена в runtime	`/readyz` возвращает 503, health unhealthy	проверить `MODEL_PATH` и COPY stage
процесс запущен на другом порту	health timeout	сопоставить CMD, EXPOSE и health URL
запрещена запись вне `/tmp`	операция завершается permission denied	проверить ожидаемый путь записи
порт хоста занят	Compose не запускает binding	изменить `APP_PORT`, не порт контейнера

Дефект	Ожидаемый сигнал	Диагностика
`.env` попал в context	нарушение security review	исправить `.dockerignore`, сменить секрет

Defect injection выполняется в отдельной ветке или незакоммиченной копии и не входит в tag `lr6-v1`.

ЛР7. Сервис предсказаний и контракт API

16 Негативные сценарии

Каждый участник воспроизводит один контролируемый отказ:

- разрешить неизвестные поля;
- убрать strict mode и принять строку `"9"`;
- удалить response model;
- перестать возвращать request ID в header;
- сделать `/readyz` успешным без модели;
- убрать документированный response 503 из OpenAPI.

До изменения студент называет тест, который должен упасть. Дефект не входит в tag `lr7-v1`.

ЛР11. Наблюдаемость ML-сервиса

19. Контролируемые сценарии

Допустимые индивидуальные сценарии:

Сценарий	Ожидаемый сигнал	Восстановление
остановить только `app`	`up=0`, Down pending/firing	запустить app, дождаться resolved
временный тестовый 500 endpoint в отдельной ветке	5xx ratio	отменить commit/ветку
контролируемая задержка в отдельной ветке	p95 alert	убрать задержку
неверный model path	`/readyz=503`, not-ready outcomes	вернуть путь и рестарт
Prometheus не видит target	scrape failure	исправить config/network

Нельзя удалять чужие volumes, портить общий dashboard, бесконечно генерировать нагрузку или публиковать сервис наружу.

ЛР12. Drift и отложенное качество модели

18. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
удалить зрелые labels	coverage падает, решение `hold`
дублировать `request_id`	явная ошибка one-to-one контракта
сдвинуть `as_of`	изменяется mature cohort
добавить недопустимый `DepHour`	data quality incident до drift
оставить один класс	ROC AUC не рассчитывается
изменить только current distribution	`investigate`, а не auto retraining

Работайте в отдельной ветке или копии входных данных. Общий baseline не портится.

ЛР13. Point-in-time признаки и Feature Store

20. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
убрать фильтр feature time	audit обнаруживает будущее

Сценарий	Ожидаемый результат
убрать available time	fixture в 08:00 выбирается неверно
уменьшить TTL	coverage падает
дублировать request ID	явный отказ контракта
изменить entity key	появляются пропуски или неверный join
использовать online latest для training	расхождение с historical output

Сценарий выполняется в отдельной ветке или копии данных.

ЛР16. Интеграция, release manifest и защита проекта

26. Сценарий командной демонстрации

Используйте `docs/final-demo-script.md`. Продолжительность 10–12 минут. Назначьте ведущего каждого шага и запасной вариант на случай сетевой недоступности. Запись экрана не заменяет live explanation.

31. Набор диагностических заданий преподавателя

- заменить image ref на `latest` в копии spec и объяснить отказ;
- изменить evidence и запустить verify;
- показать exact version за alias;
- определить, почему manifest остаётся draft;
- найти owner/expiry исключения;
- сопоставить API version с image;
- выбрать rollback или roll-forward;
- найти поле, которое нельзя логировать;
- объяснить, почему attestation не доказывает качество модели.